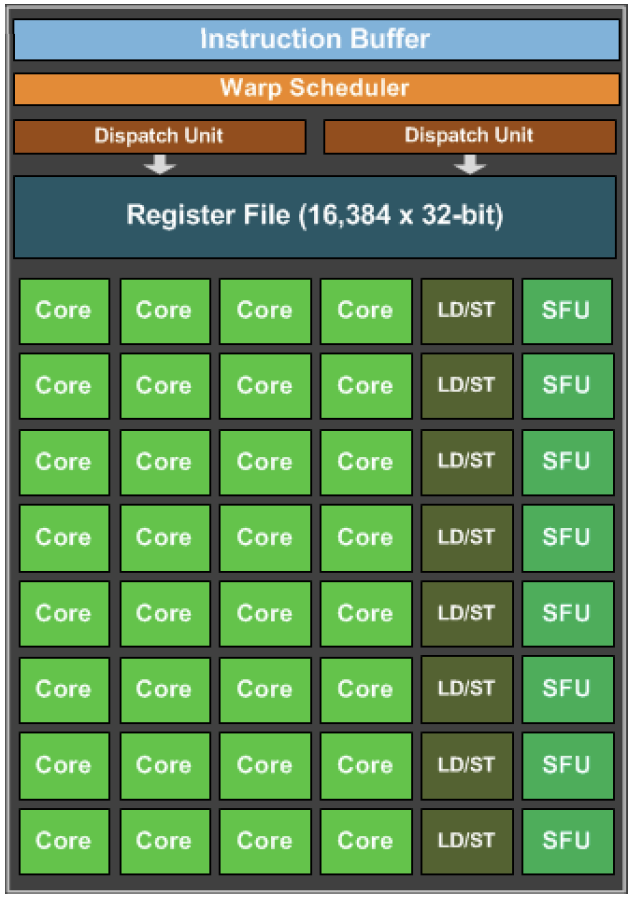
CSE113: Parallel Programming

- **Topics**:
  - Scheduling
  - Intro to GPUs



Announcements

- HW#2 graded
- Midterm being graded. By Thursday.

- Tips to improve understanding going into the final:
  - Attend class, study the slides
  - Do readings
  - Experiment with homework
  - Focus on understanding and problem solving

Announcements

- HW3 due on Thursday (+ 3 days if you need it)

Announcements

- Module 4: GPU for three sessions

- Module 5: advanced topics – Concurrent sets, barriers and memory models

Schedule

- DOALL Loops

- **Parallel Schedules:**
  - Static
  - **Global Worklists**
  - Local Worklists

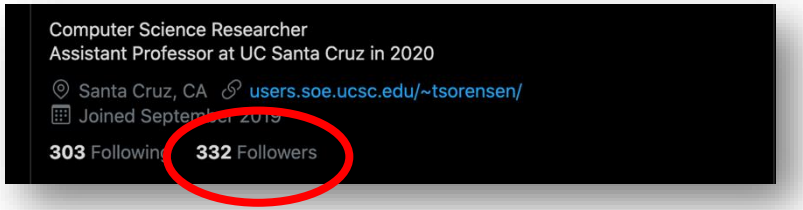
Irregular parallelism in loops

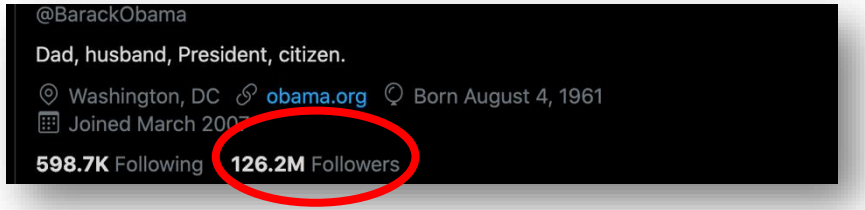
- Tasks are not balanced
- Appears in lots of emerging workloads

Irregular parallelism in loops

- Tasks are not balanced
- Appears in lots of emerging workloads

  social network analytics where threads are parallel across users





Irregular parallelism in loops

- Independent iterations have different amount of work to compute
- Threads with longer tasks take longer to compute.
- Threads with shorter tasks are under utilized.

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

irregular (or unbalanced) parallelism: each x iteration performs different amount of work.

Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

Calculate work done by second thread:

$$\text{t2_work} = \sum_{n=0}^{SIZE/2} n$$

Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

Calculate work done by second thread:

$$\text{t2_work} = \sum_{n=0}^{SIZE/2} n$$

Calculate work work done by first thread:

t1_work = total_work − t2_work

Irregular parallelism in loops

Example: SIZE = 64

total_work = 2016
t2_work = 496
t1_work = 1520

t1 does ~3x more work than t2

Only provides ~1.3x speedup

Potential solution:
Have T1 do only ¼ of the iterations
Gives a better speedup of 1.77x

Not a feasible solution because often times load imbalance is not given by a static equation on loop bounds!

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

Calculate work done by first thread:

$$\text{t1_work} = \sum_{n=0}^{SIZE/2} n$$

Calculate work work done by second thread:

t2_work = total_work − t1_work

Work stealing

- Tasks are dynamically assigned to threads.

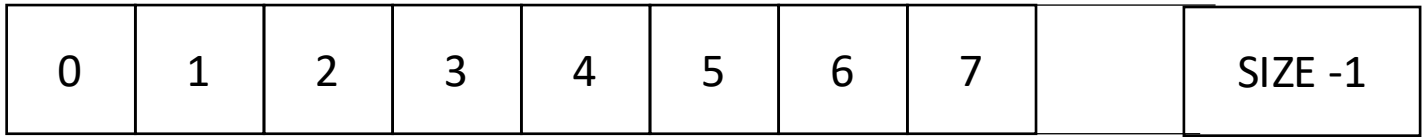
Work stealing - global worklist

- Pros
  - Simple to implement

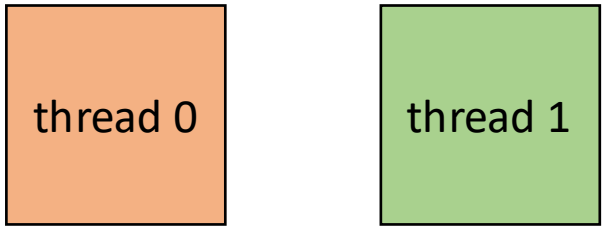
- Cons:
  - High contention on global counter
  - Potentially bad memory locality.

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically

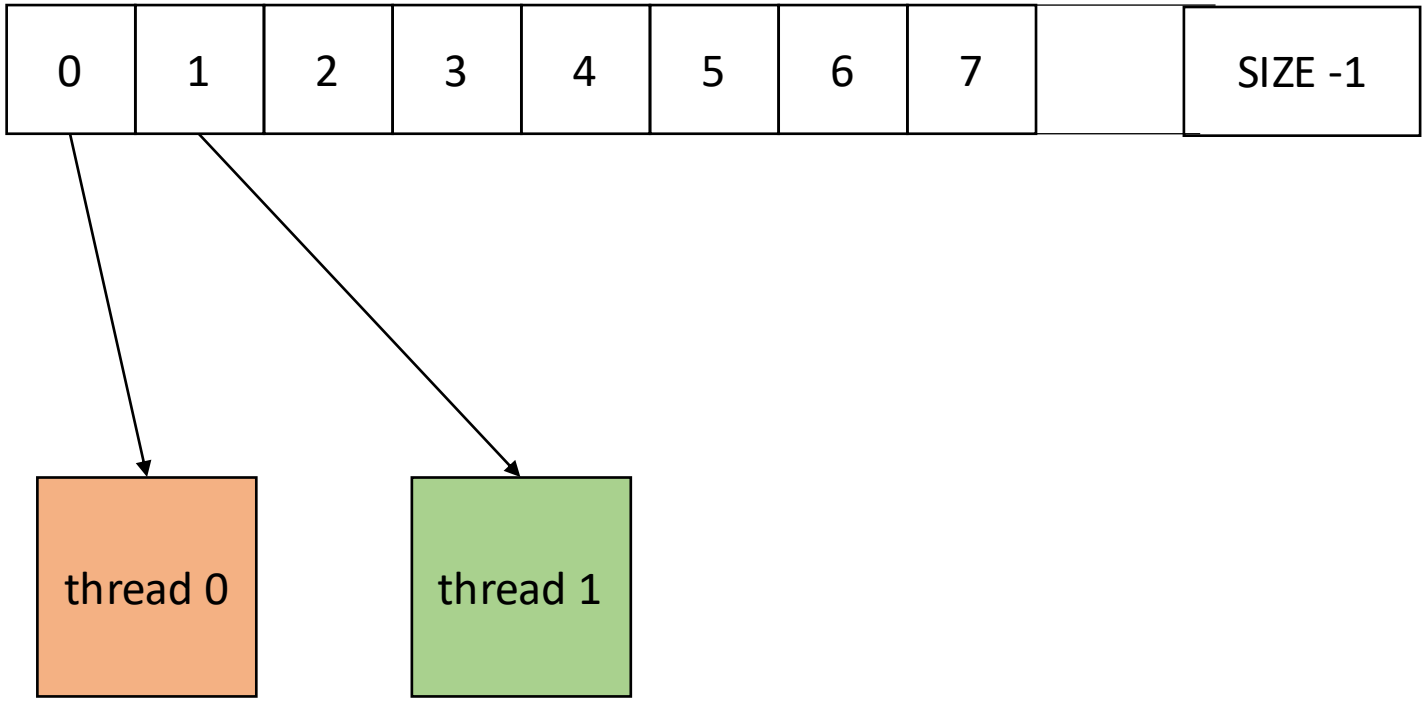


cannot color initially!



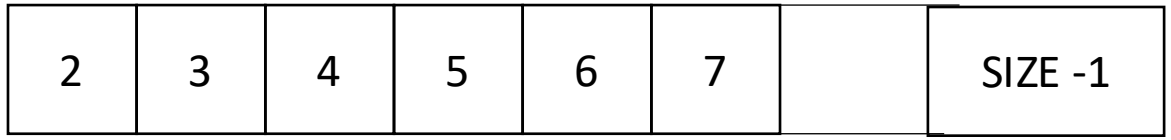
Work stealing - global worklist

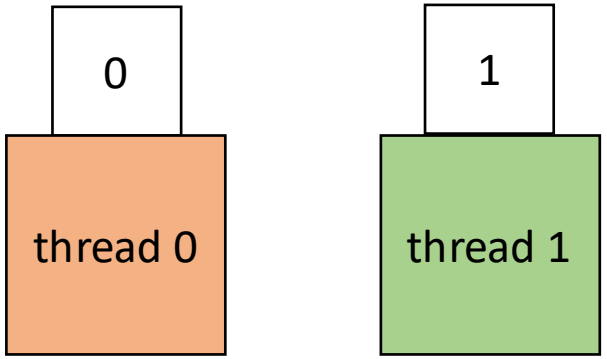
- Global worklist: threads take tasks (iterations) dynamically



Work stealing - global worklist

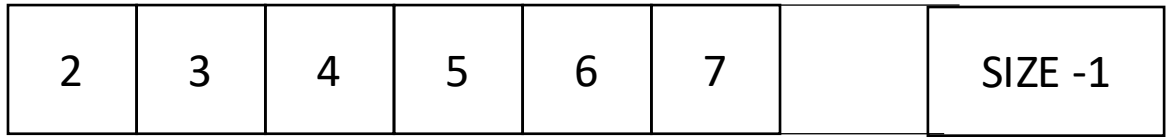
- Global worklist: threads take tasks (iterations) dynamically

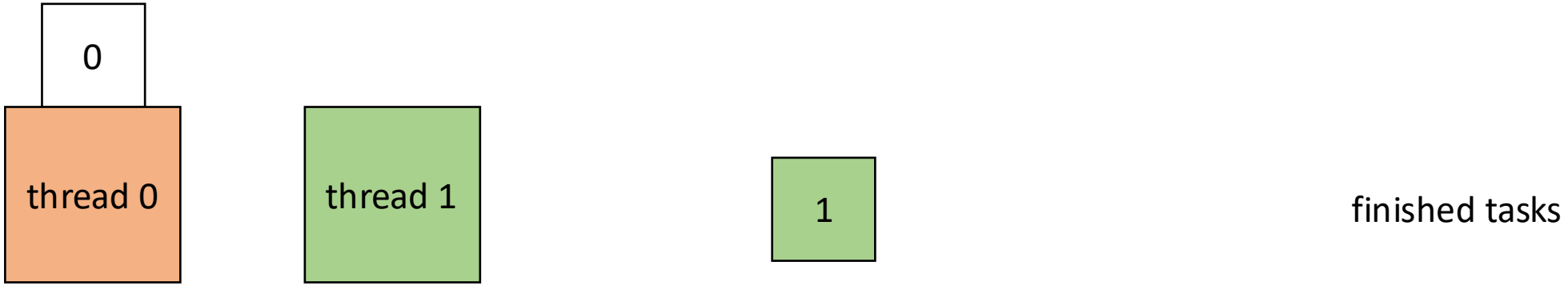




Work stealing - global worklist

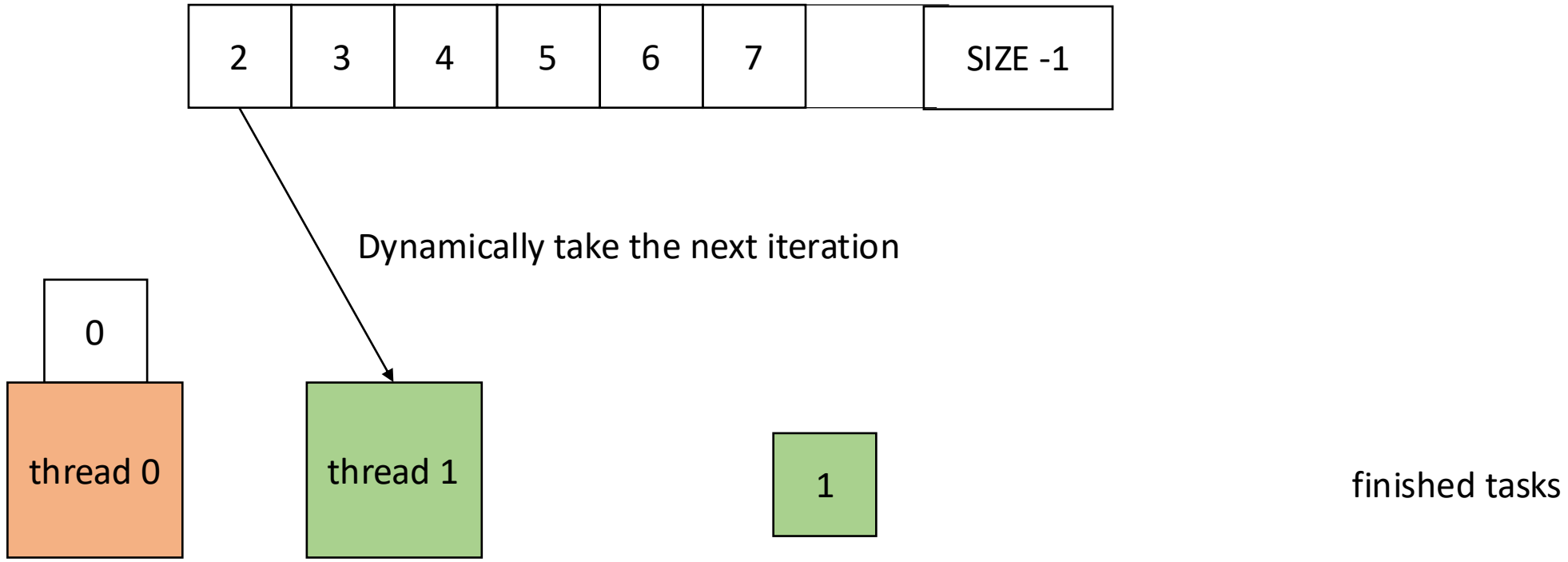
- Global worklist: threads take tasks (iterations) dynamically





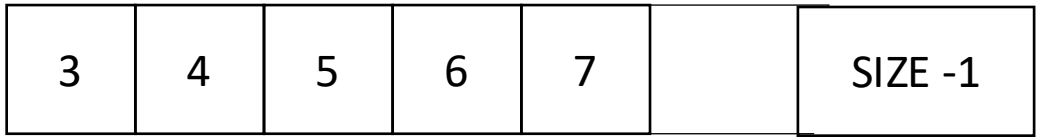
Work stealing - global worklist

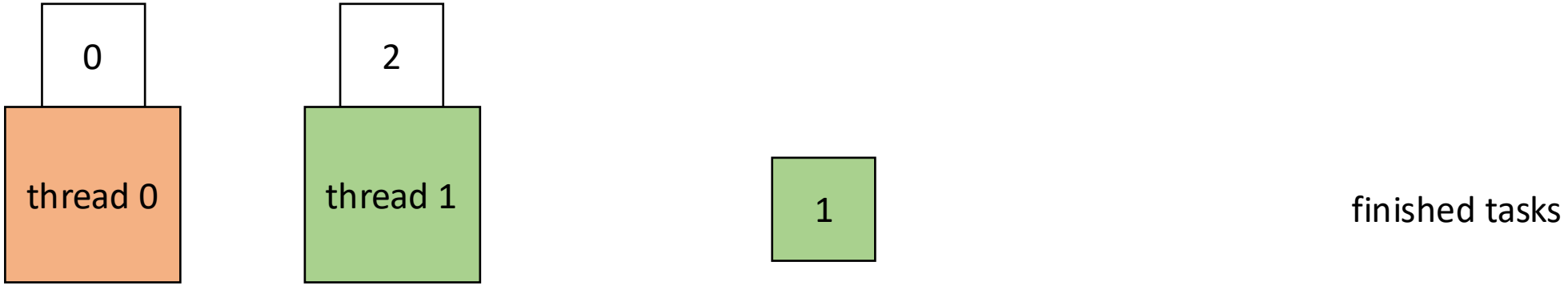
- Global worklist: threads take tasks (iterations) dynamically



Work stealing - global worklist

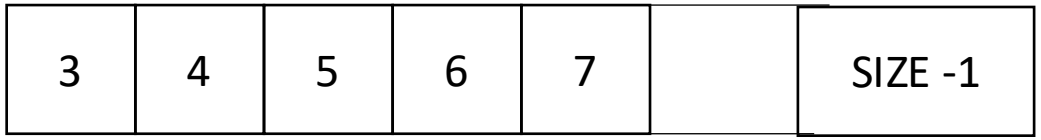
- Global worklist: threads take tasks (iterations) dynamically

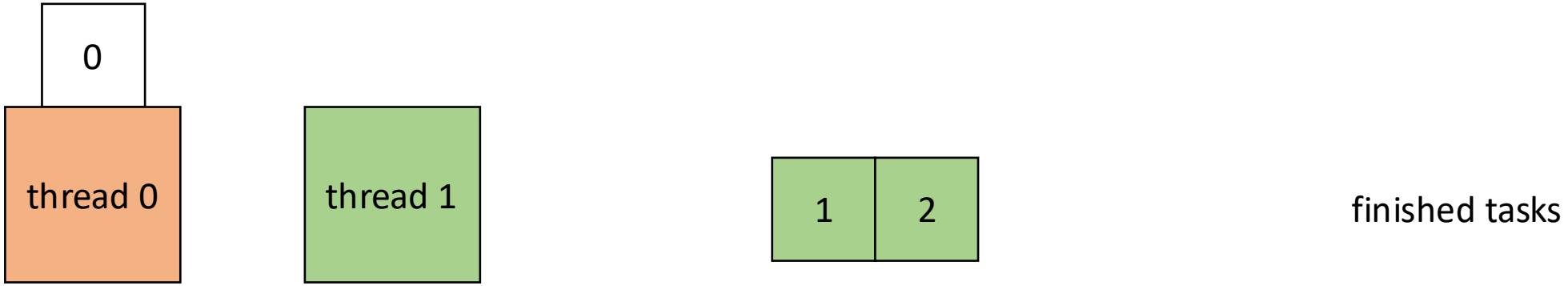




Work stealing - global worklist

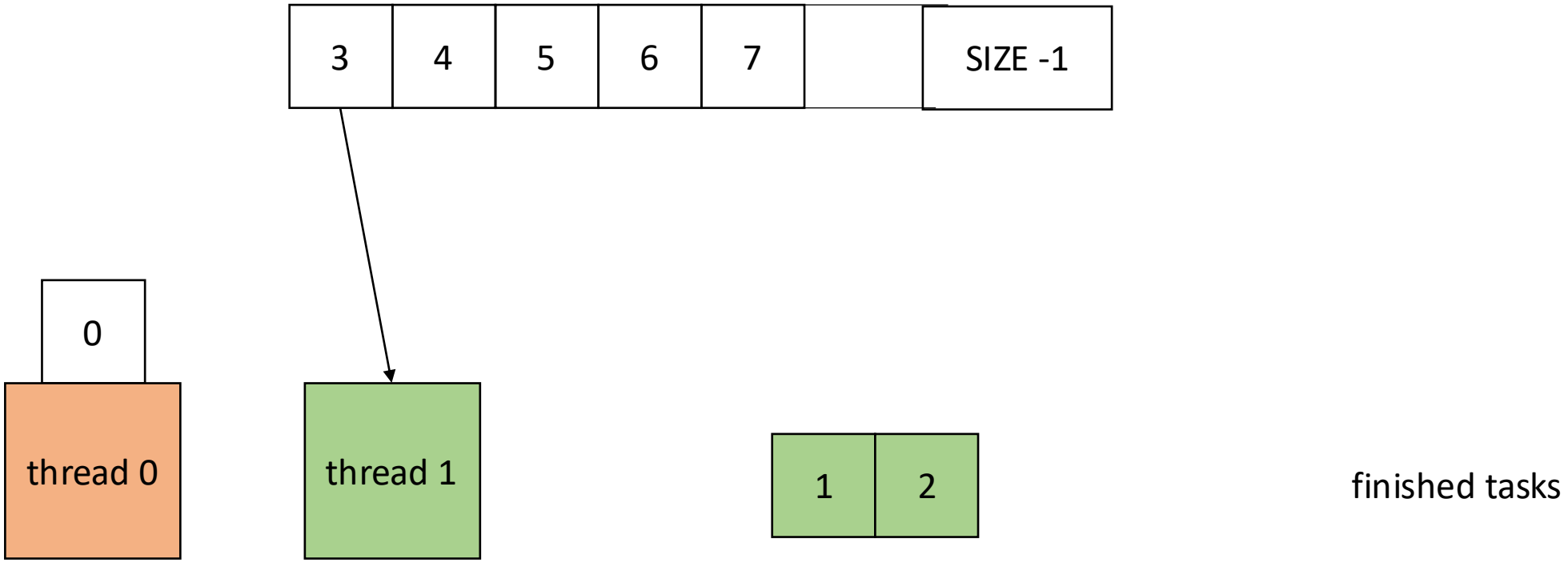
- Global worklist: threads take tasks (iterations) dynamically





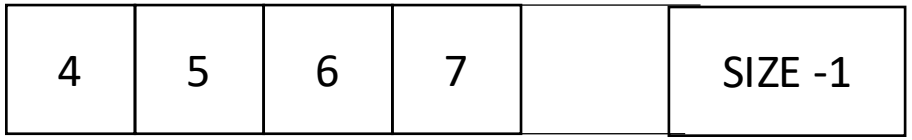
Work stealing - global worklist

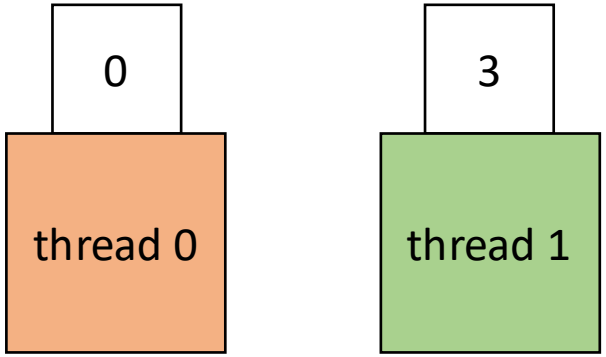
- Global worklist: threads take tasks (iterations) dynamically

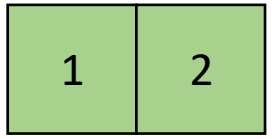


Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically



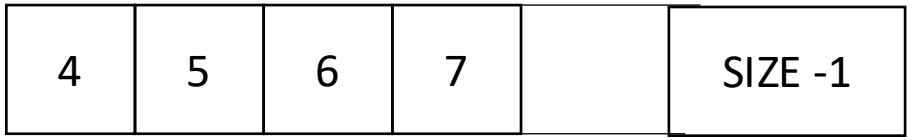


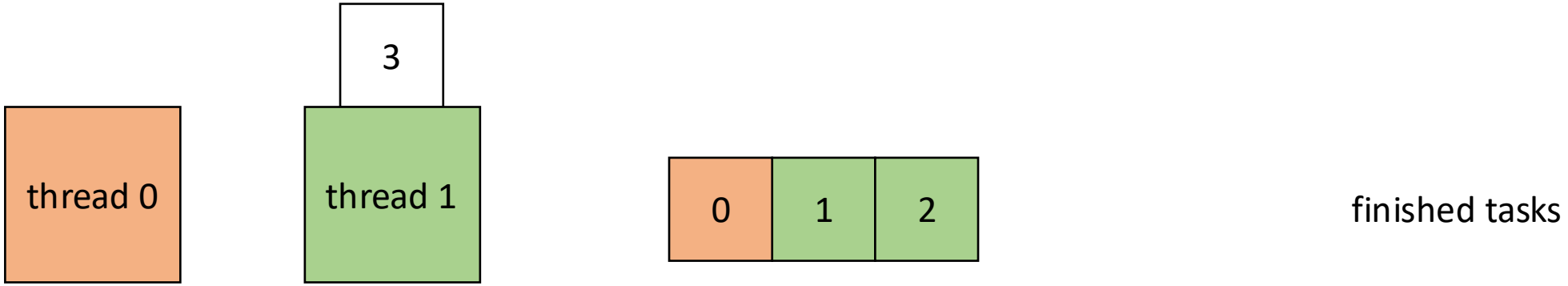


finished tasks

Work stealing - global worklist

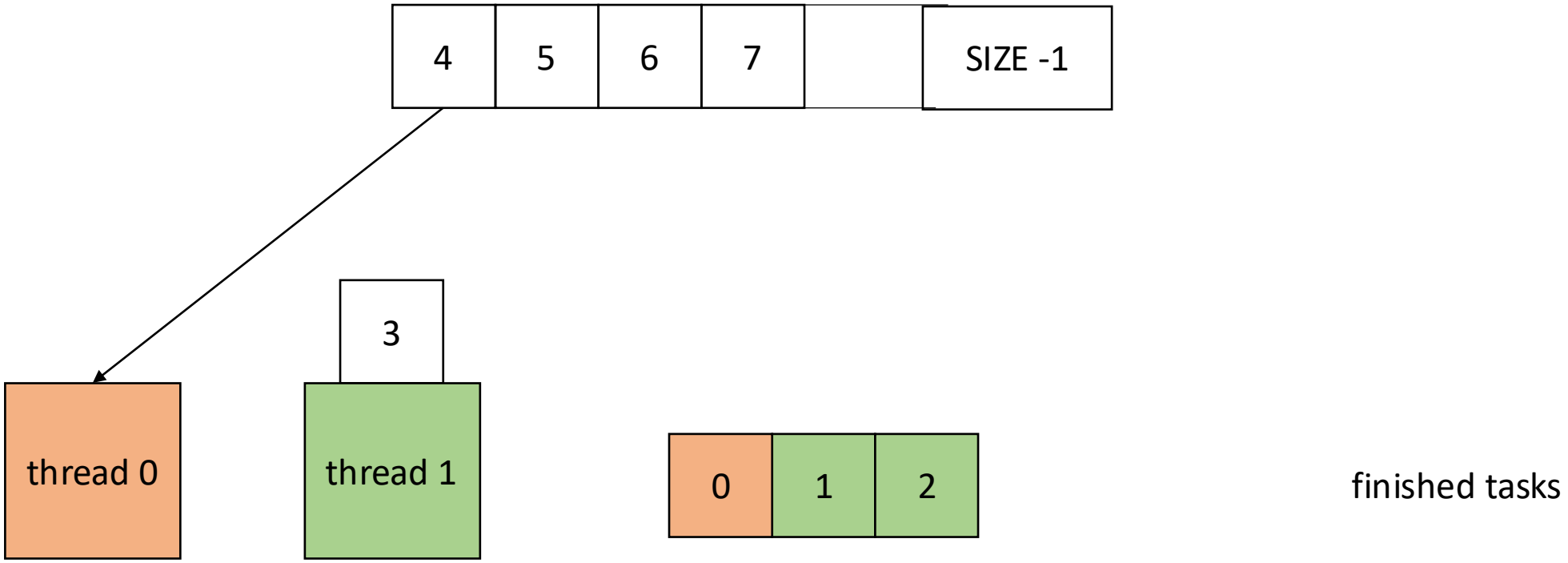
- Global worklist: threads take tasks (iterations) dynamically





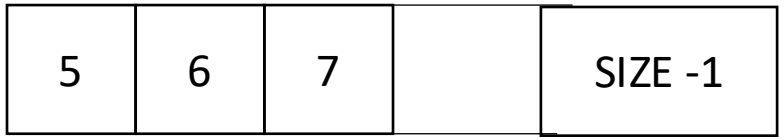
Work stealing - global worklist

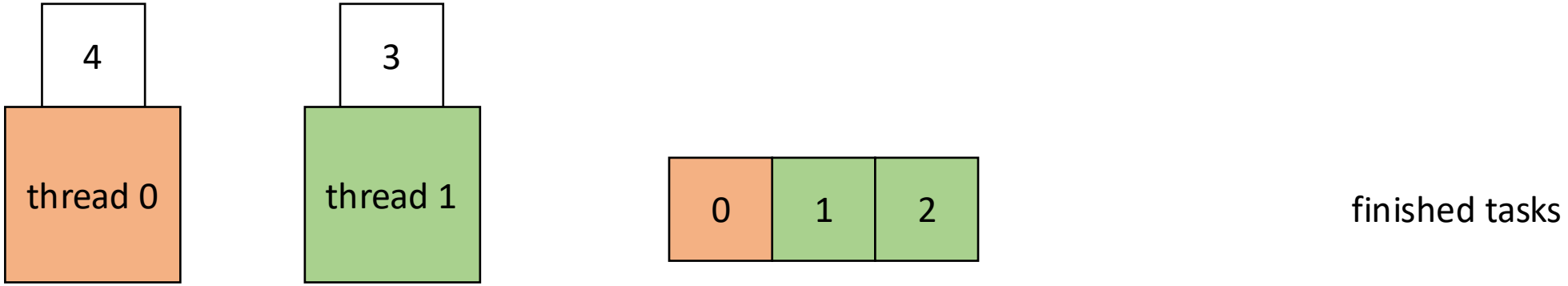
- Global worklist: threads take tasks (iterations) dynamically



Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically





Work stealing - global worklist

- How to implement

```
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

Work stealing - global worklist

- How to implement

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
   ...
}
```

```
void parallel_loop(...) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

Replicate code in a new function. Pass all needed variables as arguments.

Work stealing - global worklist

- How to implement

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
   ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

move loop variable to be a global atomic variable

Work stealing - global worklist

- How to implement

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
      // dynamic work based on x
   }
   ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

   for (int local_x = ??
            local_x < SIZE;
            local_x = ??) {

      // dynamic work based on x
   }
}
```

change loop bounds in new function to use a local variable using global variable.

Work stealing - global worklist

- How to implement

These must be atomic updates!

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
      // dynamic work based on x
   }
   ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

   for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

      // dynamic work based on x
   }
}
```

change loop bounds in new function to use a local variable using global variable.

Work stealing - global worklist

- How to implement

```
void foo() {
  ...
  for (t = 0; x < THREADS; t++) {
    spawn(parallel_loop);
  }
  join();
  ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

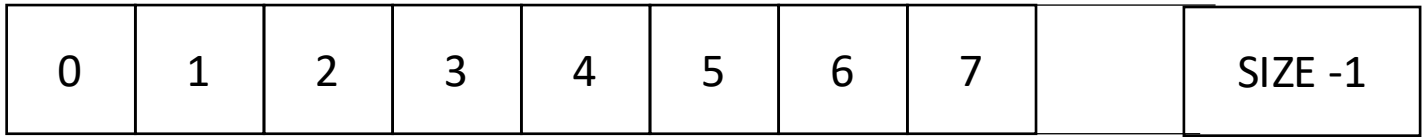
     // dynamic work based on x
  }
}
```

Spawn threads in original function and join them afterwards

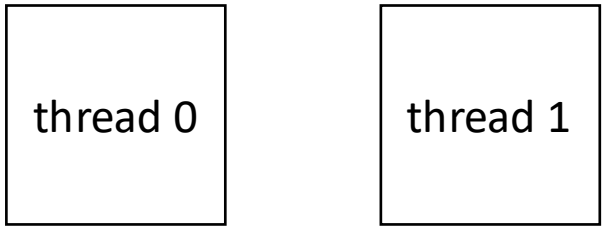
You will have to change to C++ syntax for the homework!

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 0
0 - local_x - UNDEF
1 - local_x - UNDEF
```



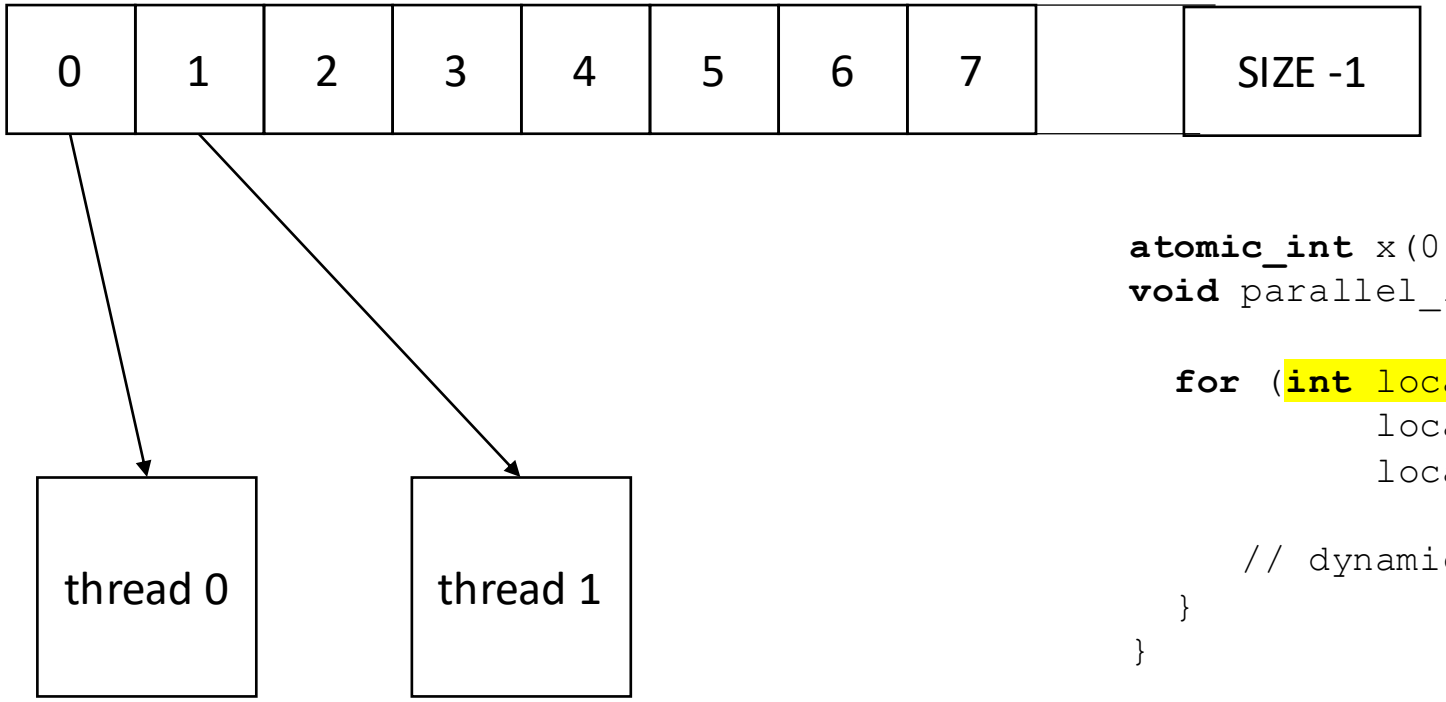

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
           local_x < SIZE;
           local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 2
0 - local_x - 0
1 - local_x - 1
```

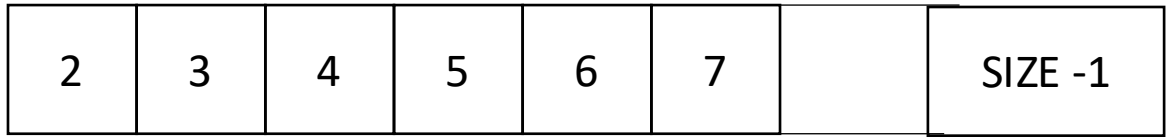
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

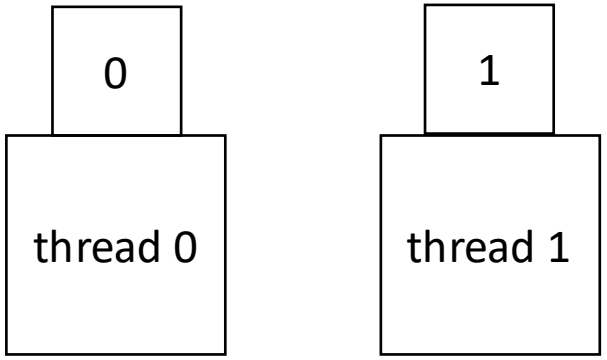
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 2
0 - local_x - 0
1 - local_x - 1
```



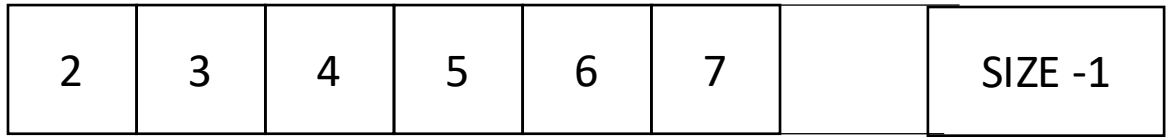

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
           local_x < SIZE;
           local_x = atomic_fetch_add(&x,1)) {

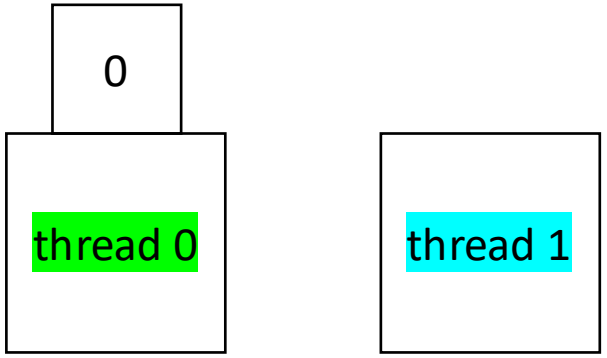
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 2
0 - local_x - 0
1 - local_x - 1
```



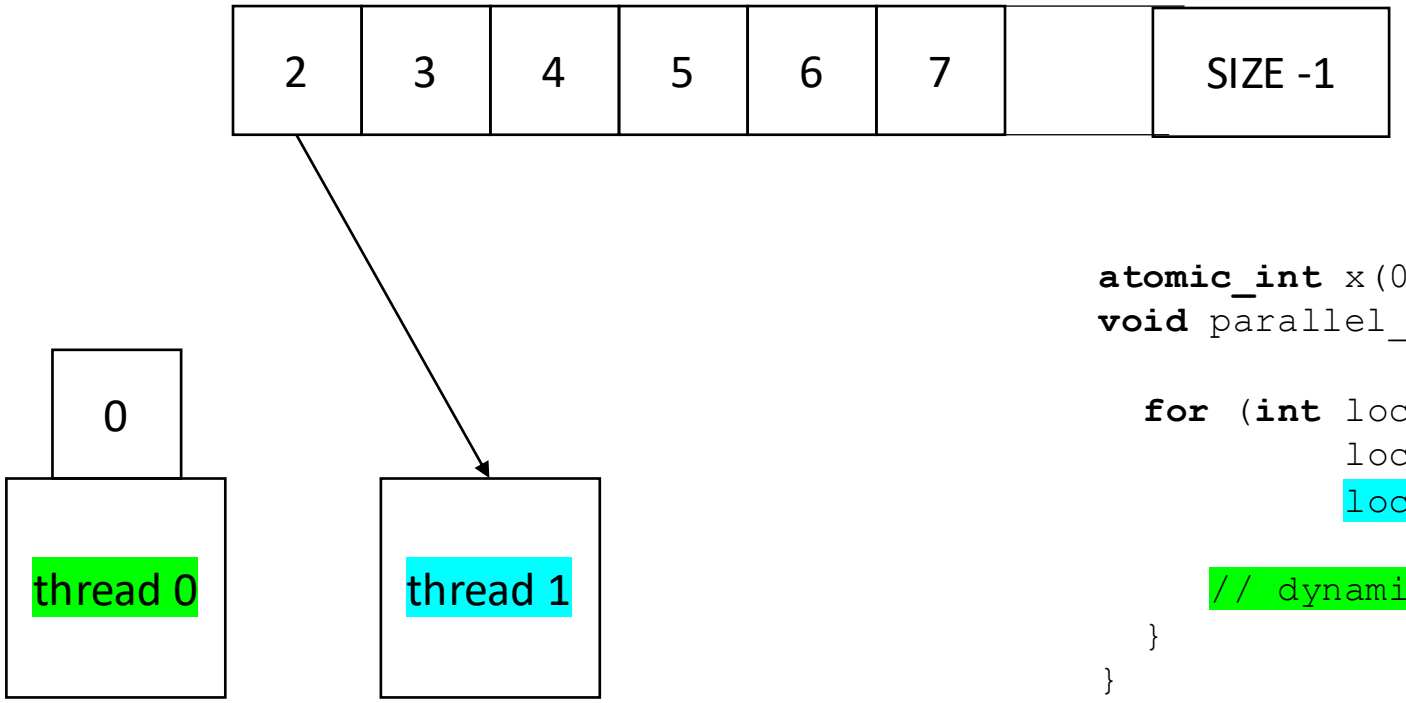

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 3
0 - local_x - 0
1 - local_x - 2
```

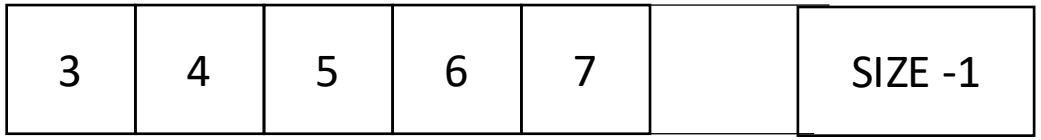
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

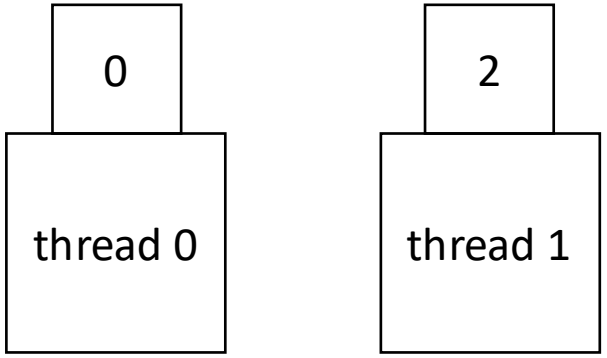
      // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 3
0 - local_x - 0
1 - local_x - 2
```



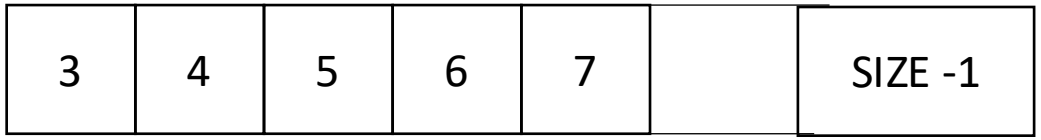

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

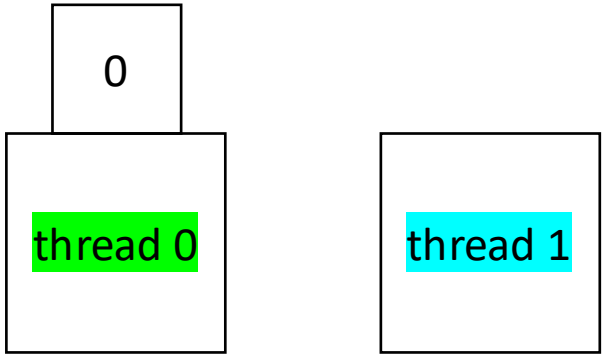
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 3
0 - local_x - 0
1 - local_x - 2
```



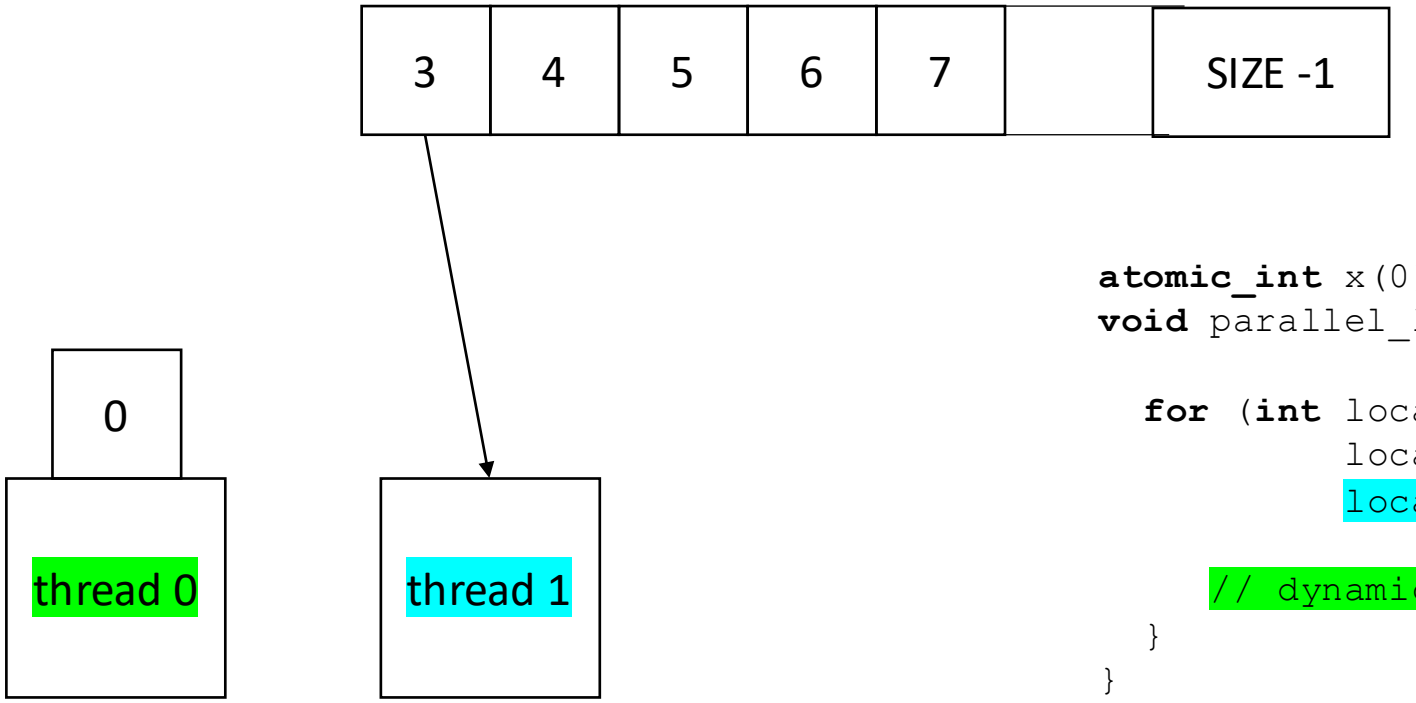

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 4
0 - local_x - 0
1 - local_x - 3
```

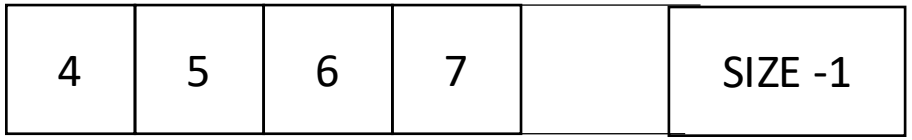
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

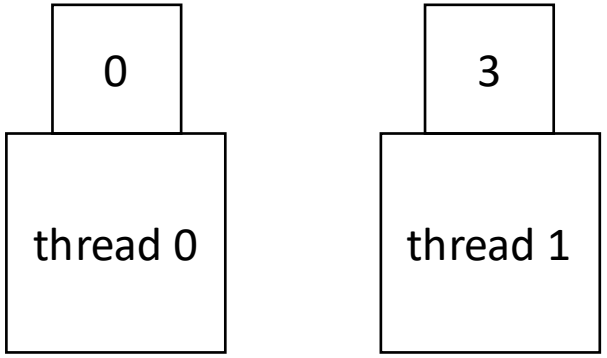
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 4
0 - local_x - 0
1 - local_x - 3
```



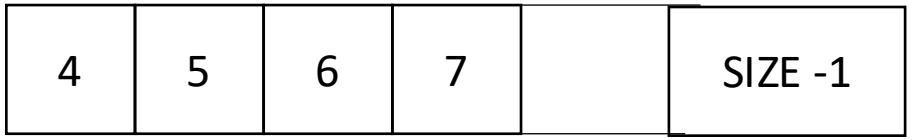

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

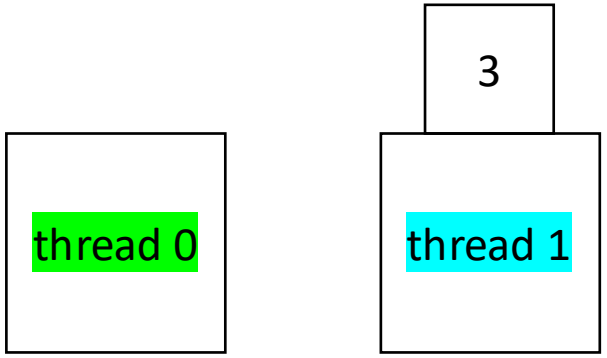
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 4
0 - local_x - 0
1 - local_x - 3
```



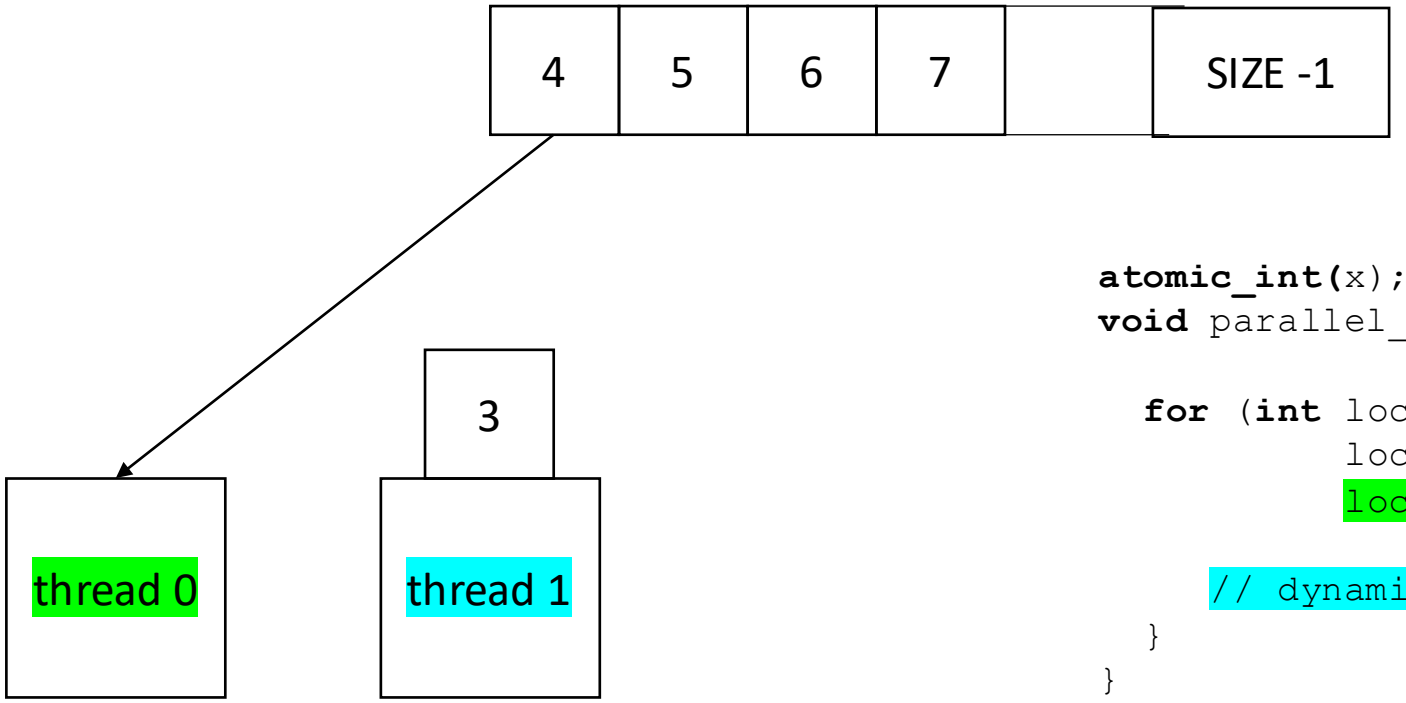

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 5
0 - local_x - 4
1 - local_x - 3
```

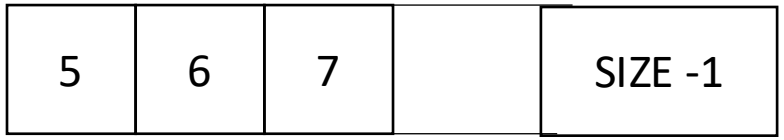
```
atomic_int(x);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

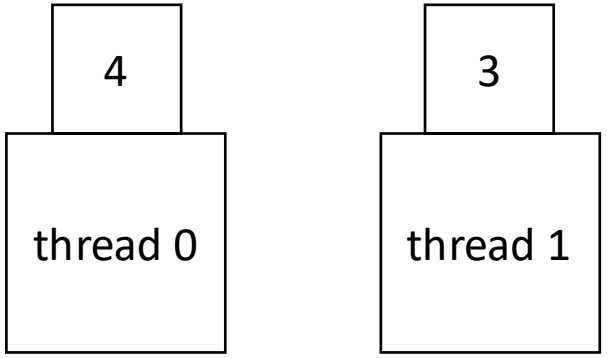
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 5
0 - local_x - 4
1 - local_x - 3
```




```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

Quiz 10.

Which one of the following is NOT a drawback of a global workstealing parallel schedule

Select one

A. Requires a concurrent data structure

B. Contention on shared cache lines

C. Contention on a single location with RMWs

Scan to answer

forms.gle/HrhkkgQiYqML2jcB9

chart: docs.google.com/forms/d/e/1FAIpQLSckAO0lpZ5Ng4HW5K7ZxxB4dclr8g6V9sTUHZoEJXz1CXRvHw/viewanalytics

Quiz 10.

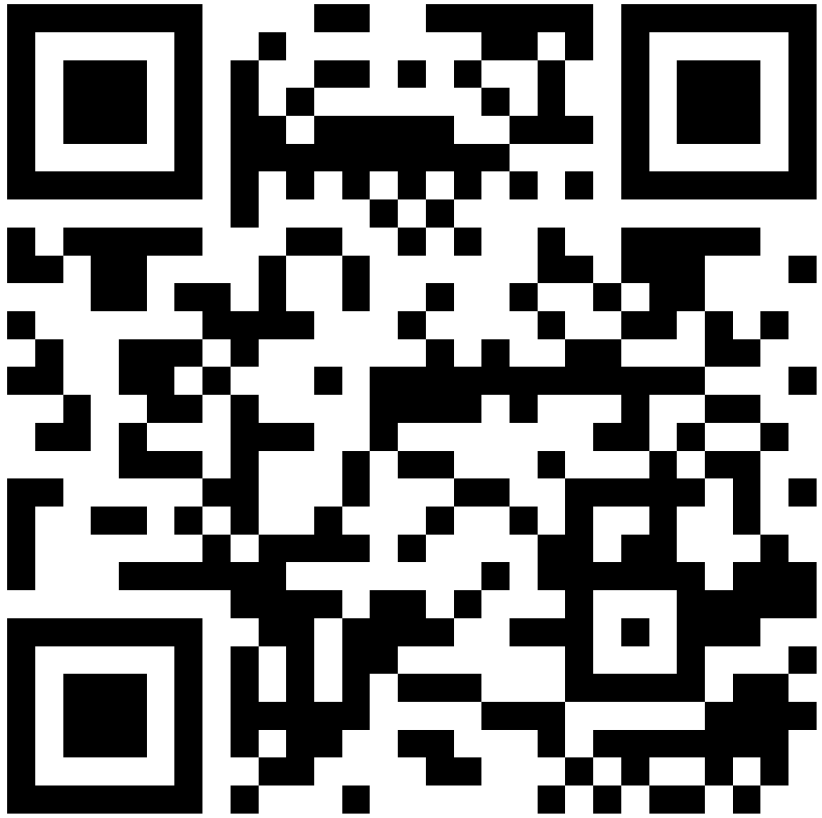
Which one of the following is NOT a drawback of a global workstealing parallel schedule

Select one

A. Requires a concurrent data structure

B. Contention on shared cache lines

C. Contention on a single location with RMWs



Scan to answer

A concurrent data structure is needed.

forms.gle/HrhkkgQiYqML2jcB9

chart: docs.google.com/forms/d/e/1FAIpQLSckAO0lpZ5Ng4HW5K7ZxxB4dclr8g6V9sTUHZoEJXz1CXRvHw/viewanalytics

Schedule

- DOALL Loops

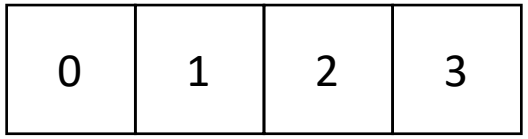
- **Parallel Schedules:**
  - Static
  - Global Worklists
  - **Local Worklists**

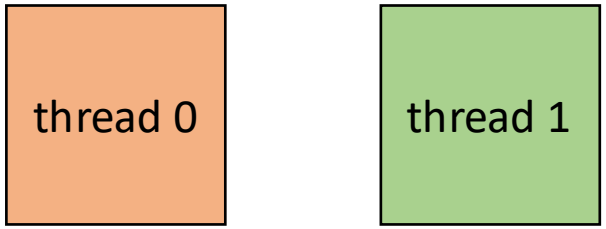
Work stealing - local worklists

- More difficult to implement
- low contention on local data-structures
- potentially better cache locality

Work stealing - local worklists

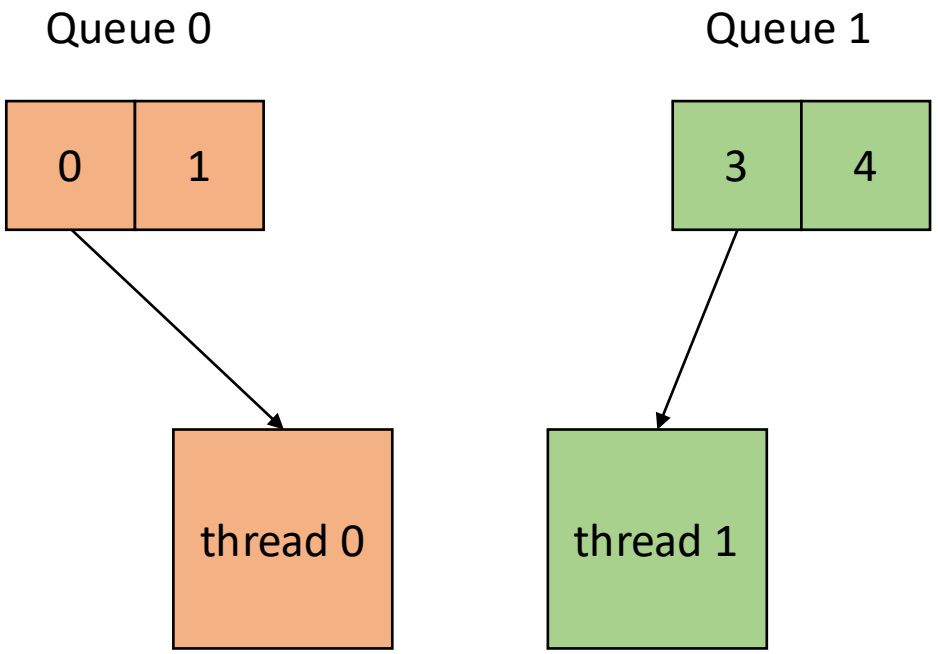
- local worklists: divide tasks into different worklists for each thread





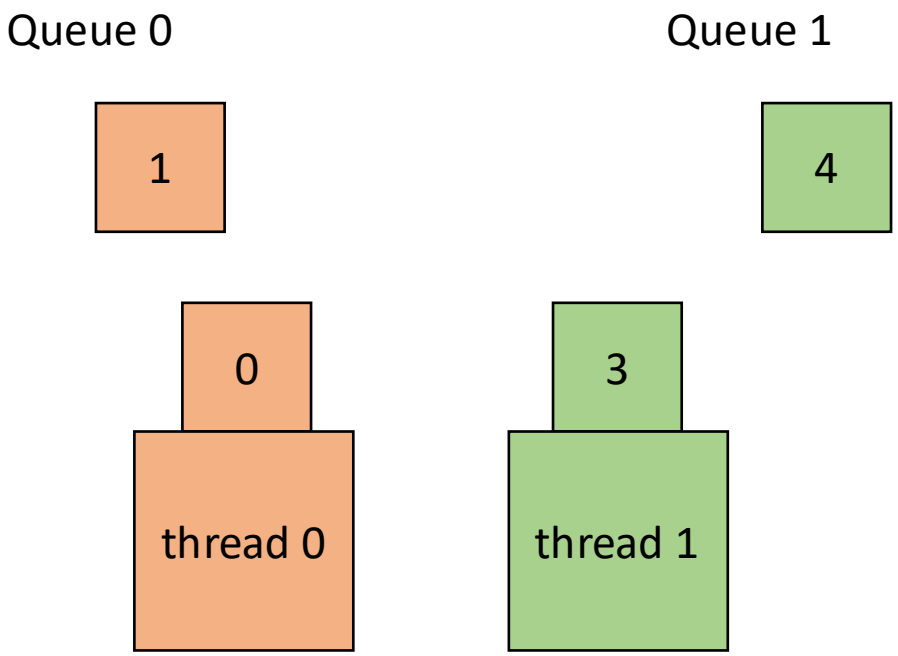
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



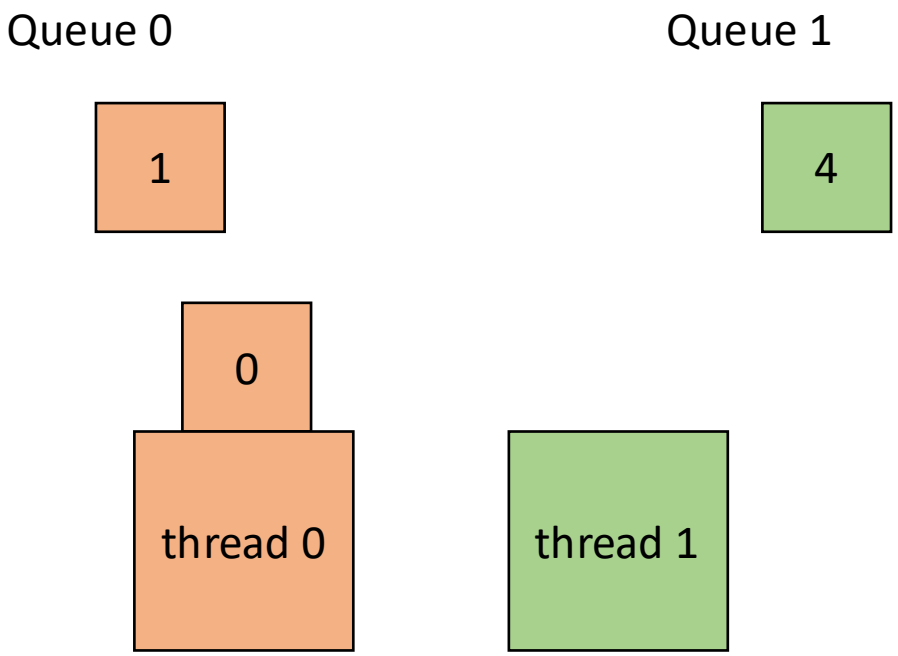
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



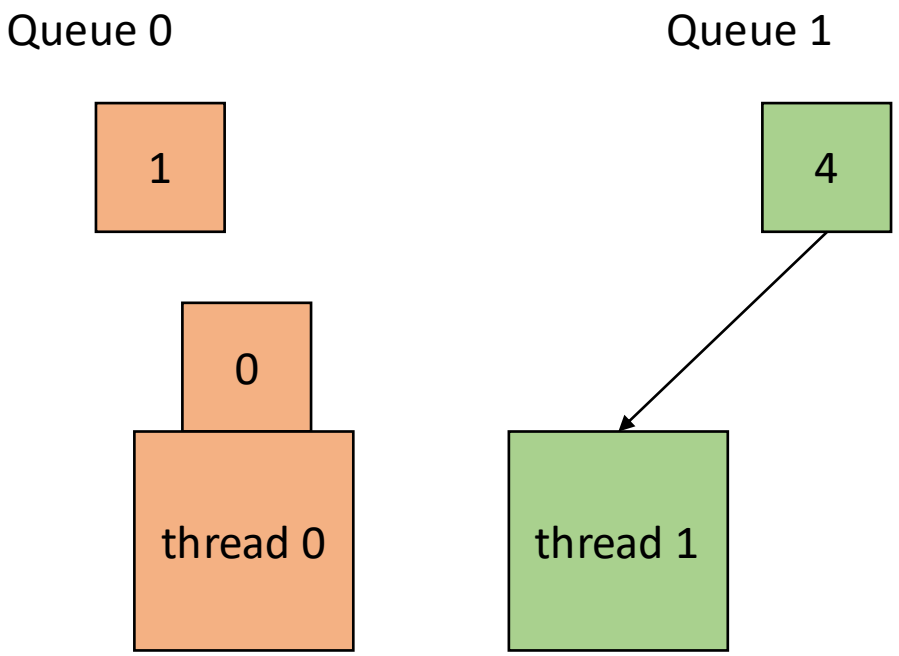
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



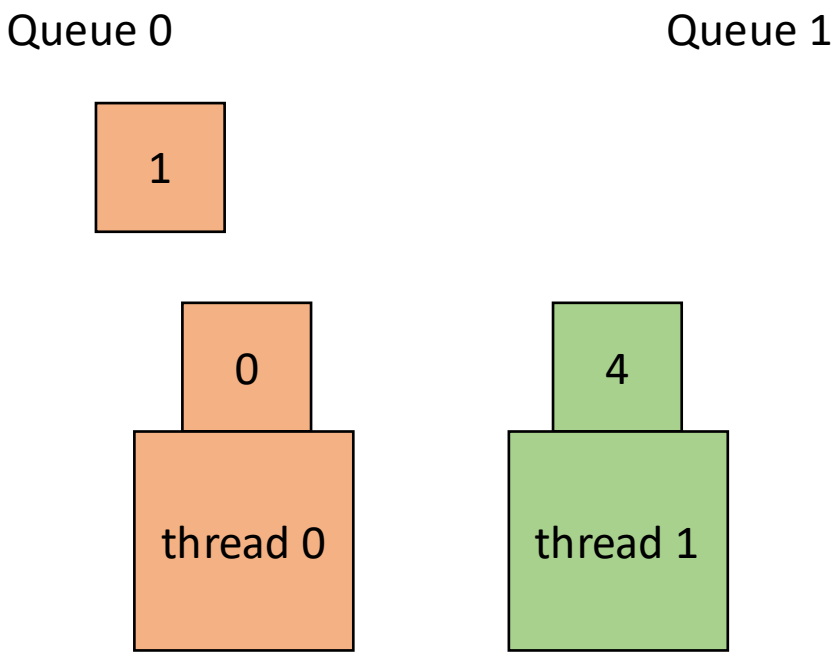
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



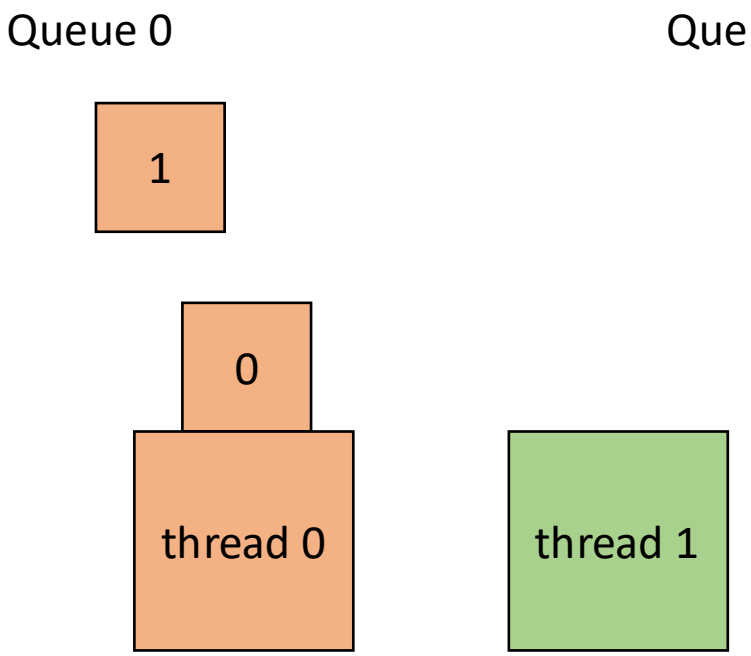
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



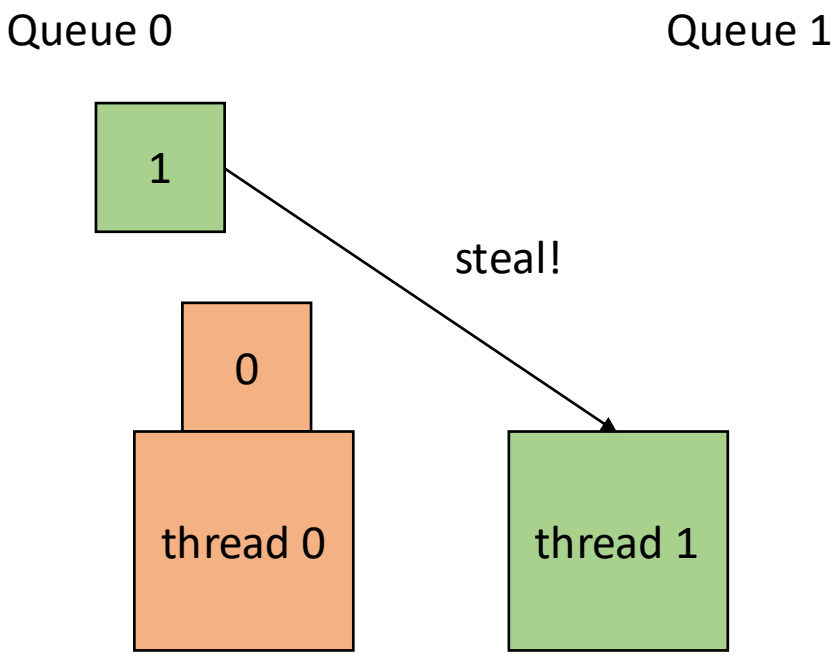
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread

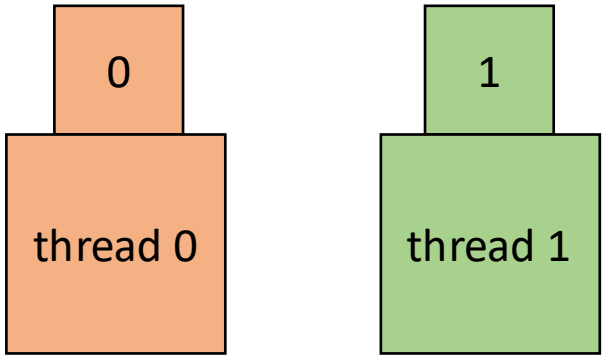


Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread

Queue 0

Queue 1



Work stealing - local worklists

- How to implement:

```
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

Work stealing - local worklists

- How to implement:

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
   ...
}
```

```
void parallel_loop(..., int tid) {

   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
}
```

Make a new function, taking any variables used in loop body as args. Additionally take in a thread id

Work stealing - local worklists

- How to implement:

```
concurrent_queue cq[NUM_THREADS];
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

```
void parallel_loop(..., int tid) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

Make a global array of concurrent queues

Work stealing - local worklists

- How to implement:

```
concurrent_queue cq[NUM_THREADS];
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

```
void parallel_loop(..., int tid) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

What type of queues?

Make a global array of concurrent queues

Work stealing - local worklists

- How to implement:

```
concurrent_queue cq[NUM_THREADS];
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

```
void parallel_loop(..., int tid) {

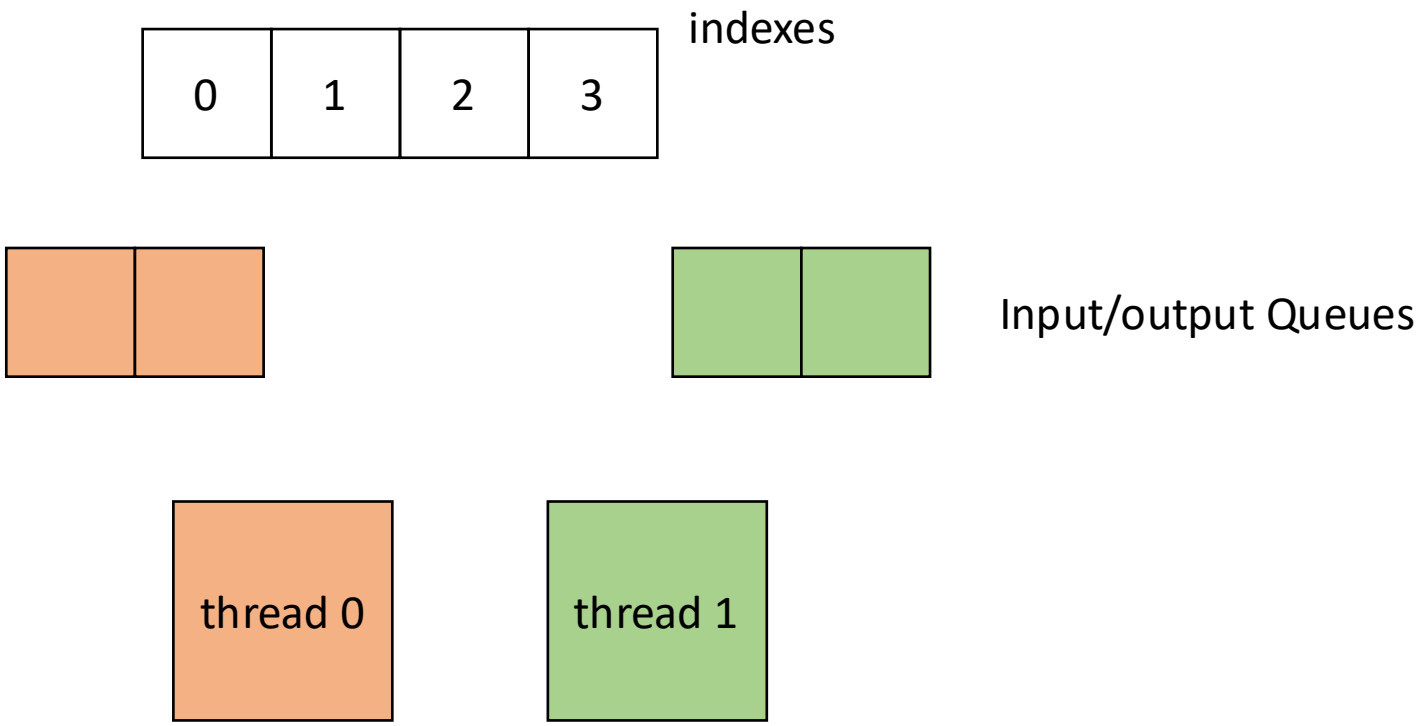
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

Make a global array of concurrent queues

What type of queues?
We're going to use InputOutput Queues!

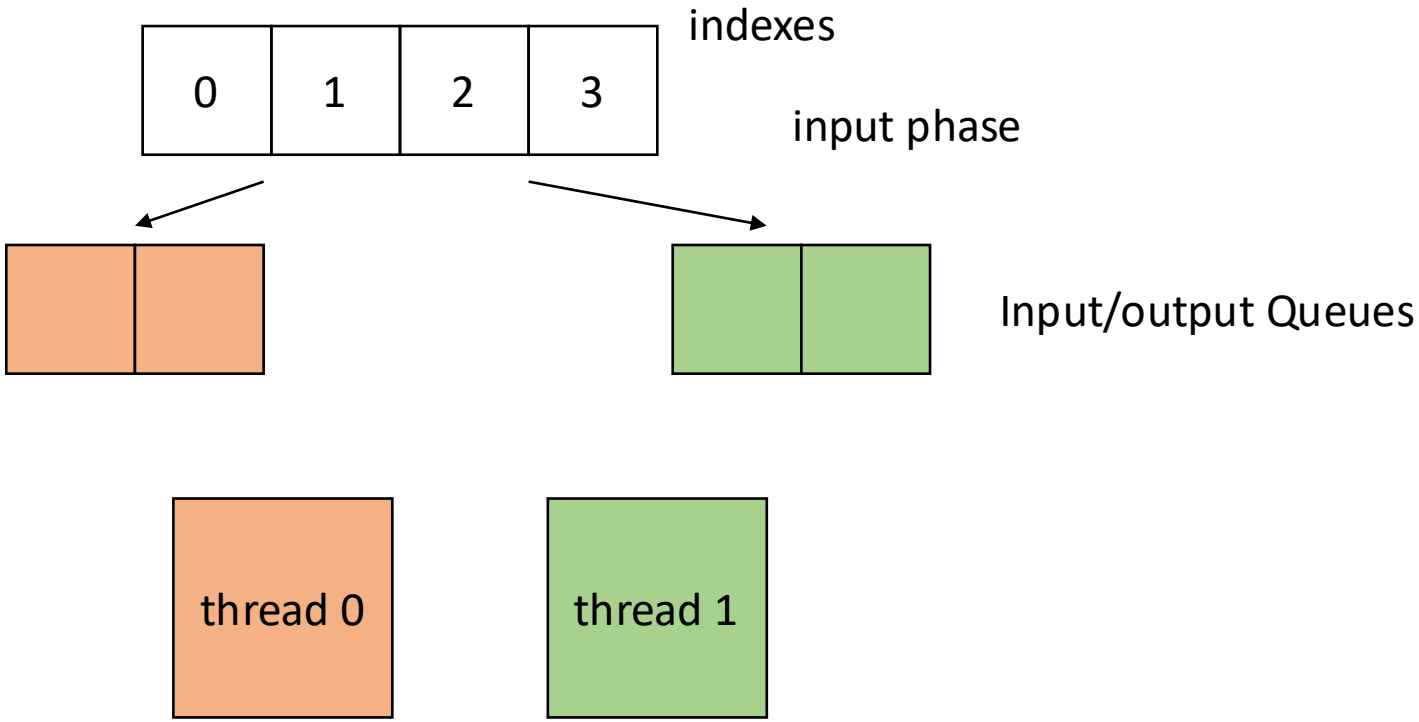
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



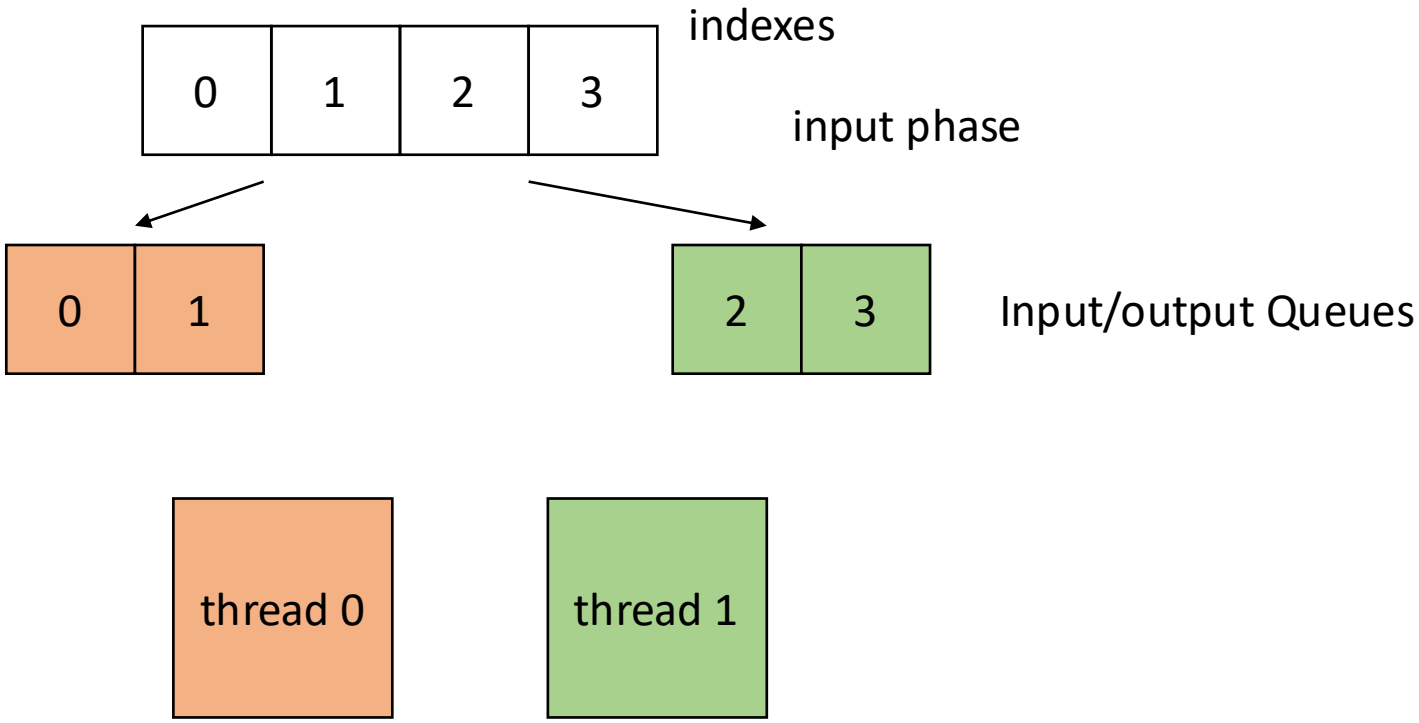
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



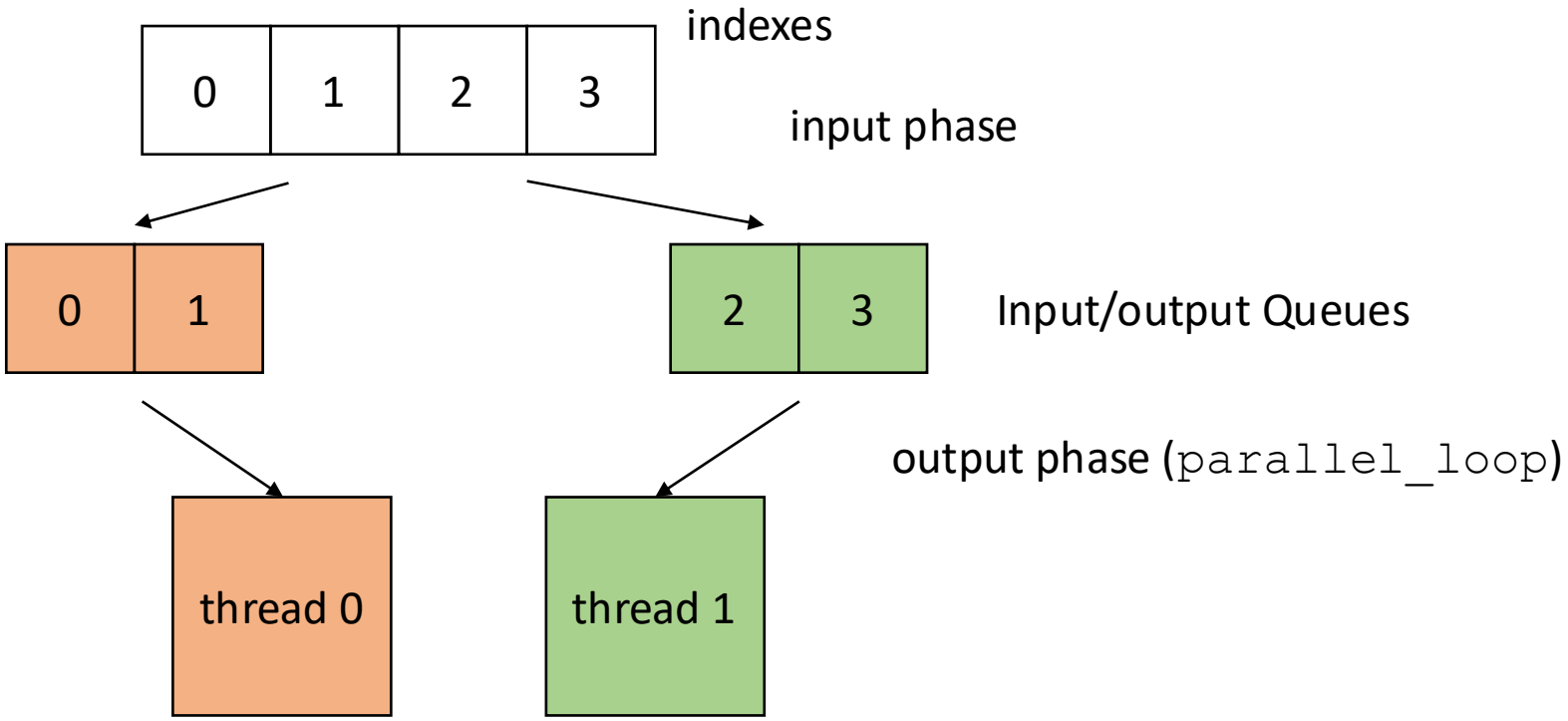
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...


   ...
}
```

First, we need to initialize the queues

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // Spawn threads to initialize
   // join initializing threads

   ...
}
```

```
void parallel_enq(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    cq[tid].enq(x);
  }
}
```

Just like the static schedule, except we are enqueuing

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // Spawn threads to initialize
   // join initializing threads

   ...
}
```

Make sure to account for boundary conditions!

```
void parallel_enq(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    cq[tid].enq(x);
  }
}
```

Just like the static schedule, except we are enqueuing

Work stealing - local worklists

- How to implement in a compiler:

```
NUM_THREADS = 2;
SIZE = 4;
CHUNK = 2;
```

x	0	1	2	3

tid	0	0	1	1

Make sure to account for boundary conditions!

```
void parallel_enq(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    cq[tid].enq(x);
  }
}
```

Just like the static schedule, except we are enqueuing

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  ...
}
```

```
void parallel_loop(..., int tid, int num_threads) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

How do we modify the parallel loop?

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  ...
}
```

```
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
}
```

loop until the queue is empty

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  ...
}
```

```
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
}
```

loop until the queue is empty
Are we finished?

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

   // launch loop function
   ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
      // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
}
```

Track how many threads are finished

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

   // launch loop function
   ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
     // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
   while (finished_threads.load() != num_threads) {

   }
}
```

While there are threads that are still working

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

   // launch loop function
   ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
     // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
   while (finished_threads.load() != num_threads) {
     int target = // pick a thread to steal from
     int task = cq[target].deq();
   }
}
```

pick a random target and steal a task

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  // join loop threads
  ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
      // perform task
    }
  }
}
```

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

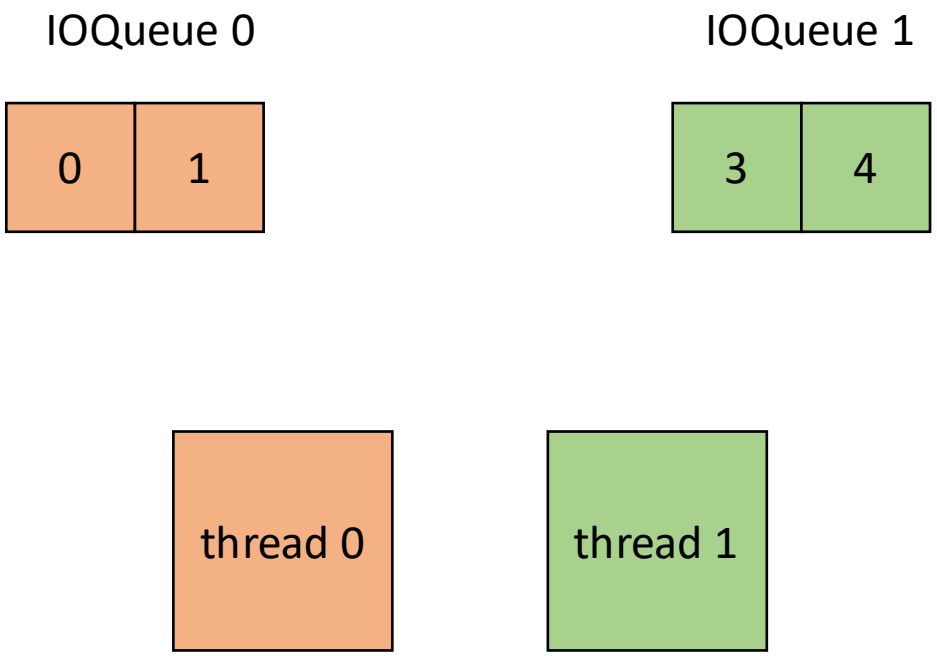
   // launch loop function
   // join loop threads
   ...
}
```

join the threads

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
      // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
   while (finished_threads.load() != num_threads) {
      int target = // pick a thread to steal from
      int task = cq[target].deq();
      if (task != -1) {
         // perform task
      }
   }
}
```

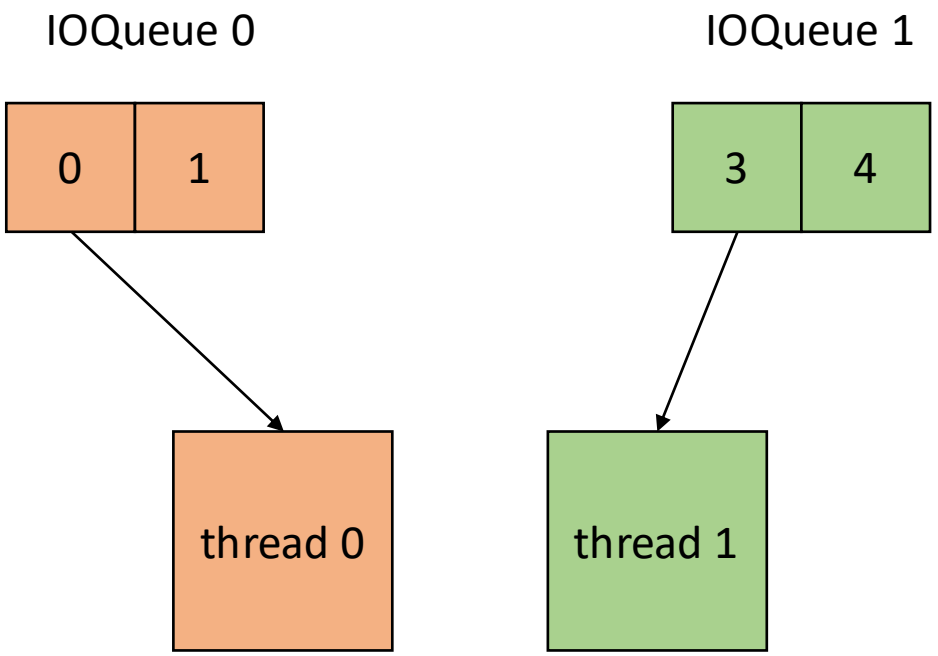
Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

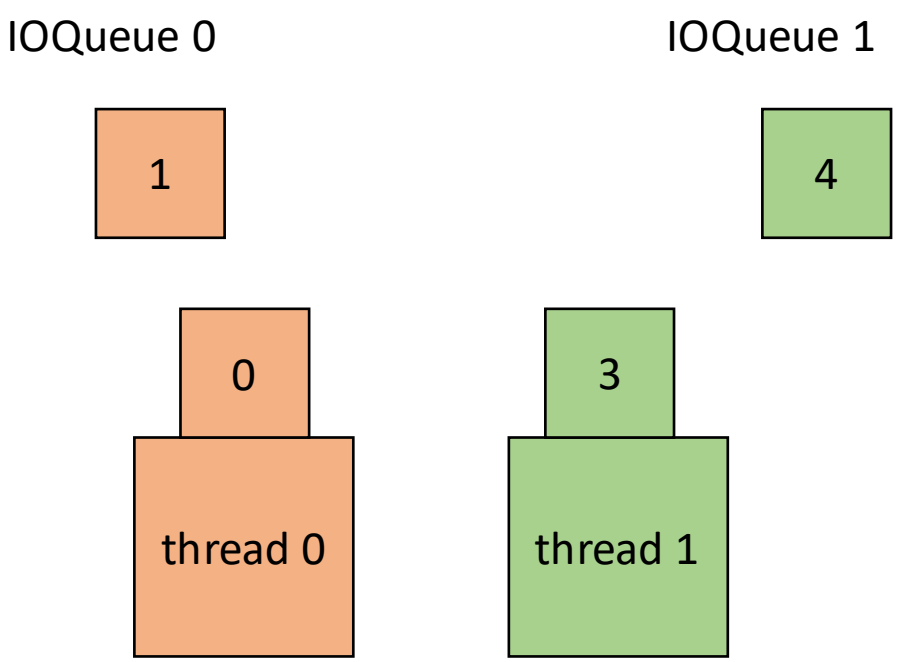
Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

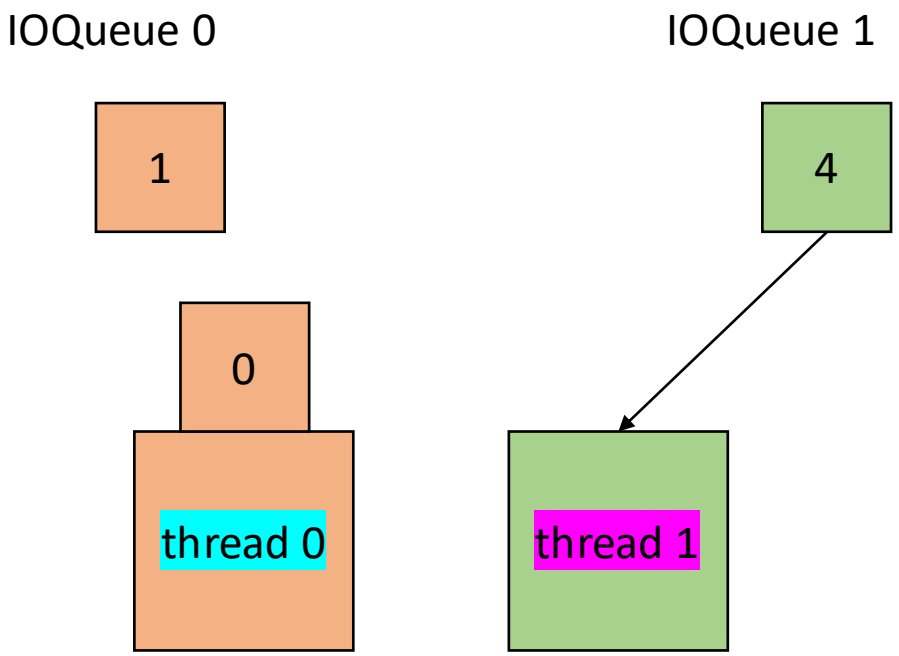
Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

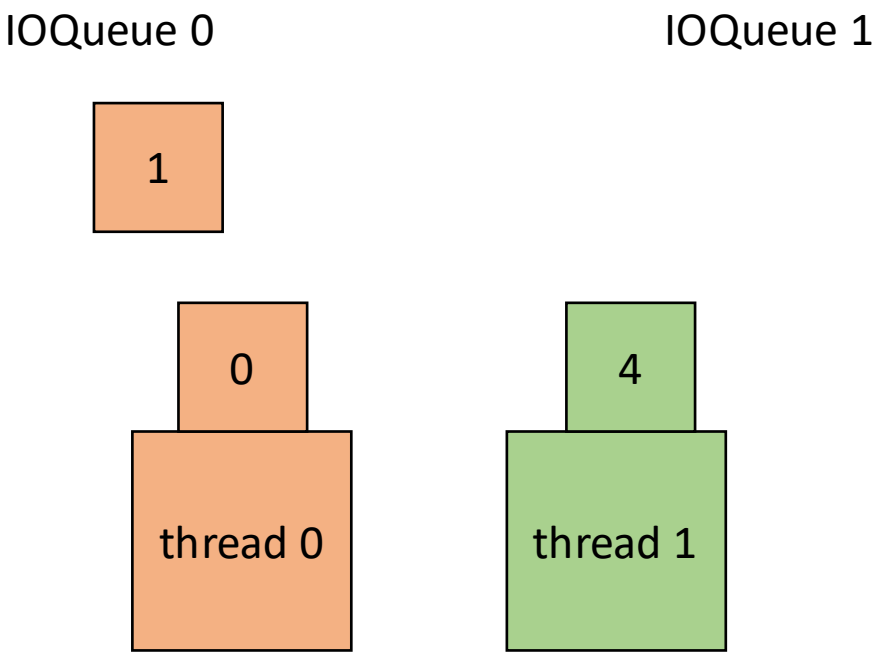
Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

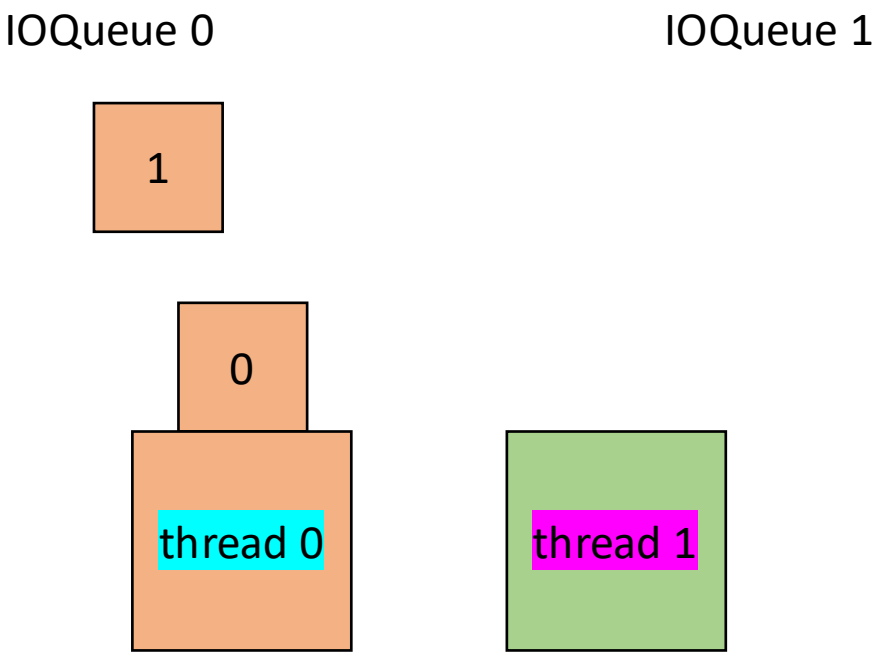
Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

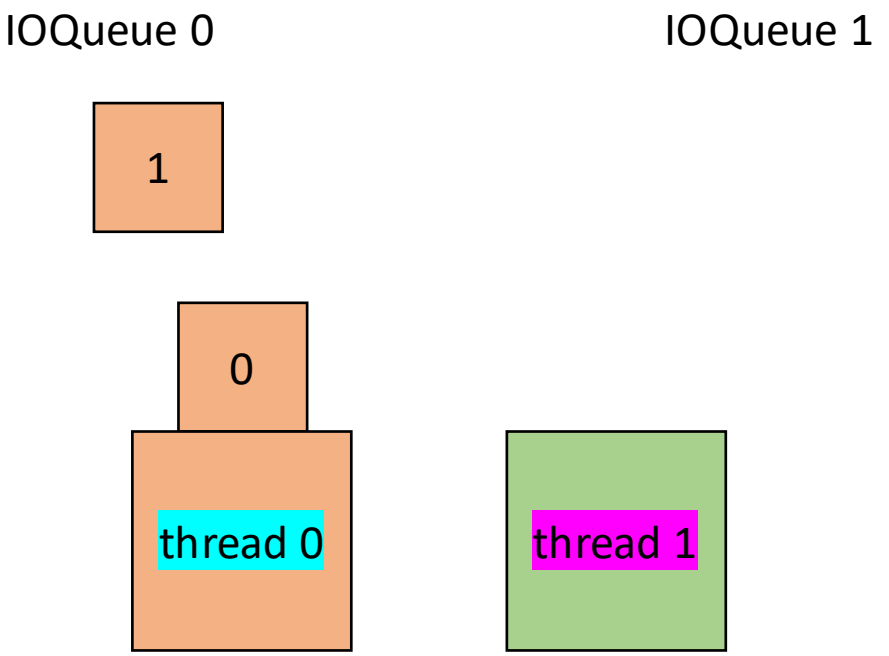



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 1

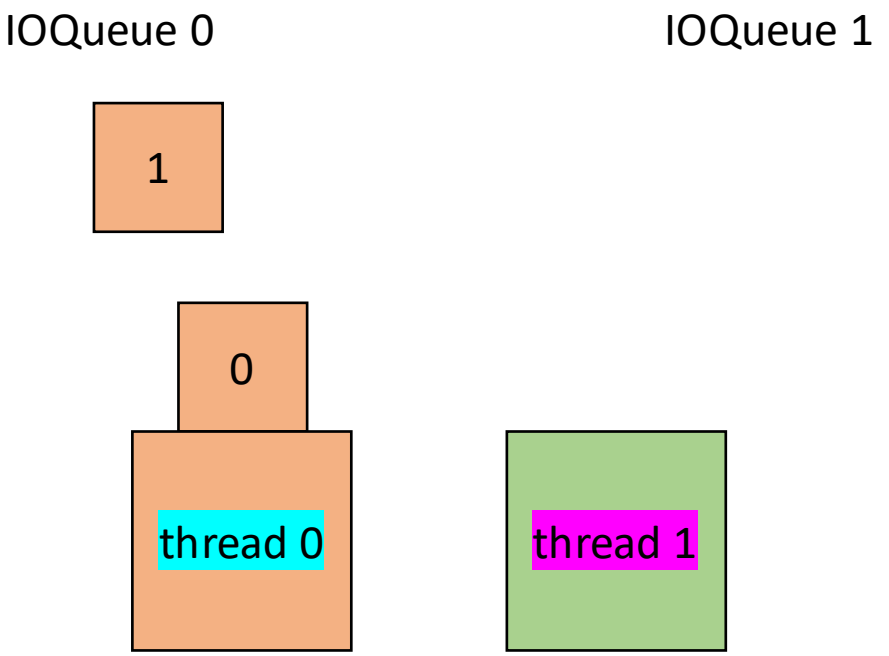



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 1

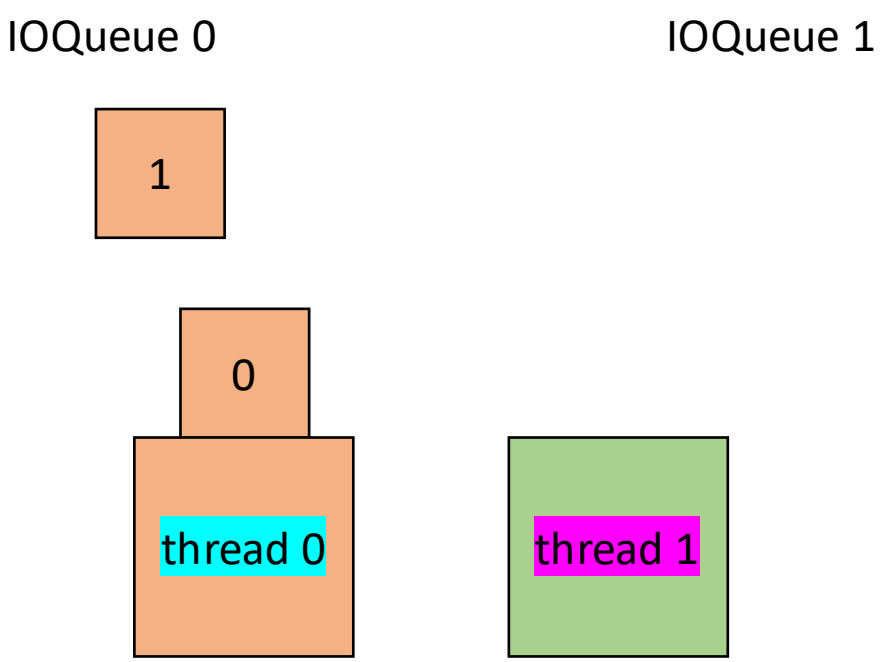



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 1

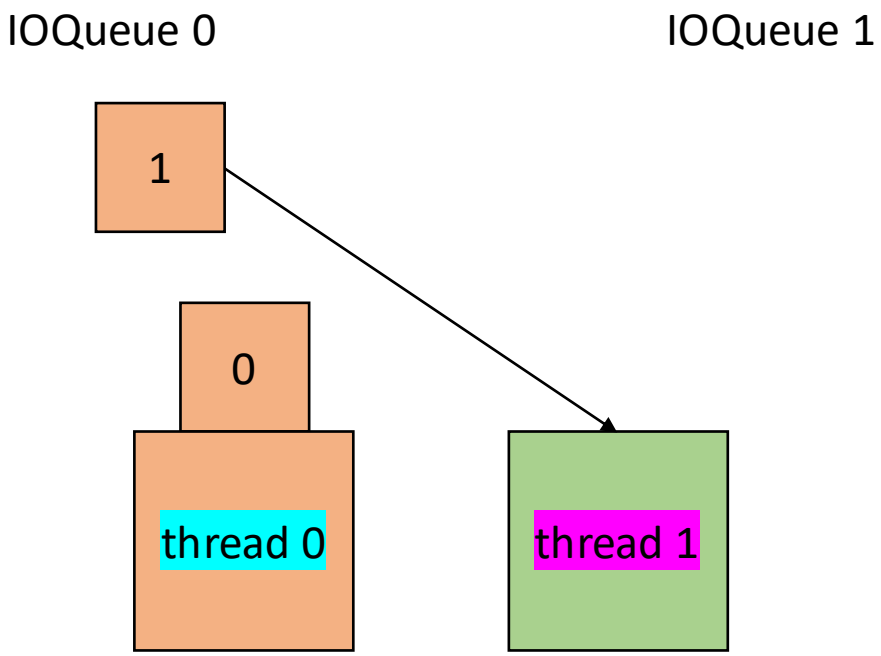



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

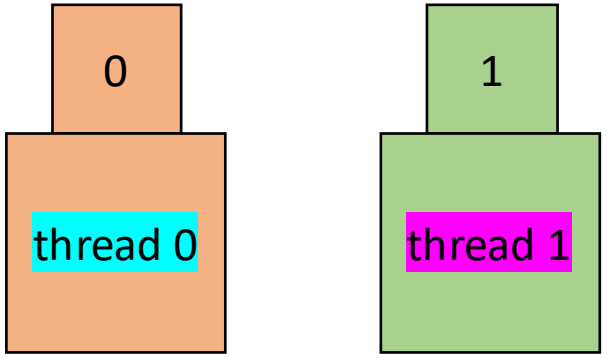
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 1

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

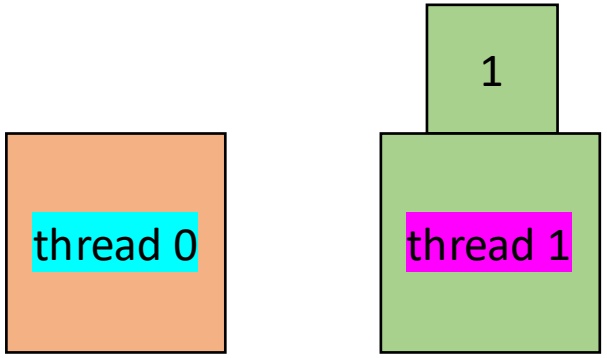
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

```
finished_threads: 1
```

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

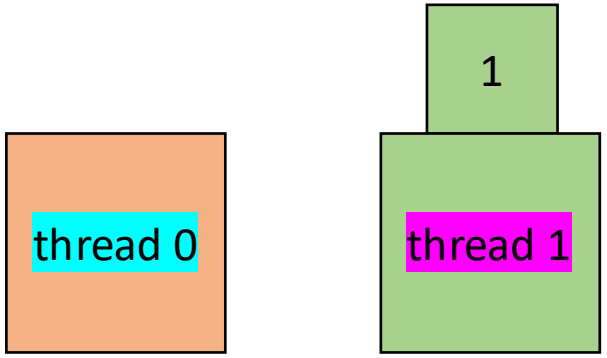
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

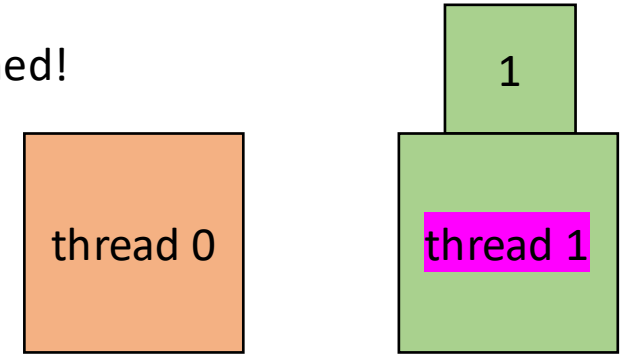
Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1

finished!




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

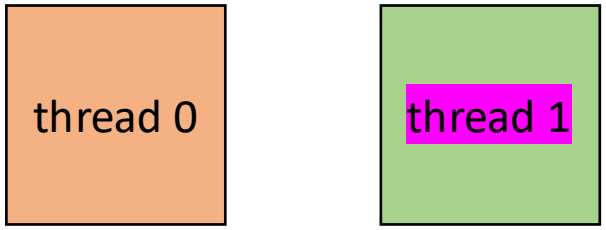
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

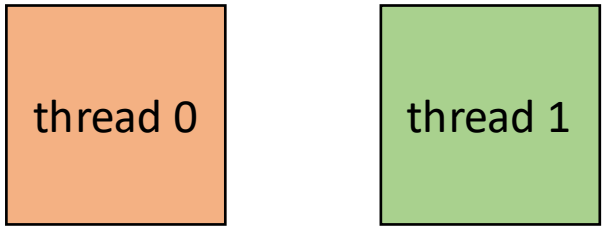
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Quiz 10.

Which of the following is NOT an overhead of the local worklist workstealing parallel schedule.

Select one

A. Initialization of the queues

B. Checking a global variable to ensure all work is completed

C. Managing concurrent enqueues to the worklists



Scan to answer

forms.gle/MmDQEFBLaqSTjkCm8

chart: docs.google.com/forms/d/e/1FAIpQLSe6ULaZ2AbL0nqH2QodJl9cJWas3N6XlaAJ4JPpO7jzyawL5A/viewanalytics

Quiz 10.

Which of the following is NOT an overhead of the local worklist workstealing parallel schedule.

Select one

A. Initialization of the queues

B. Checking a global variable to ensure all work is completed

C. Managing concurrent enqueues to the worklists

There were no concurrent enqueues.

Scan to answer

forms.gle/MmDQEFBLaqSTjkCm8

chart: docs.google.com/forms/d/e/1FAIpQLSe6ULaZ2AbL0nqH2QodJl9cJWas3N6XlaAJ4JPpO7jzyawL5A/viewanalytics

Quiz 10.

Write a few sentences about the pros and cons of using local workstealing queues over the global worklist.

Free response: a few sentences on the form

Scan to answer

forms.gle/VdDu2GrUrNz7noef9

chart: docs.google.com/forms/d/e/1FAIpQLSf7H1ZVBLzzNO-8XiHyBSIcF56WEH8PW5BMuFJ-d8tDuC9fNw/viewanalytics

Quiz 10.

Write a few sentences about the pros and cons of using local workstealing queues over the global worklist.

Free response: a few sentences on the form

Simpler vs. More efficient.

Scan to answer

forms.gle/VdDu2GrUrNz7noef9

chart: docs.google.com/forms/d/e/1FAIpQLSf7H1ZVBLzzNO-8XiHyBSIcF56WEH8PW5BMuFJ-d8tDuC9fNw/viewanalytics

Open mp

- Code examples

GPUs: a brief history

First chapter of CUDA by Example
https://www.techspot.com/article/650-history-of-the-gpu/

The very beginning

- Specialized hardware to accelerate graphics rendering

- One of the first real-time computers: Whirlwind 1 at MIT (1951)
  - Flight simulator for bombers
  - Vector graphics

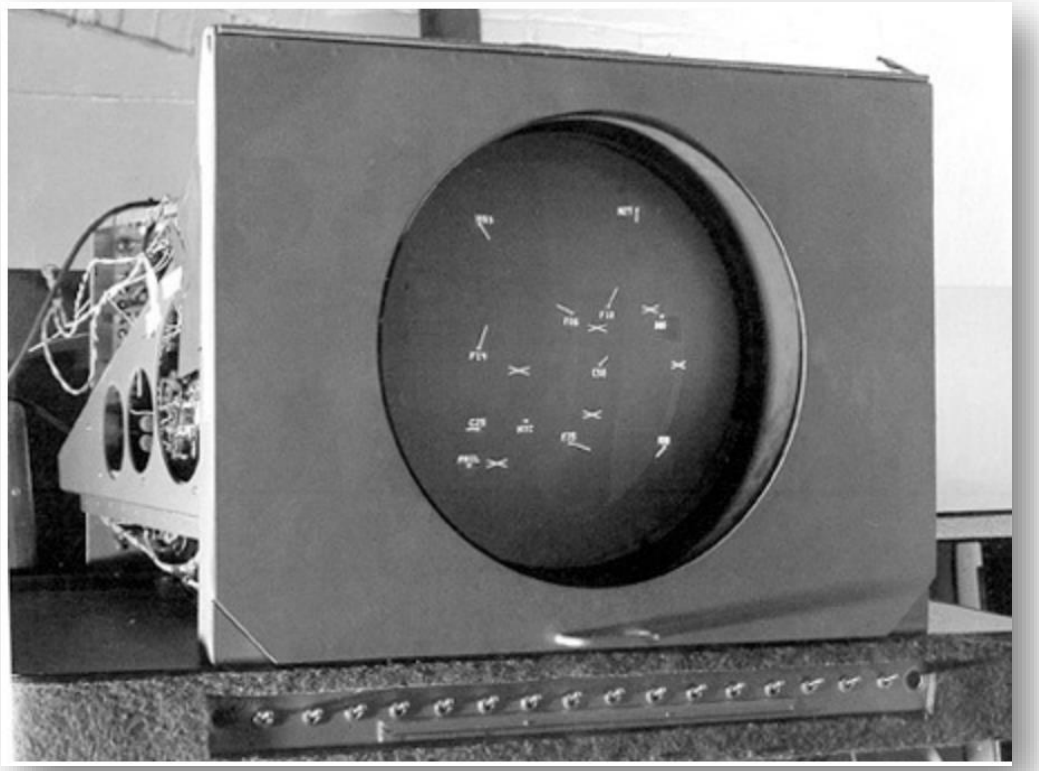


Image from: https://ohiostate.pressbooks.pub/graphicshistory/chapter/2-1-whirlwind-and-sage/

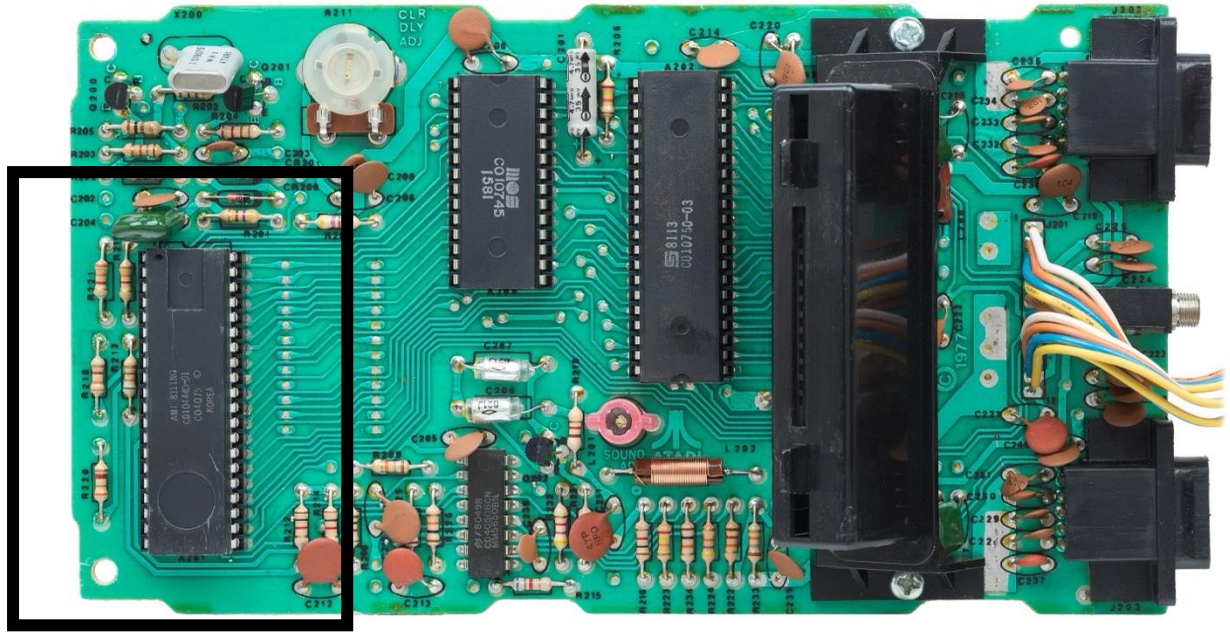
Specialization

- Next 30 years, specialized hardware for specialized software to display 2D graphics

- Specialized
  - Typically ran specific programs
  - Portability was not a top priority
  - Even the idea of portable ISAs for CPUs were not mainstream

Multi-program devices

- 1977: Television Interface Adapter
  - One of the first (and widely produced) portable (i.e. multiple program) GPUs





from: https://en.wikipedia.org/wiki/Television_Interface_Adaptor

River Raid Game



https://en.wikipedia.org/wiki/DirectX

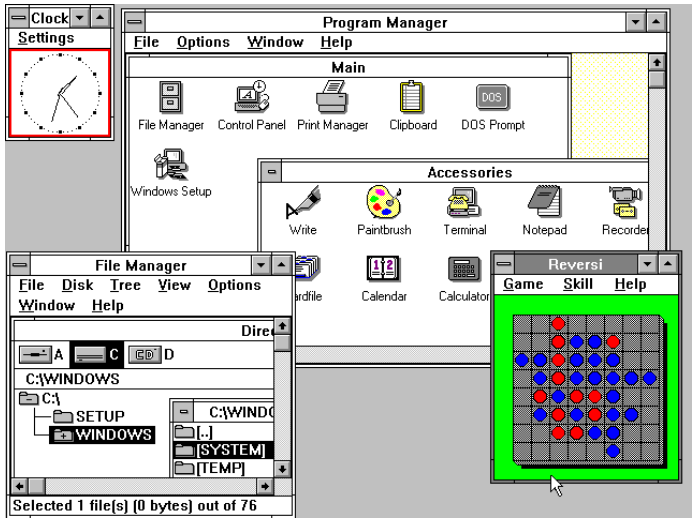
https://en.wikipedia.org/wiki/Microsoft_Windows

https://en.wikipedia.org/wiki/OpenGL

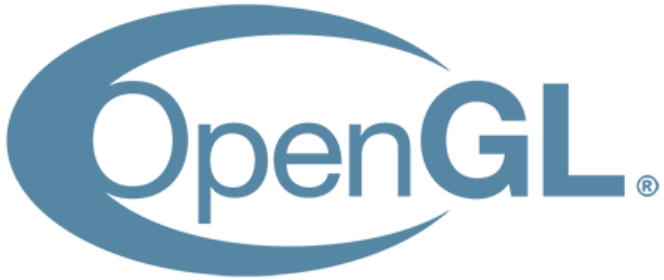
OS integration

- 1990s: Windows: a graphical operating systems, required chips to support 2D graphics.

- New APIs started appearing, to enable GUI programs



Windows 3 (1990)



1992



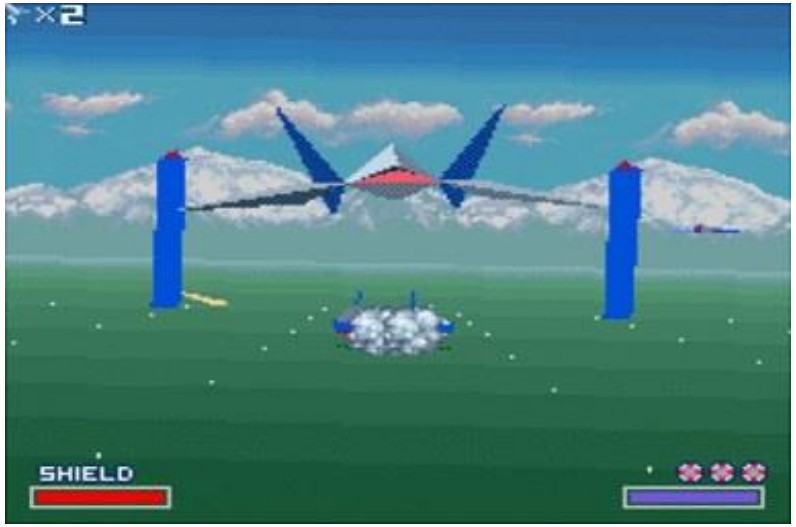
1995

3D graphics in consoles (1993)

- Super Nintendo was not powerful enough to draw 3D graphics
- Shigeru Miyamoto (Super Mario developer) really wanted a 3D flight simulator though
- Worked with a British software company to develop...

3D graphics in consoles (1993)

- Super Nintendo was not powerful enough to draw 3D graphics
- Shigeru Miyamoto (Super Mario developer) really wanted a 3D flight simulator though
- Worked with a British software company to develop…





https://en.wikipedia.org/wiki/Star_Fox_(1993_video_game)

3D graphics in consoles (1993)

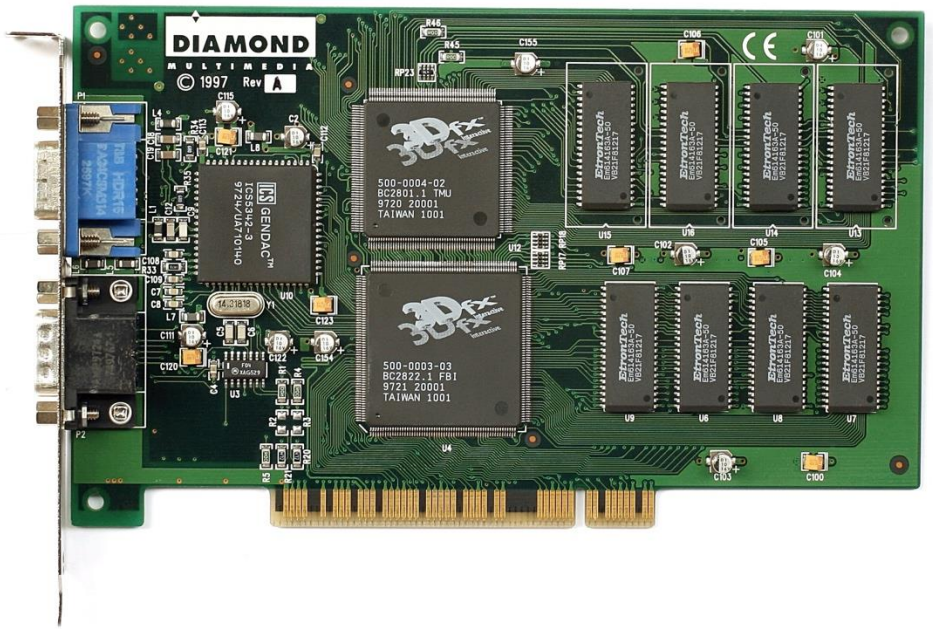
- Game cartridges shipped with a “mini GPU” on them:
  - the Super FX



https://twitter.com/gameminesocials/status/1322946537077526528?s=20

3D graphics acceleration

- 1996 : First 3D graphics accelerator: Vodoo from 3Dfx
  - Discrete GPU
  - Early 3D games: e.g. tomb raider
  - Acquired by Nvidia in 2002



https://en.wikipedia.org/wiki/3dfx_Interactive

3D graphics acceleration

- 3D accelerators continued, many companies competing:
  - Nvidia
  - ATI (acquired by AMD)
  - 3Dfx
  - and more...
- Next milestone in 1999:
  - Nvidia coins the term “GPU”
  - Compare with modern website

https://web.archive.org/web/20030814003456/www.nvidia.com/object/gpu.html

Programmable 3D accelerators

- 2001: Microsoft DirectX 8 required programmable vertex and pixel shaders.

- 2001: First GPU to satisfy the requirement was Nvidia GeForce 3
  - Used on the original Xbox

- Programmers started writing general programs for these GPUs:
  - Present your data as a graphical input (e.g. Textures and Triangles)
  - Read the output after a series of “graphics” API calls

GPGPU Programming

- 2006: Nvidia releases CUDA: programming language for their GPUs
  - Supported by 8th generation CUDA devices.
  - Integrated vertex and pixel cores into “shader cores”
  - Support for IEEE floating point

- Soon after...

GPGPU Programming

- 2006: Nvidia releases CUDA: programming language for their GPUs
  - Supported by 8th generation CUDA devices.
  - Integrated vertex and pixel cores into “shader cores”
  - Support for IEEE floating point
- Soon after...
- 2008: The Khronos Group launches OpenCL for cross vendor GPGPU:
  - including AMD, Intel, Qualcomm

Khronos Group



- Started in 2000 by Apple as a standards body for graphics API to counter Microsoft's DirectX:
  - A way to unify APIs across many different vendors
  - at the time: ATI, Nvidia, Intel, Sun Microsystems (and a few others)
  - now: Many companies, including AMD, Nvidia, Intel, Qualcomm, ARM, Google

  - OpenGL is maybe the biggest standard they maintain (for graphics)
  - OpenCL is big for compute
  - Vulkan unifies compute + graphics (mostly graphics)

- Apple deprecated Khronos group standards to support Metal in 2018
  - iPhone used to have GPUs from a company called Imagination, a British company. Apple took their engineers.
  - Our laptops may have GPUs from Intel or ADM. Now apple laptops has apple GPUs.

https://en.wikipedia.org/wiki/Khronos_Group

Where are we now?

- Nvidia CUDA is widely used, driving many HPC and ML applications
  - Nvidia heavily invested in its software stack and made it accessible, and this made it widely used.
  - Nvidia GPUs are not designed power efficient, and are not used much in mobile devices.
- OpenCL is used to program other GPUs (although it is not as widely used)
- Metal is used for Apple devices
- Vulkan has momentum, but mostly in graphics

- New GPGPU programming languages are on the horizon:
  - WebGPU - a javascript interface to unite Metal, Vulkan and DirectX
  - Its ambitious! Will it work?!

GPU Shortages?

- Cryptocurrency:
  - 2018 reported tripling of GPU prices and shortages due to increase demand from miners.
  - Still happening with lots of market fluctuations.
  - Still plenty of GPUs in your phone, laptop, etc. ☺

GPUs now

- Nvidia is #2 most valuable company in the world at roughly $4.94 trillion,
  - More valuable than Microsoft, Alphabet and Amazon
- Graphics are still a valuable domain, but AI seems to be more valuable
  - Valuable to learn general purpose GPU programming (GPGPU)
- Many companies make their own GPUs:
  - AMD, Apple, Qualcomm, ARM, Intel, Imagination, Broadcom
- Many startups in the Valley making new AI chips similar to GPUs

Teaching GPU programming

- This is difficult!
- Nvidia GPUs have the most straightforward programming model (CUDA).
- It is extremely difficult to get a class of students access to Nvidia GPUs these days.
  - AWS? Expensive and often oversubscribed w.r.t. GPUs
  - Department? ML folks get priority and super computing clusters are painful

Plan

- The GPU programming lectures will use CUDA
  - It is widely used
  - The programming model is straightforward

- Homework will use WebGPU, because it is widely supported
  - *There are more non-Nvidia GPUs in this room than Nvidia GPUs*

Plan

- The homework uses Javascript as its ”CPU” language, and WebGPU as its ”GPU” language.

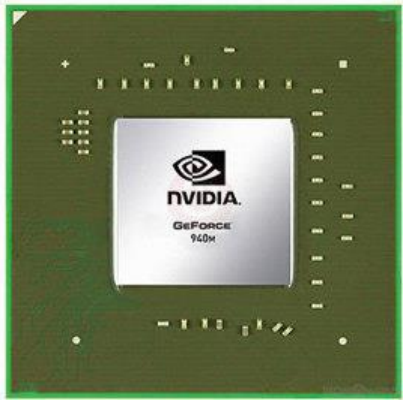
- We have provided generous skeletons for the homework. We can go over some javascript, but it is a high-level language and should not be hard to figure out what you need to do.

- The WebGPU portion is straightforward, and I will provide a mapping directly from what we talked about to what you need.

Start on GPU lectures

Programming a GPU

Tiny GPU in an embedded system



Fight!



The CPU



Nvidia Jetson Nano (whole chip, CPU + GPU)
2 Billion transistors
10 TDP
Est. $99

Intel i7-9700K
2.16 Billion transistors
95 TDP
Est. $316

https://www.techpowerup.com/gpu-specs/geforce-940m.c2648
https://www.alibaba.com/product-detail/Intel-Core-i7-9700K-8-Cores_62512430487.html
https://www.prolast.com/prolast-elevated-boxing-rings-22-x-22/

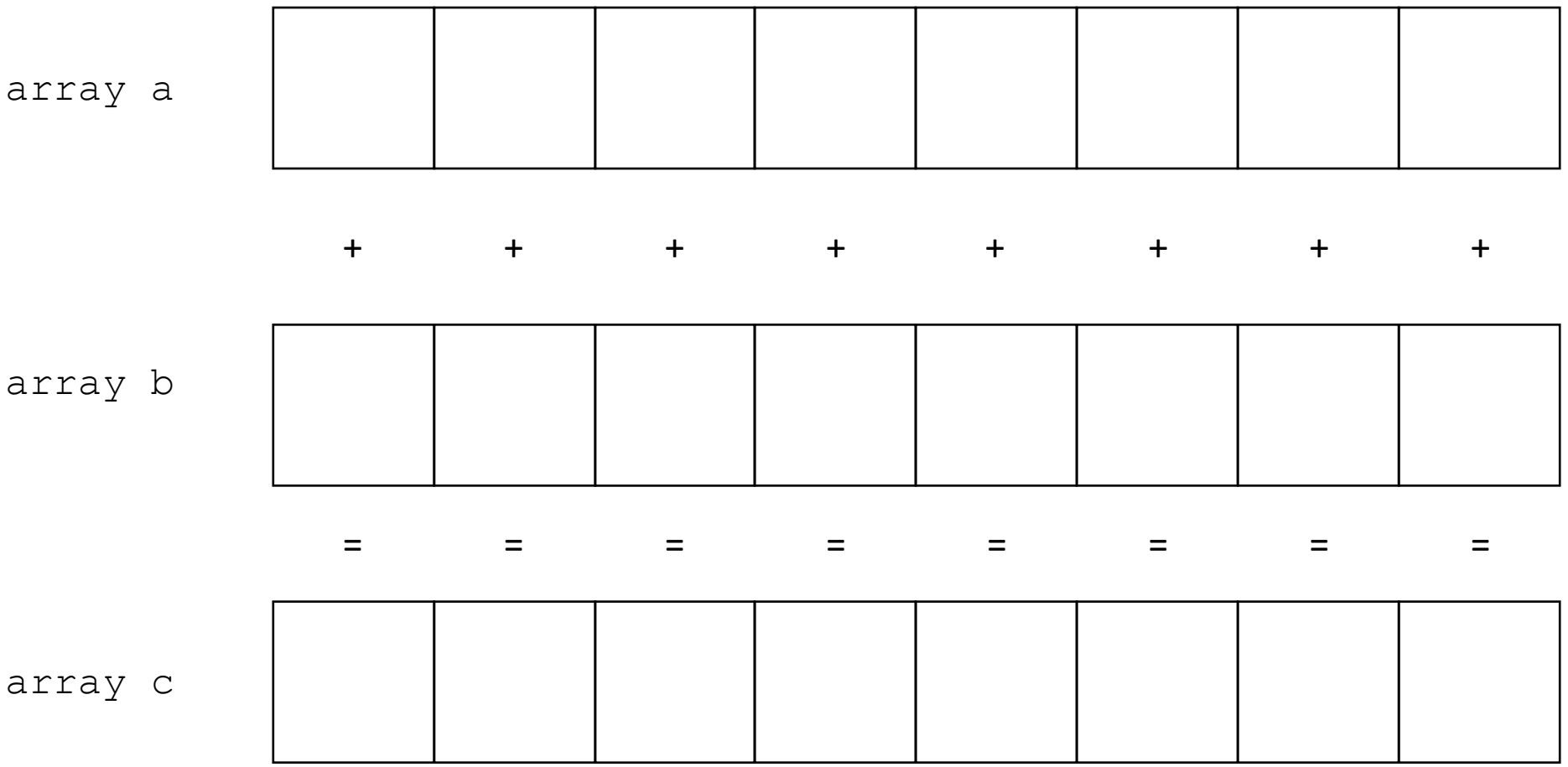
Programming a GPU

- The problem: Vector addition

Embarrassingly parallel

Computation can easily be divided into threads

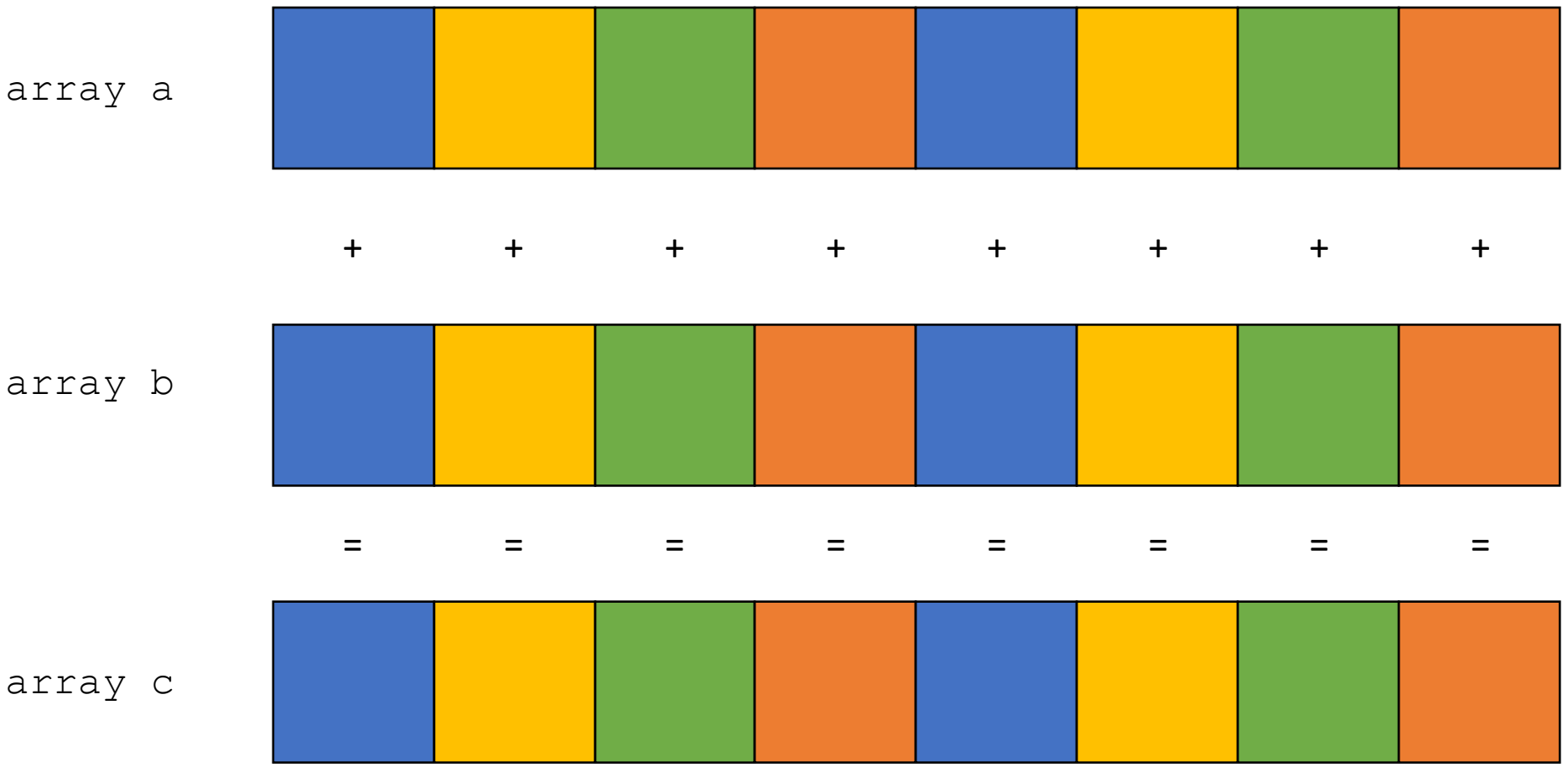
Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Parallel Schedules

Computation can easily be divided into threads

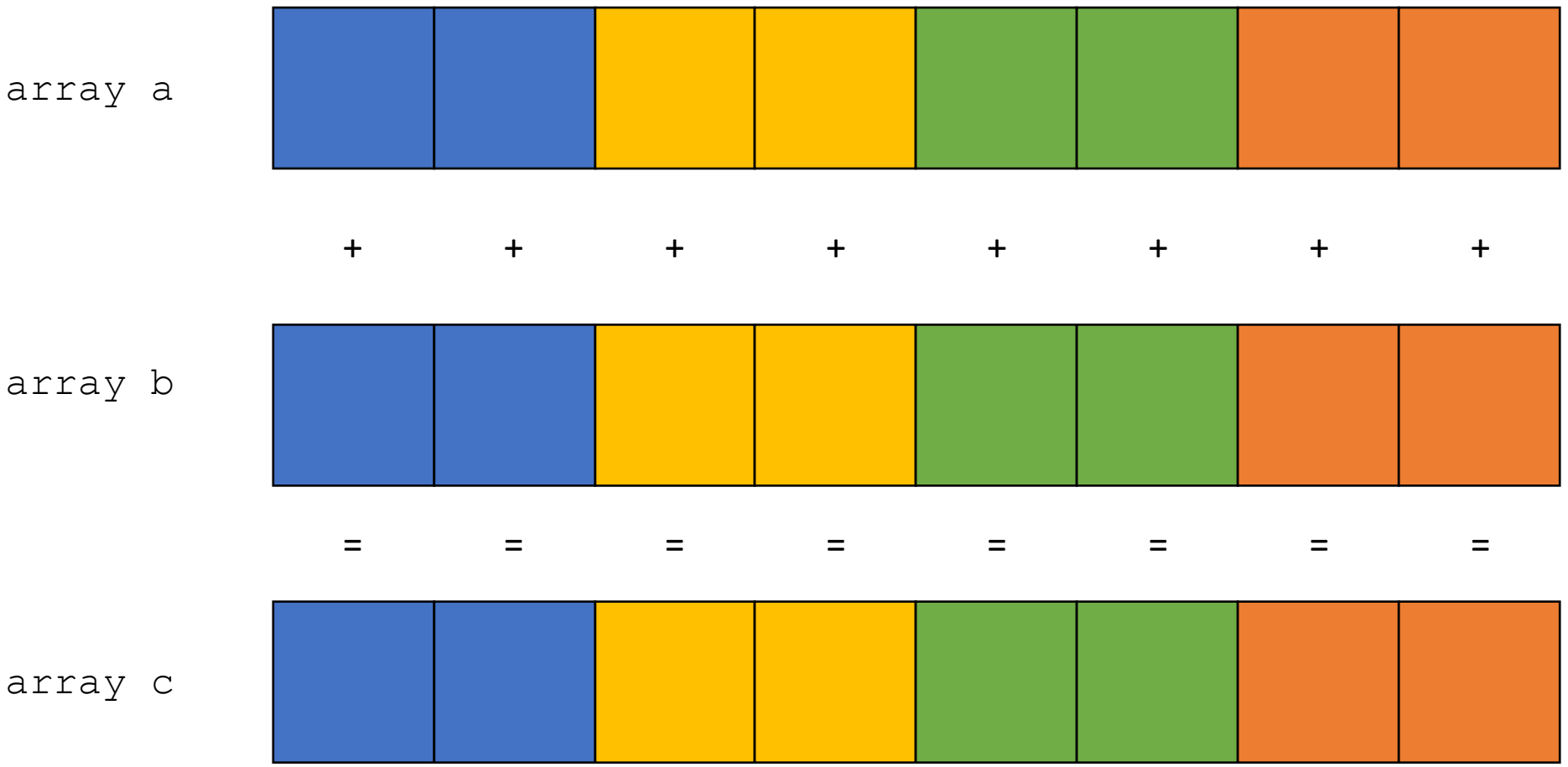
Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Parallel Schedules

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Programming a GPU

- The problem: Vector addition

- Who can do it faster?

Lets set up the CPU

```
void vec_add(double *a, double *b, double *c, int size,
int tid, int num_threads) {
  int chunk = size / num_threads;
  int start = chunk * tid;
  int end = start + chunk;
  for (int j = 0; j < OUTER; j++) {
    for (int i = start; i < end; i++) {
      a[i] = b[i] + c[i];
    }
  }
}
```

```
for (int i = 0; i < num_threads; i++)
  thread_array[i] = thread(vec_add, a, b, c, SIZE, i,
num_threads);

for (int i = 0; i < num_threads; i++)
  thread_array[i].join();
```

Now for the GPU

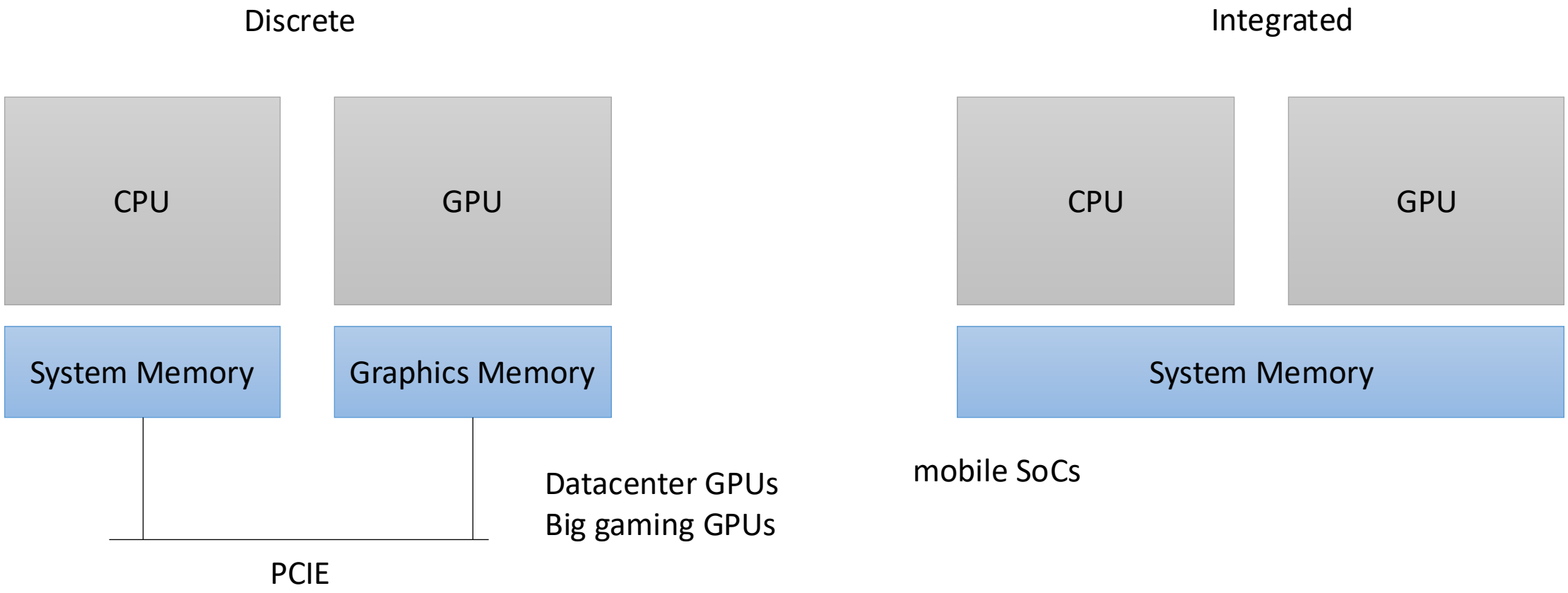
- Its going to take a bit of work....

GPU set up

- We need to allocate and initialize memory

GPU set up

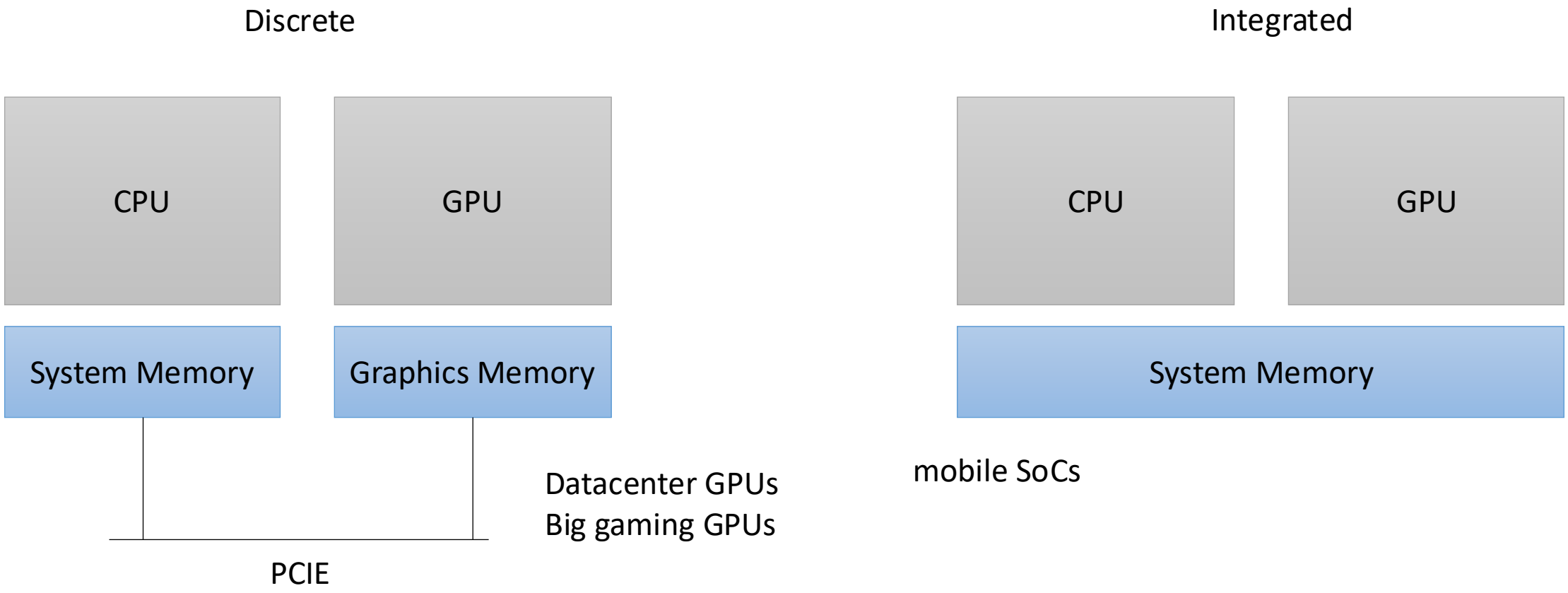
- GPUs come in two flavors



GPU set up

Pros and cons of each?

- GPUs come in two flavors

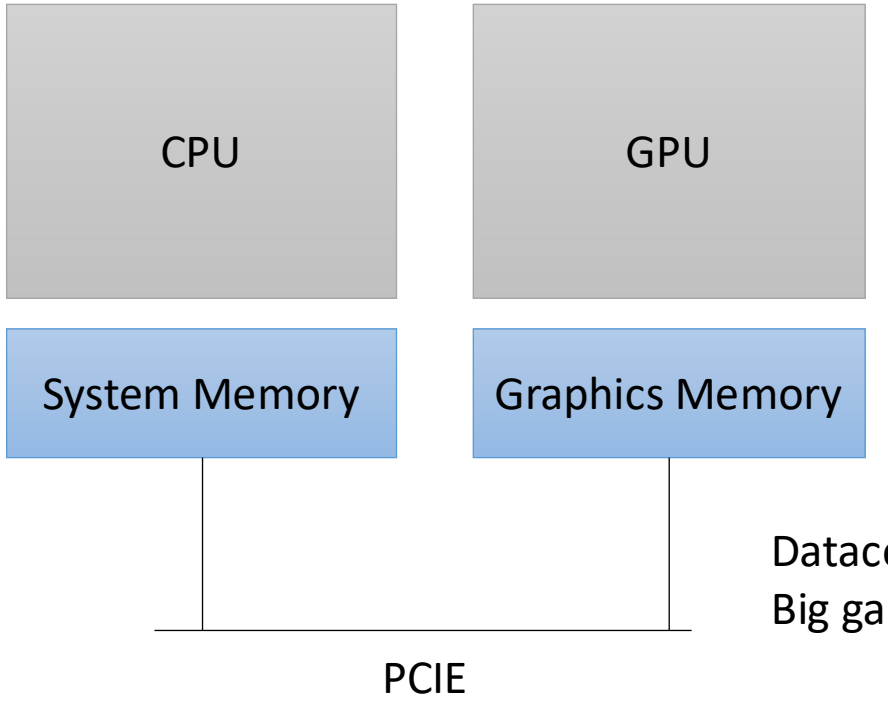


GPU set up

- GPUs come in two flavors

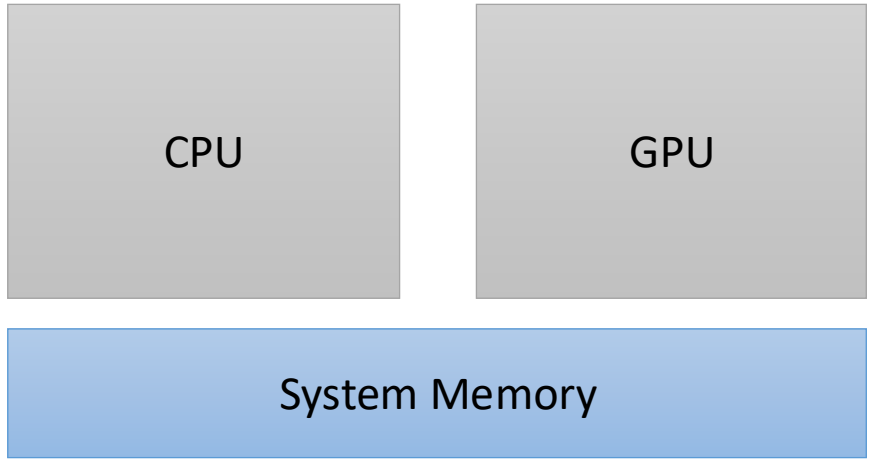
Pros and cons of each?
*Different types of memory for discrete
*Swappable for discrete
*More energy efficient for integrated
*Better memory utilization for integrated

Discrete



Datacenter GPUs
Big gaming GPUs

Integrated



mobile SoCs

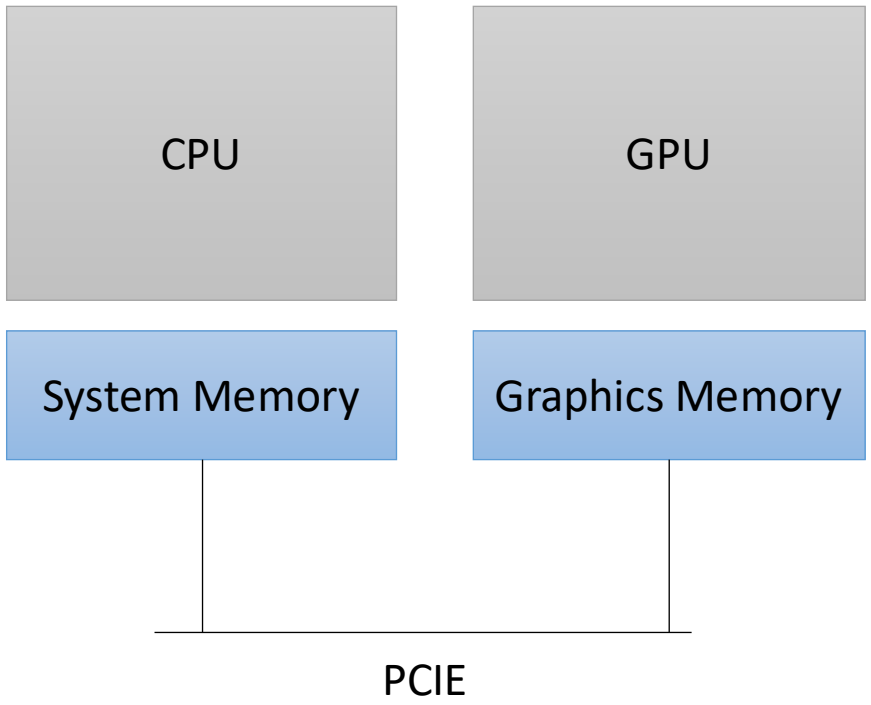
GPU set up

- GPUs come in two flavors

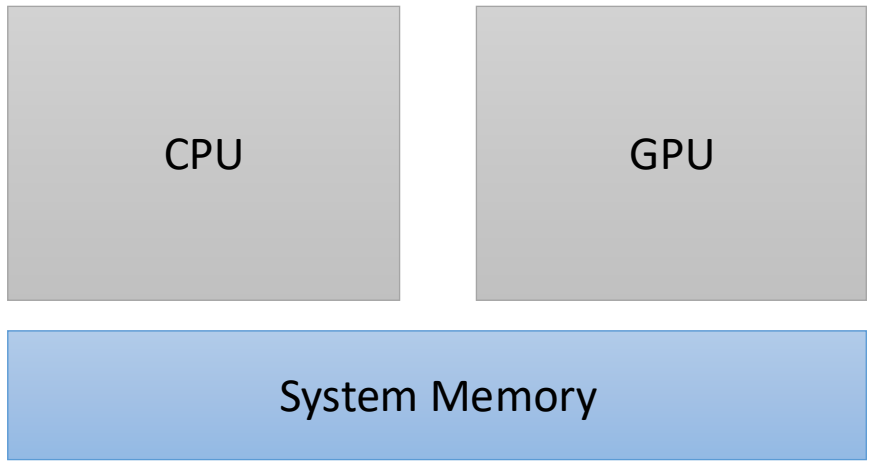
Although mobile GPUs share the system memory, Most still require you to program as if they didn't have shared memory.

Why?

Discrete



Integrated



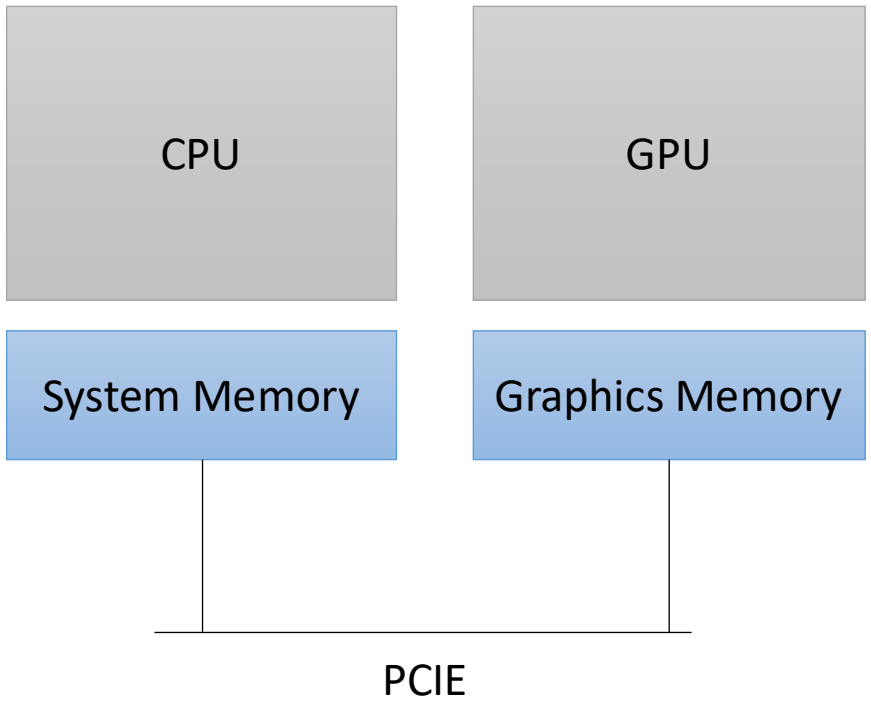
GPU set up

- GPUs come in two flavors

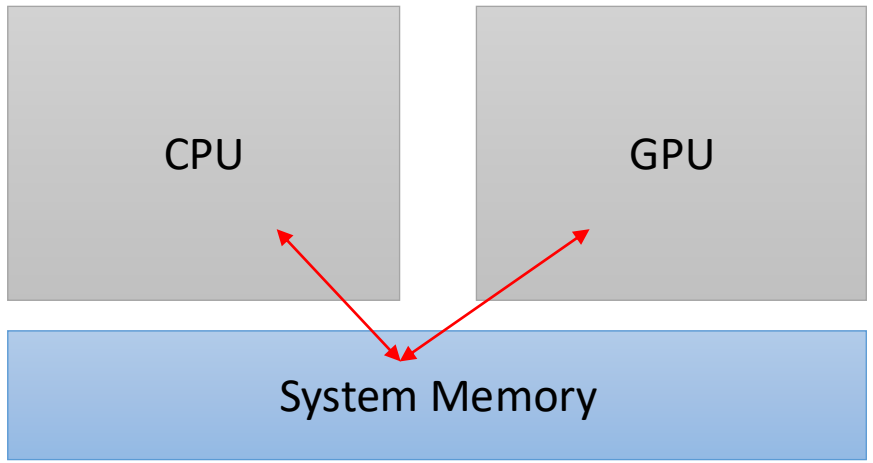
Although mobile GPUs share the system memory, Most still require you to program as if they didn't have shared memory.

Why?

Discrete



Integrated



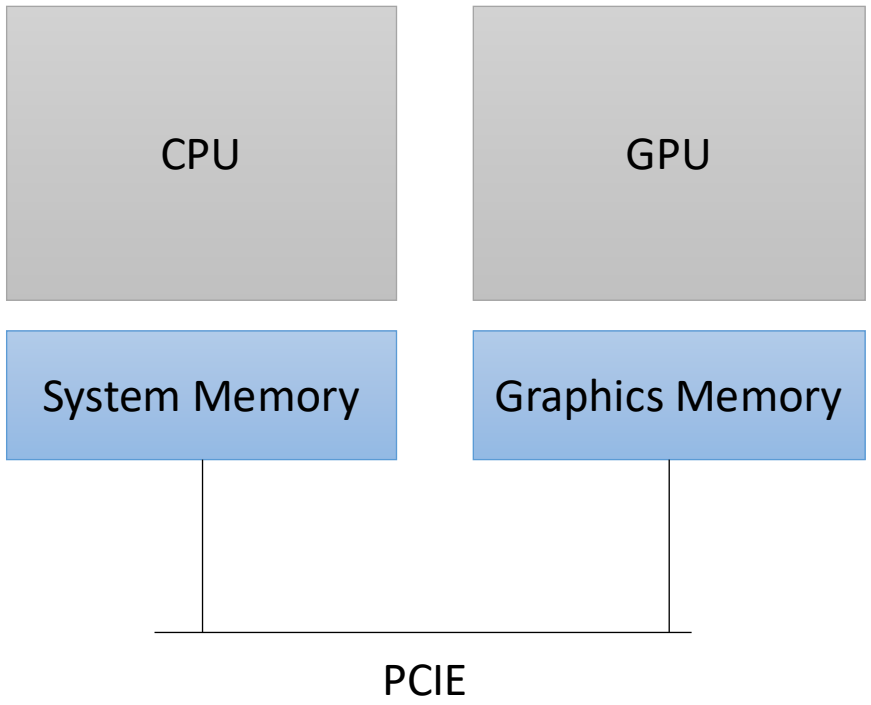
GPU set up

- GPUs come in two flavors

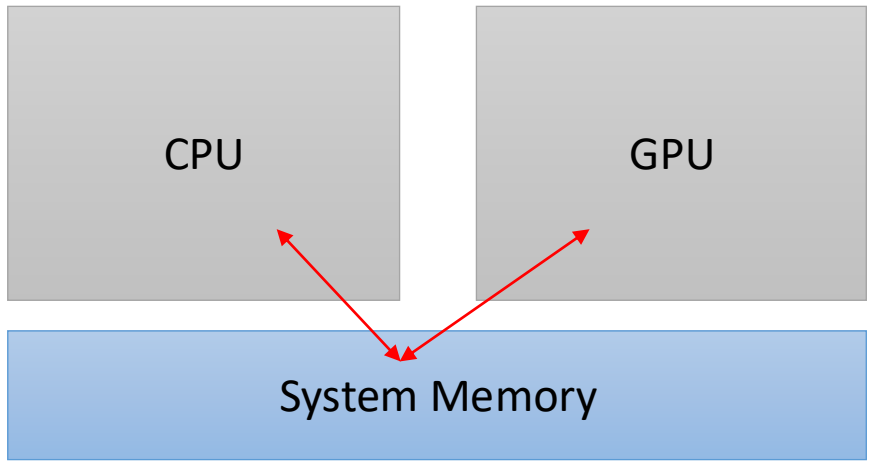
Although mobile GPUs share the system memory, Most still require you to program as if they didn't have shared memory.

Why?

Discrete



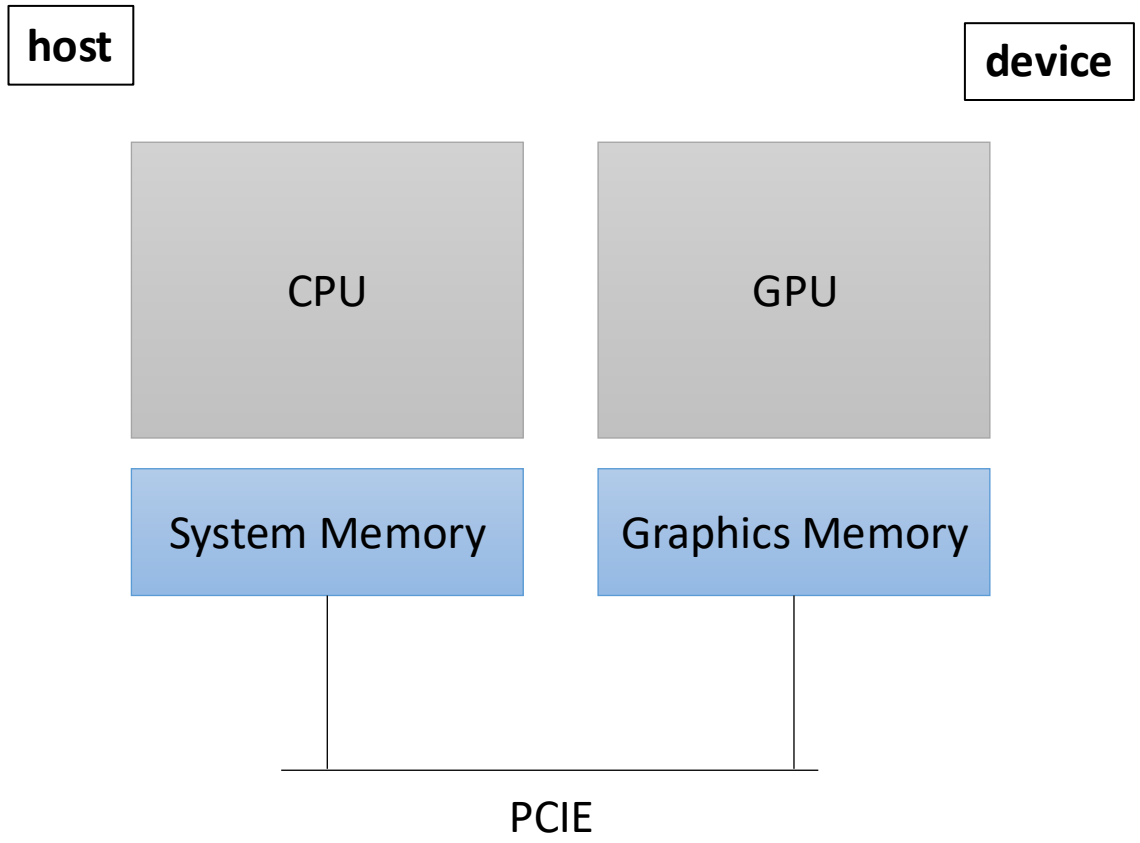
Integrated



In many cases, CPU-GPU communication is not fully supported. Coherence, fences, and RMWs might not be supported.

GPU set up

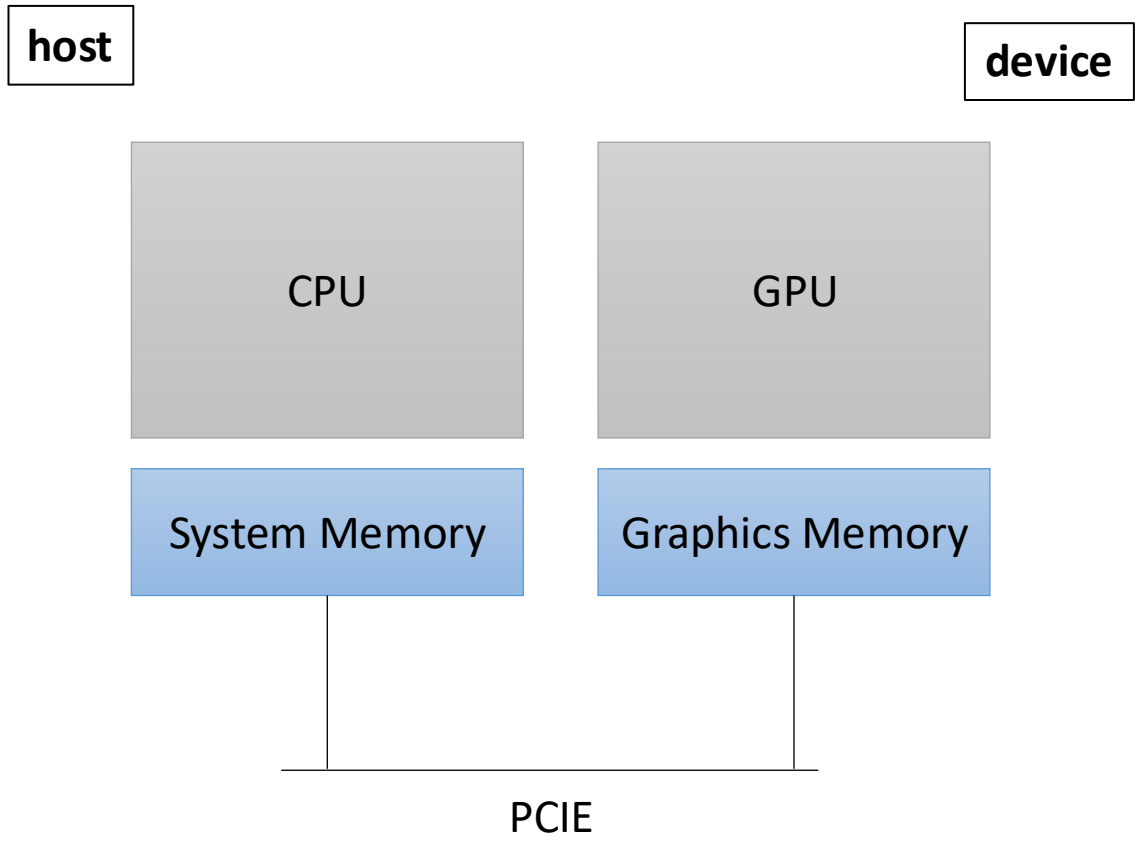
- Our heterogeneous, parallel, programming model



GPU set up

- Our heterogeneous, parallel, programming model

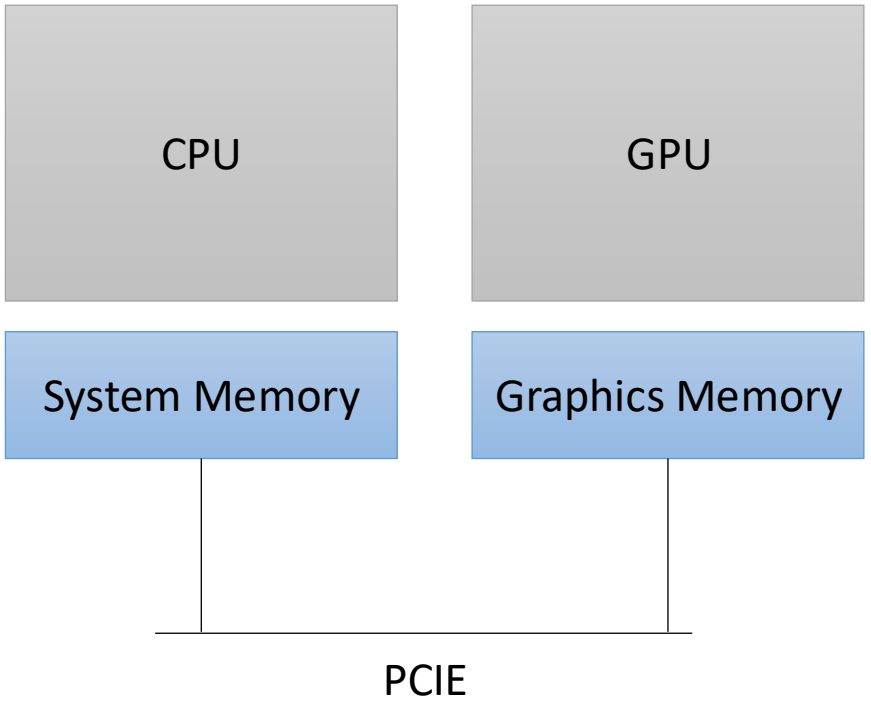
The host (CPU) will write a C++-like program that allocates and sets up memory on the GPU. The host will then call a GPU program called a kernel.



GPU set up

How do we allocate memory on a CPU?

- Our heterogeneous, parallel, programming model

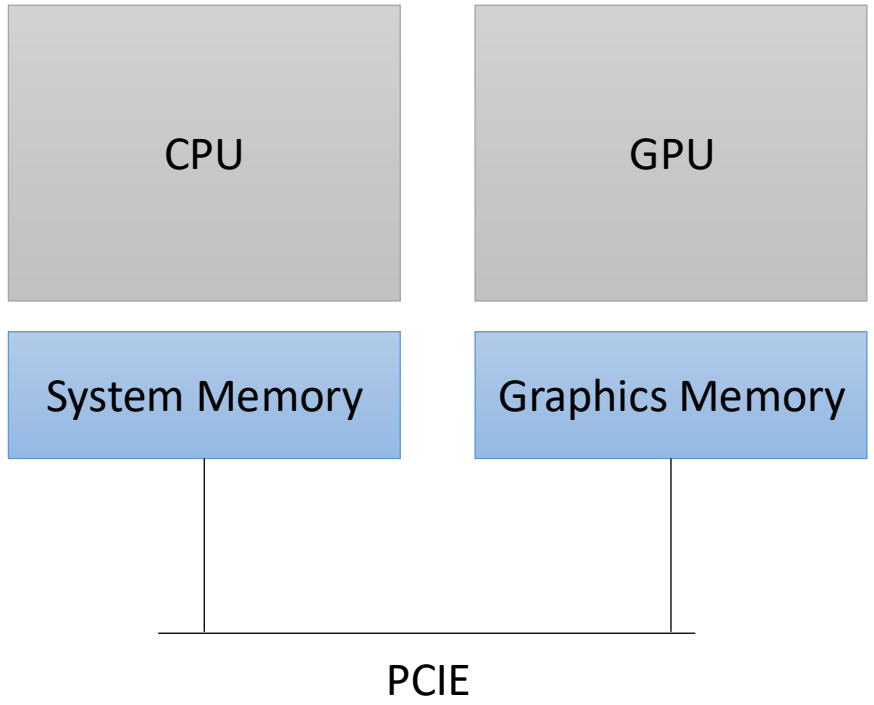


GPU set up

How do we allocate CPU memory on the host?

- Our heterogeneous, parallel, programming model

```
int *x = (int*) malloc(sizeof(int)*SIZE);
```

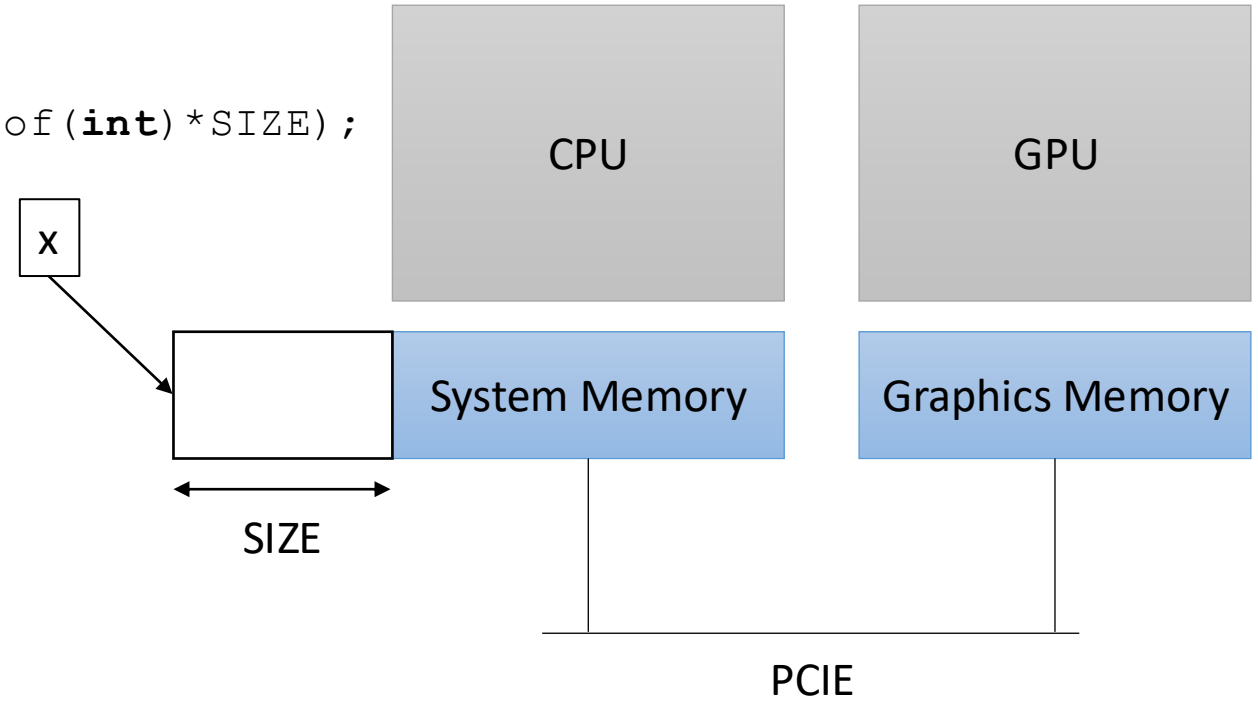


GPU set up

How do we allocate CPU memory on the host?

- Our heterogeneous, parallel, programming model

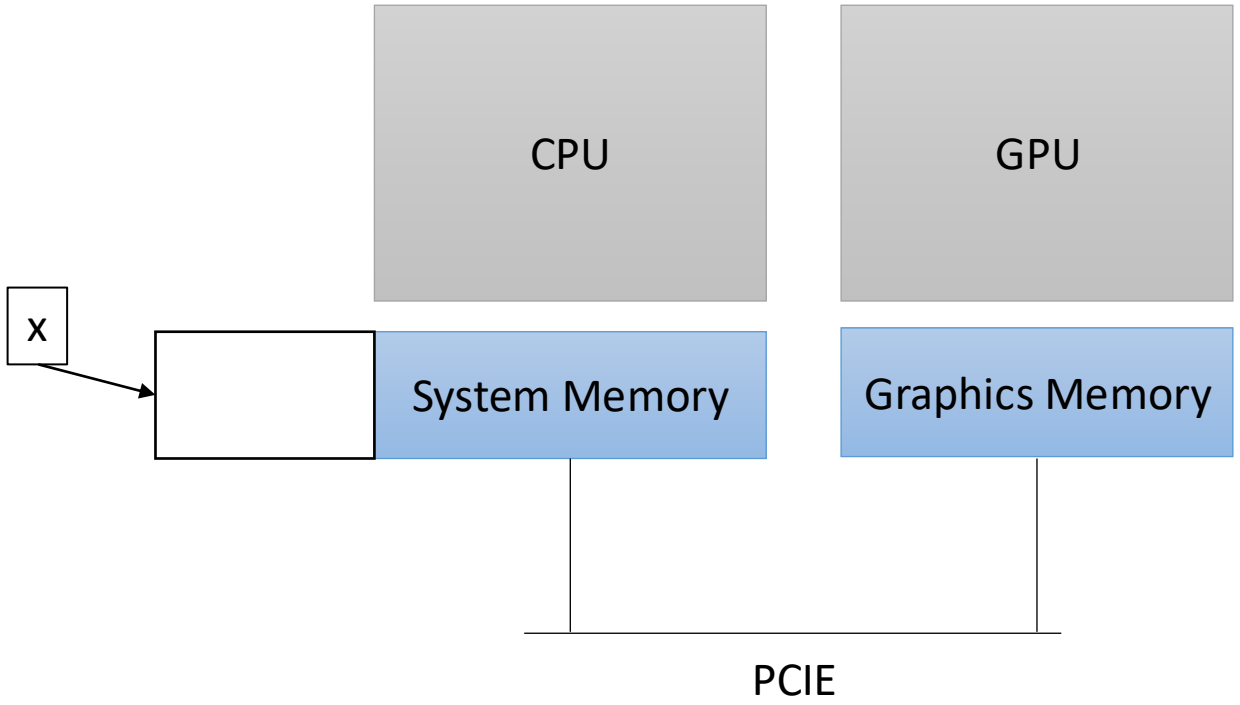
```
int *x = (int*) malloc(sizeof(int)*SIZE);
```



GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

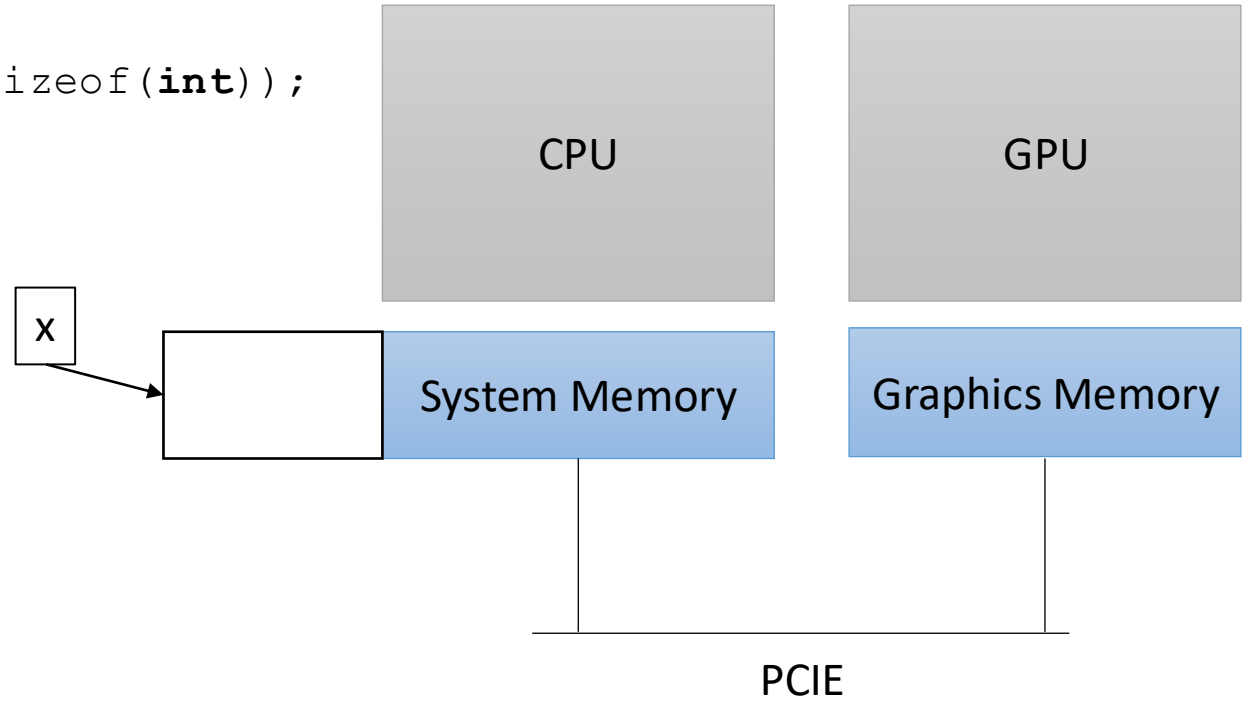


GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

```
int *d_x;
cudaMalloc(&d_x, SIZE*sizeof(int));
```

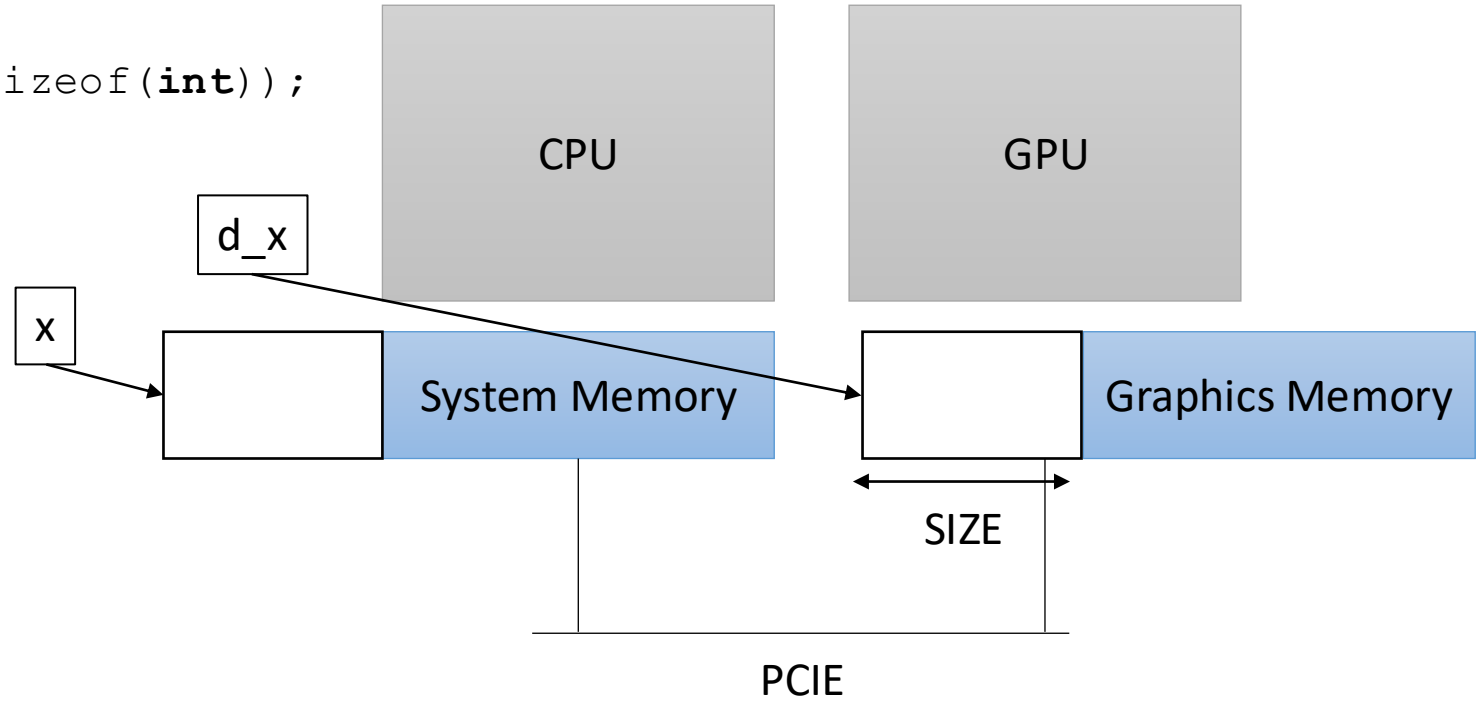


GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

```
int *d_x;
cudaMalloc(&d_x, SIZE*sizeof(int));
```

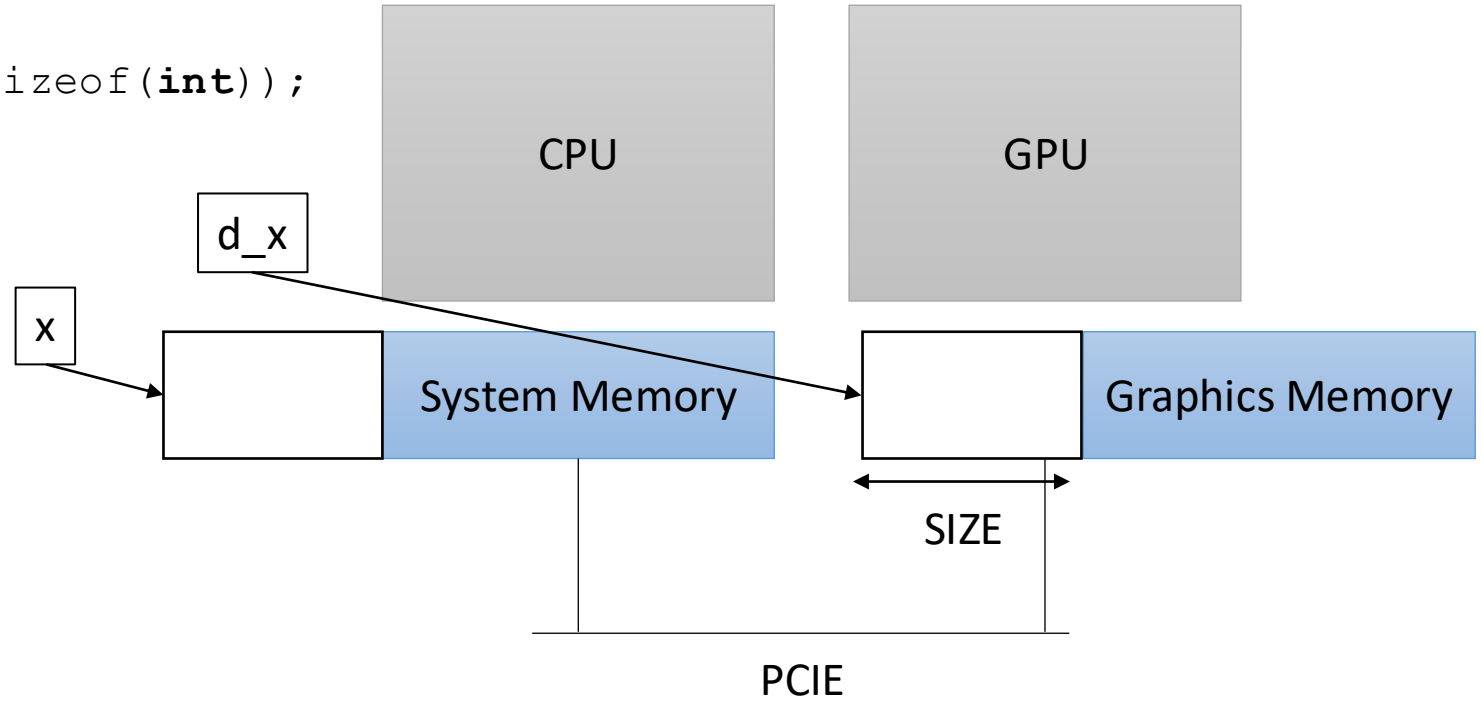


GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

```
int *d_x;
cudaMalloc(&d_x, SIZE*sizeof(int));
```

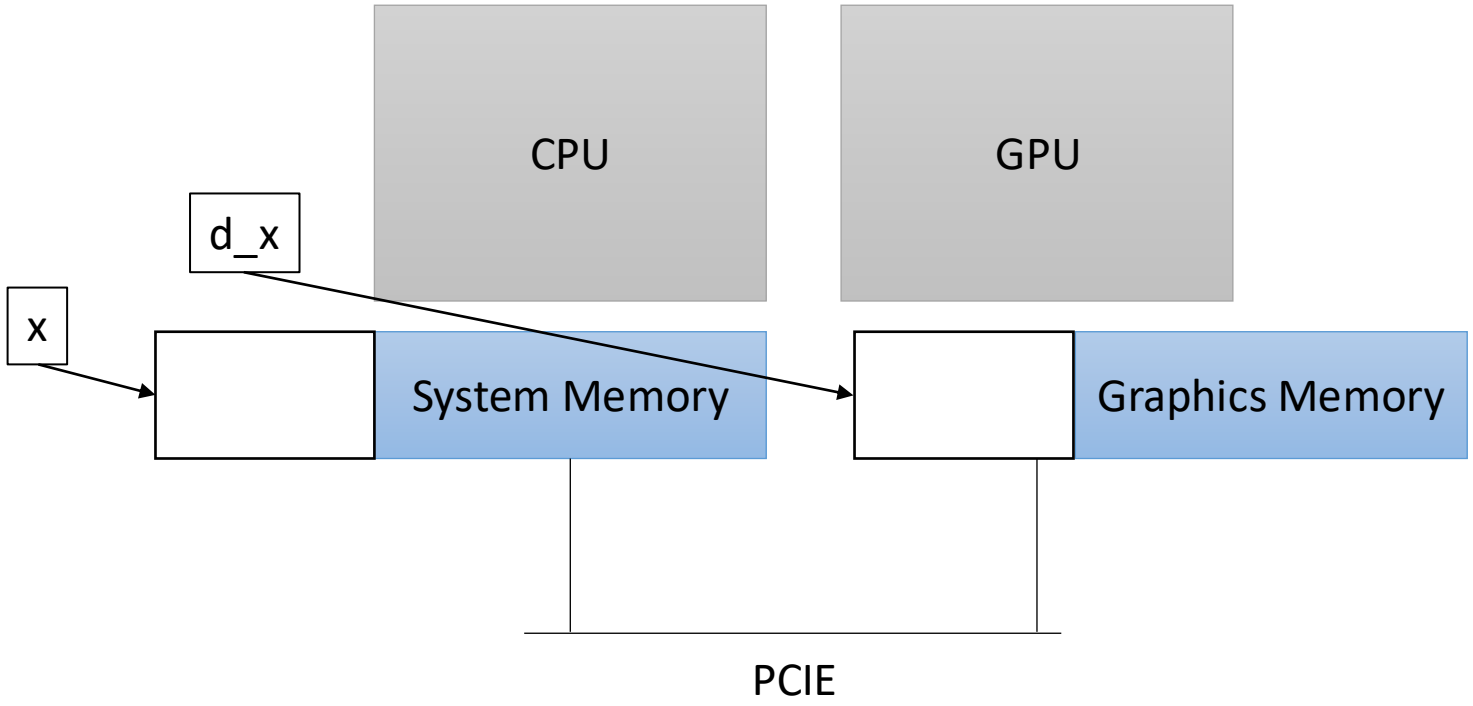


`d_x` is a pointer, in the CPU program, that points to memory on the GPU.

We can pass the pointer around, but the CPU cannot access the data i.e. `d_x[0]` gives an error!

GPU set up

- Our heterogeneous, parallel, programming model

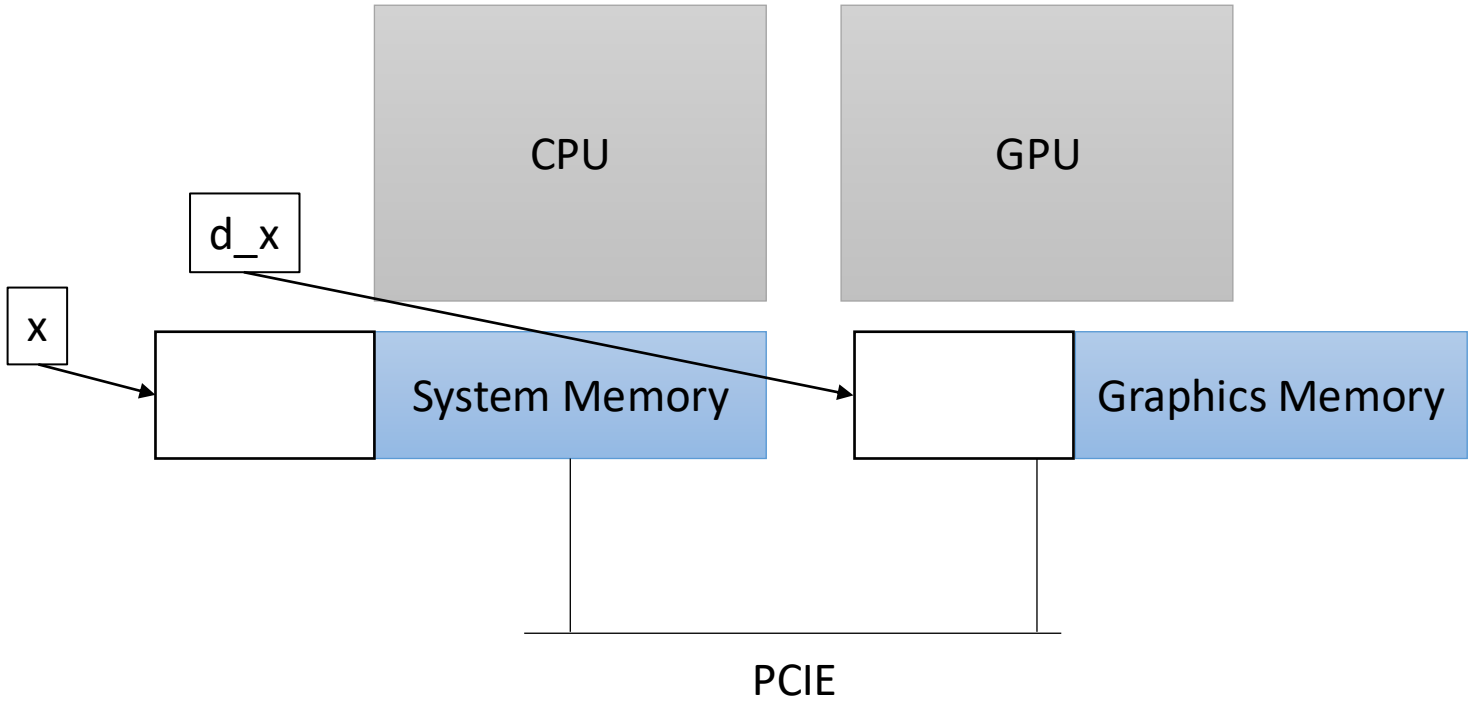


GPU set up

- Our heterogeneous, parallel, programming model

If we can't access d_x on the CPU, how do we initialize the memory?

GPU has no access to input devices e.g. disk



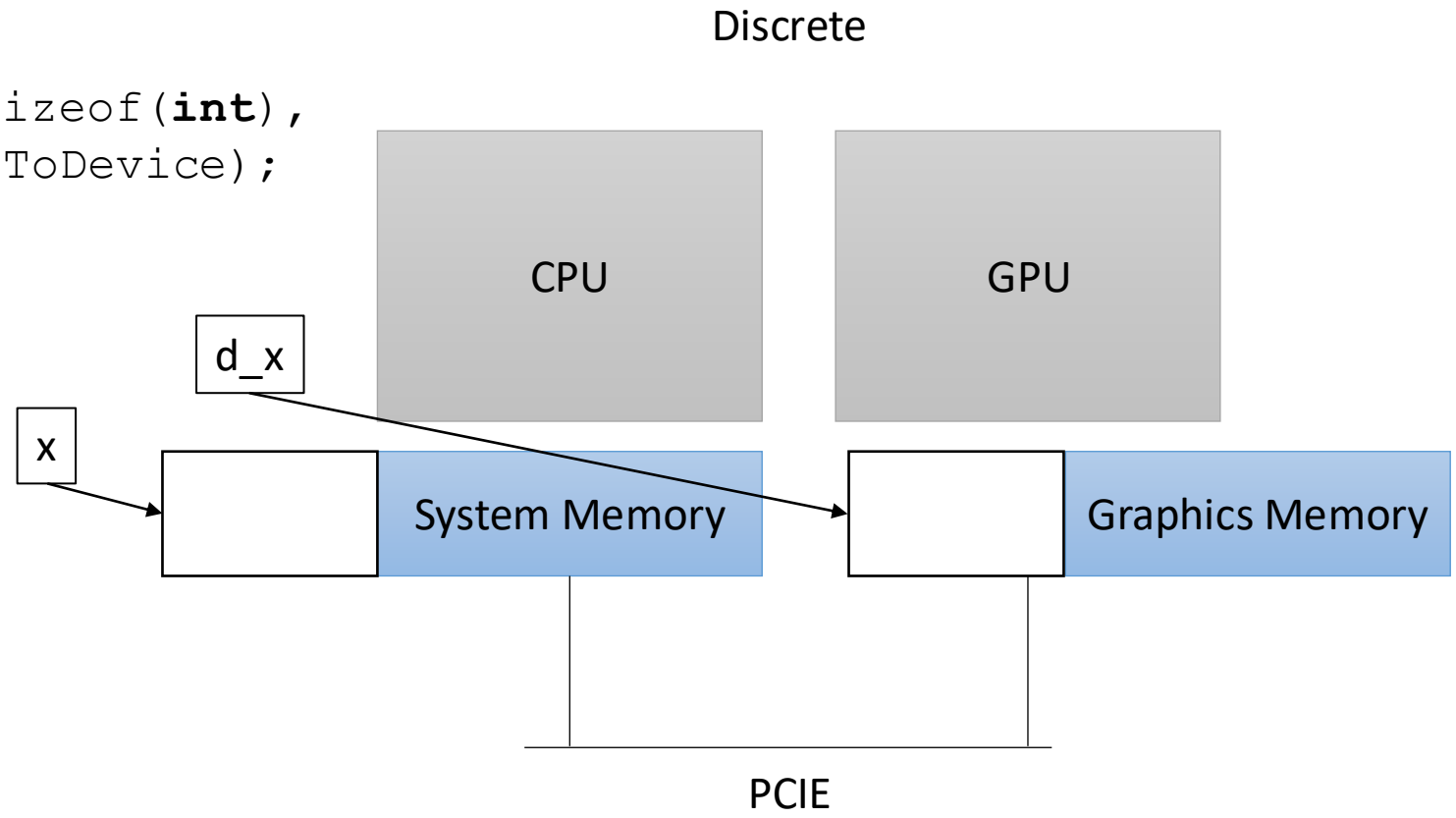
GPU set up

- Our heterogeneous, parallel, programming model

If we can't access d_x on the CPU, how do we initialize the memory?

GPU has no access to input devices e.g. disk

```
//initialize x on host
cudaMemcpy(d_x, x, SIZE*sizeof(int),
           cudaMemcpyHostToDevice);
```



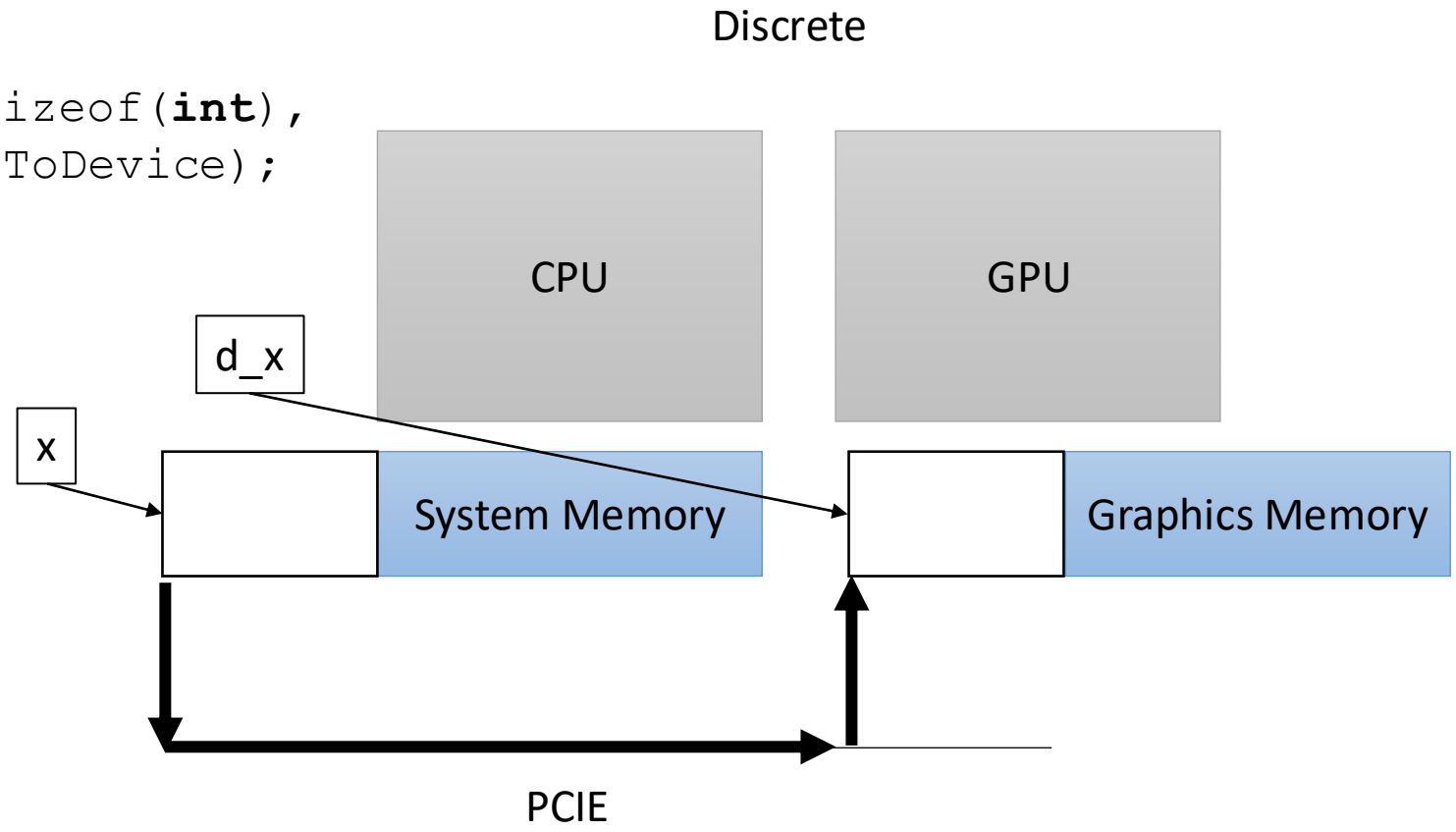
GPU set up

- Our heterogeneous, parallel, programming model

If we can't access d_x on the CPU, how do we initialize the memory?

GPU has no access to input devices e.g. disk

```
//initialize x on host
cudaMemcpy(d_x, x, SIZE*sizeof(int),
           cudaMemcpyHostToDevice);
```



How does this look in code?

```
float *a = new float[SIZE];
float *b = new float[SIZE];
float *c = new float[SIZE];

for (int i = 0; i < SIZE; i++) {
  a[i] = 0.0f;
  b[i] = i;
  c[i] = 1.0f;
}

float *d_a, *d_b, *d_c;
int e = 0;
e = cudaMalloc(&d_a, SIZE*sizeof(float));
e |= cudaMalloc(&d_b, SIZE*sizeof(float));
e |= cudaMalloc(&d_c, SIZE*sizeof(float));

e |= cudaMemcpy(d_a, a, SIZE*sizeof(float), cudaMemcpyHostToDevice);
e |= cudaMemcpy(d_b, b, SIZE*sizeof(float), cudaMemcpyHostToDevice);
e |= cudaMemcpy(d_c, c, SIZE*sizeof(float), cudaMemcpyHostToDevice);
```

The GPU Program

- Write a special function in your C++ code.
  - Called a Kernel
  - Use the new keyword `__global__`
  - Keywords in
    - OpenCL `__kernel`
    - Metal `kernel`

- Write it how you'd write any other function

The GPU Program

```
__global__ void vector_add(int * a, int * b, int * c, int size) {
    for (int i = 0; i < size; i++) {
      a[i] = b[i] + c[i];
   }
}
```

The GPU Program

```
__global__ void vector_add(int * a, int * b, int * c, int size) {
    for (int i = 0; i < size; i++) {
      a[i] = b[i] + c[i];
   }
}
```

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

The GPU Program

```
__global__ void vector_add(int * a, int * b, int * c, int size) {
    for (int i = 0; i < size; i++) {
      a[i] = b[i] + c[i];
   }
}
```

What in the world?
special new CUDA syntax. We will talk more soon

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

The GPU Program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

Pass in pointers to memory on the device

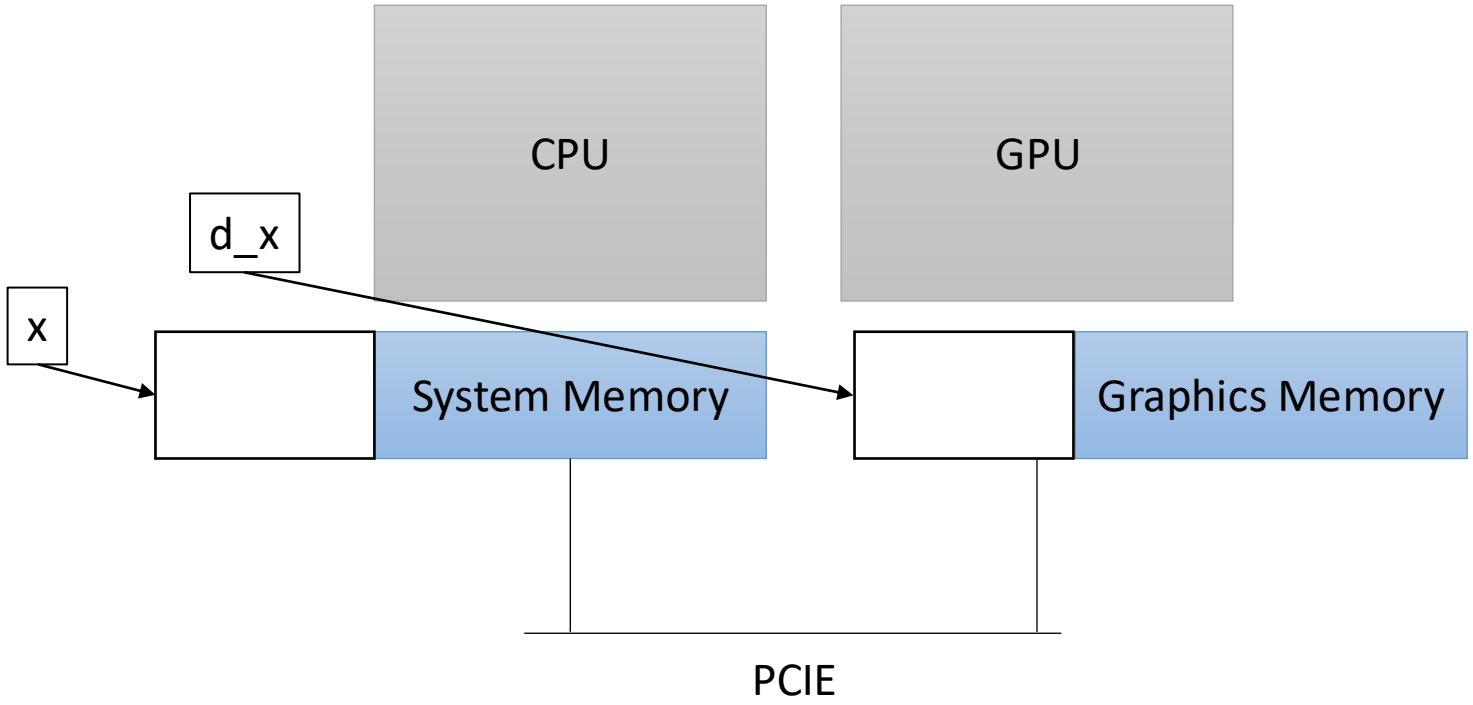
calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

GPU set up

- Our heterogeneous, parallel, programming model

Remember, GPU needs to access its own memory



The GPU Program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

Constants can be passed in regularly

calling the function

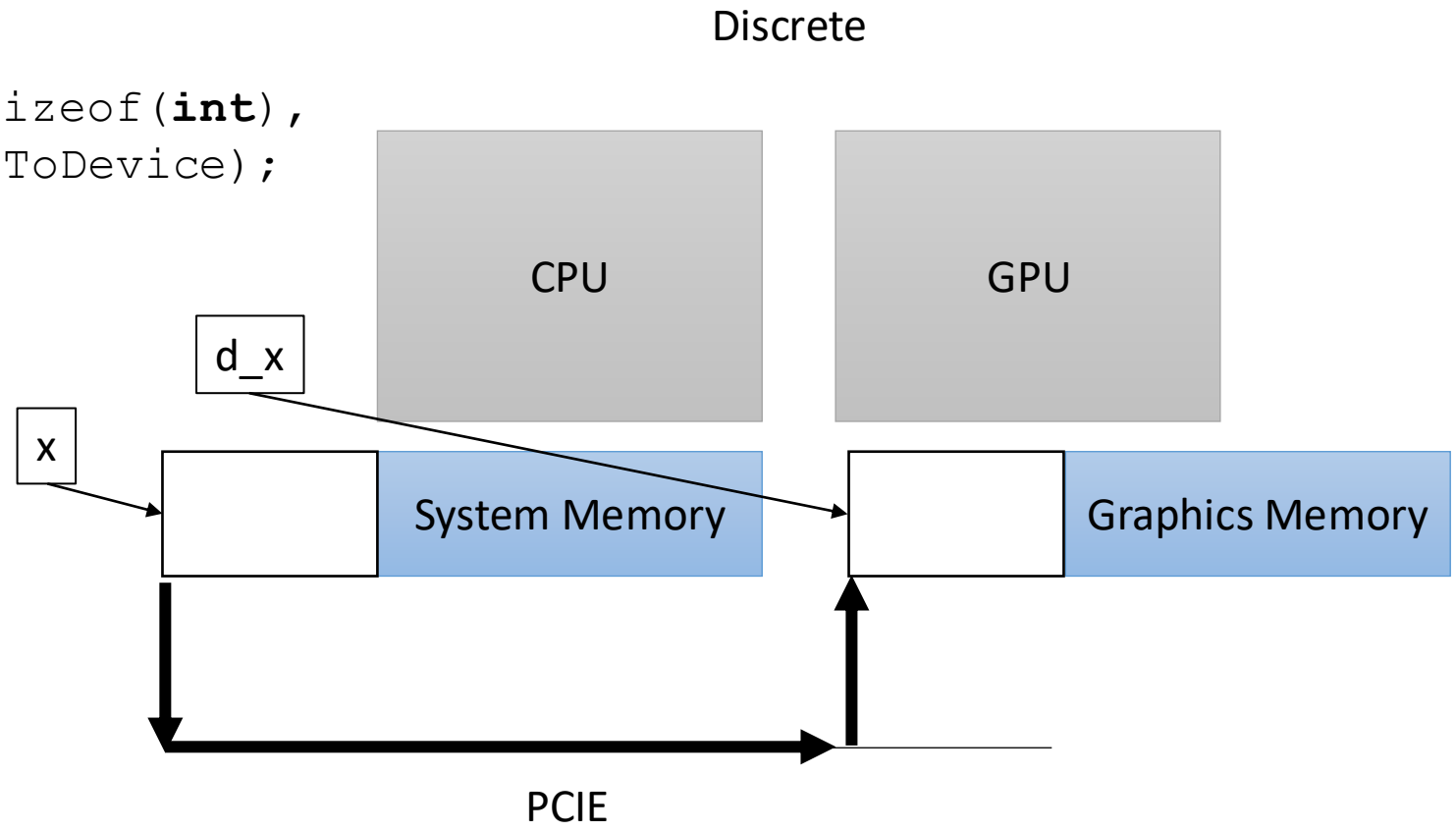
```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
e |= cudaDeviceSynchronize();
```

```
$ nvcc main.cu -o main
```

GPU set up

- Our heterogeneous, parallel, programming model

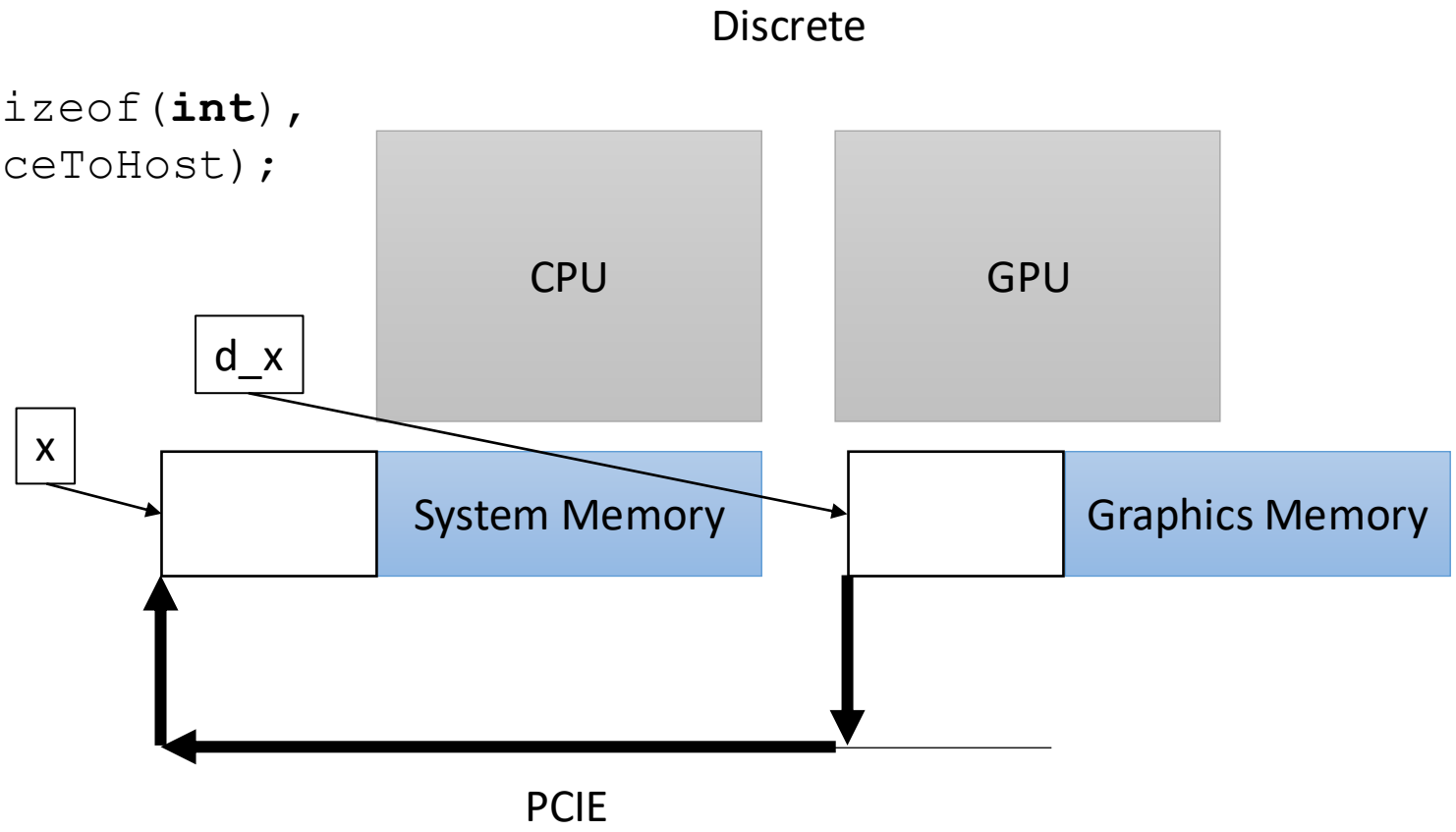
```
//initialize x on host
cudaMemcpy(d_x, x, SIZE*sizeof(int),
           cudaMemcpyHostToDevice);
```



GPU set up

- Our heterogeneous, parallel, programming model

```
//initialize x on host
cudaMemcpy(x, d_x, SIZE*sizeof(int),
           cudaMemcpyDeviceToHost);
```



The GPU Program

- The operating system expects the GPU kernel to be fast to render graphics.
- It kills the kernel if it takes more than ~5 seconds.
- If GPU is used for both display and CUDA.

The GPU Program



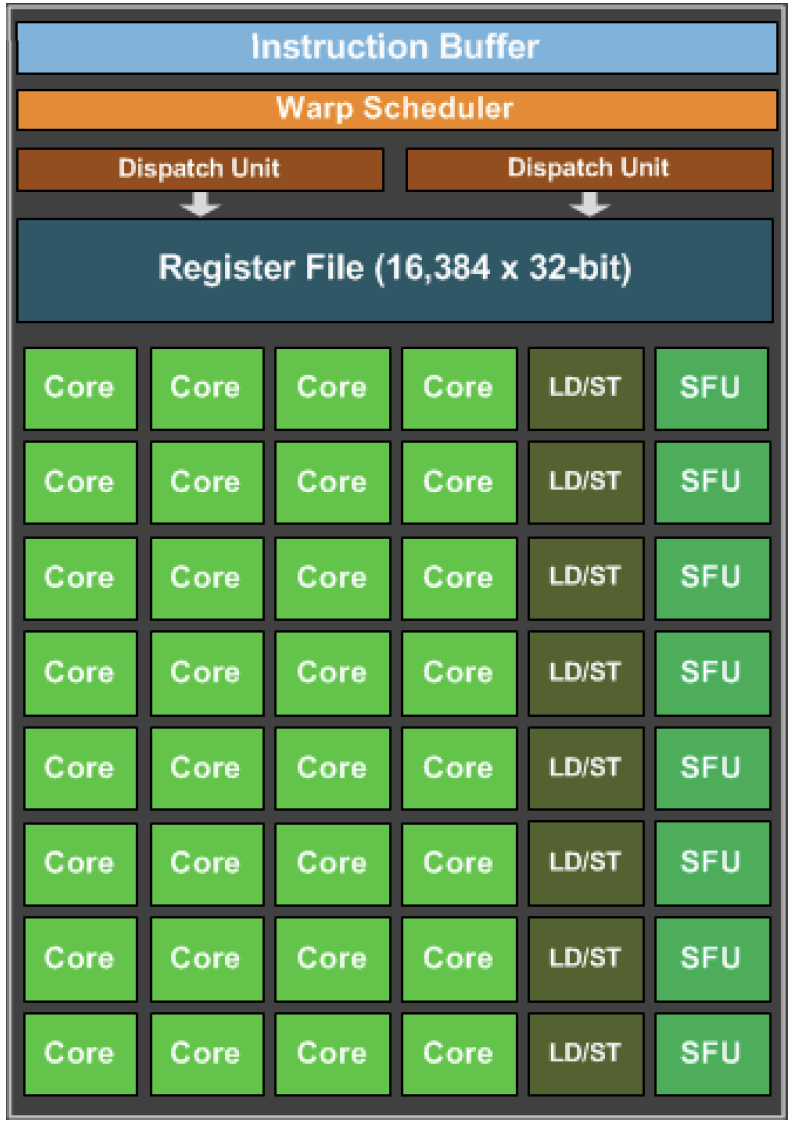
1000x slower on GPU?
It didn’t do so well...

One GPU core is much slower than a single CPU core

First parallelization attempt

- Lets look at some GPU documentation.

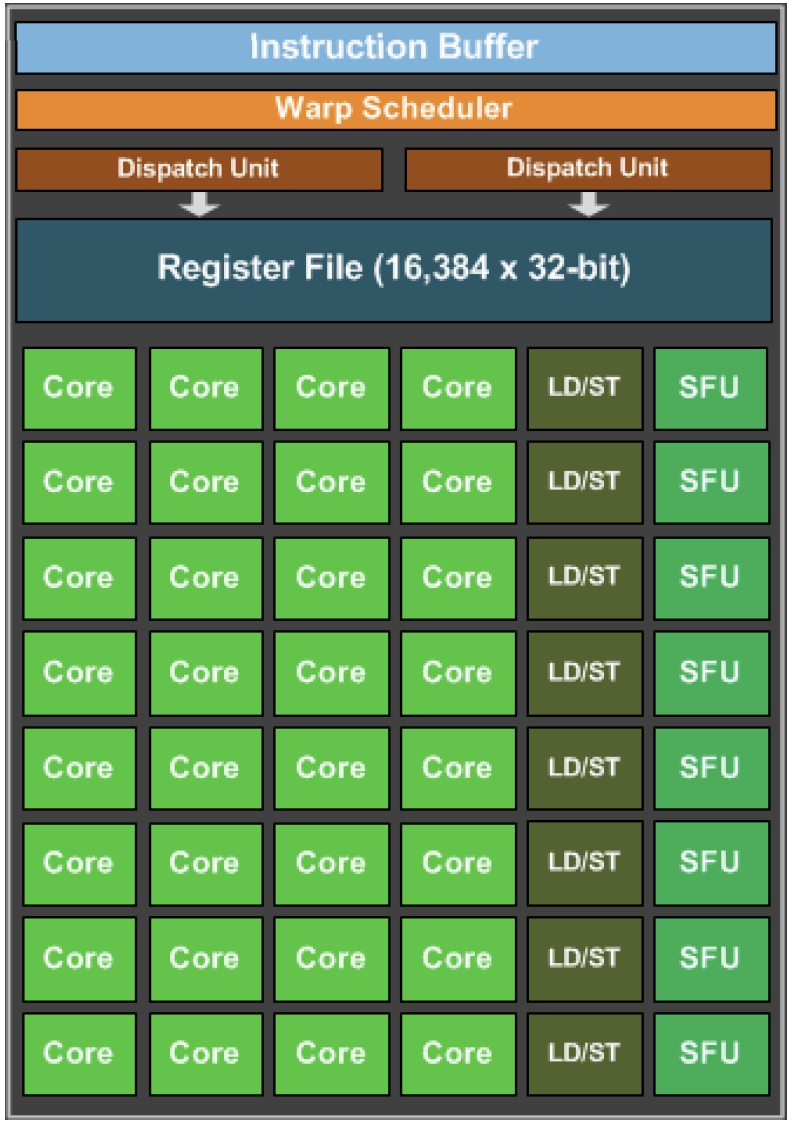
- The Maxwell whitepaper shows a diagram of one of the GPU cores



https://www.techpowerup.com/gpu-specs/docs/nvidia-gtx-980.pdf

woah, 32 cores!

We should parallelize our application!



https://www.techpowerup.com/gpu-specs/docs/nvidia-gtx-980.pdf

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

number of threads to launch the program with

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + end;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

number of threads

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + end;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

number of threads
thread id

First parallelization attempt

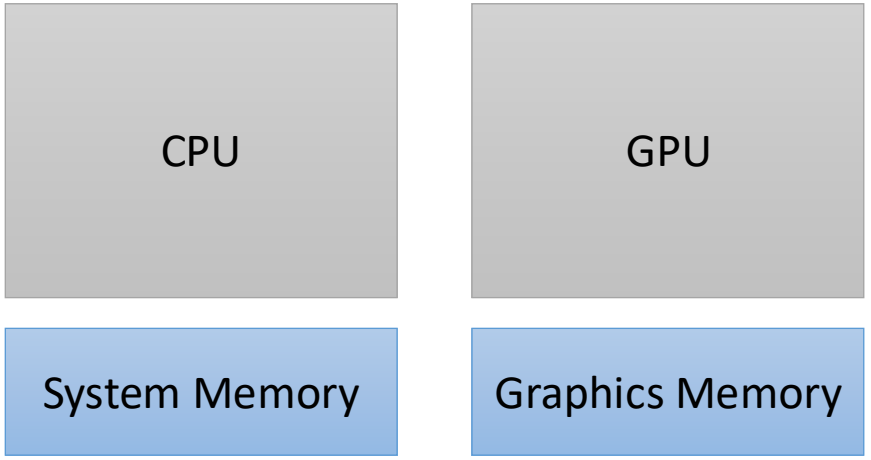
That gives us about 14x speedup!

First parallelization attempt



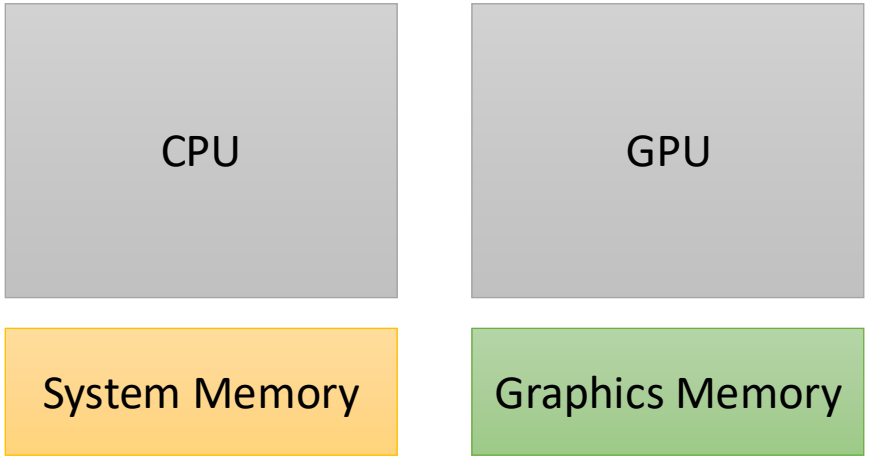
Getting better but we have a long ways to go!

GPU Memory



GPU Memory

CPU Memory:
Fast: Low Latency
Easily saturated: Low Bandwidth
Scales well: up to 1 TB
DDR



GPU Memory:
slow: High Latency
hard to saturate: High Bandwidth
doesn't scale: 32 GB
GDDR, HBM

2-lane straight highway driven on by sports cars

Different technologies

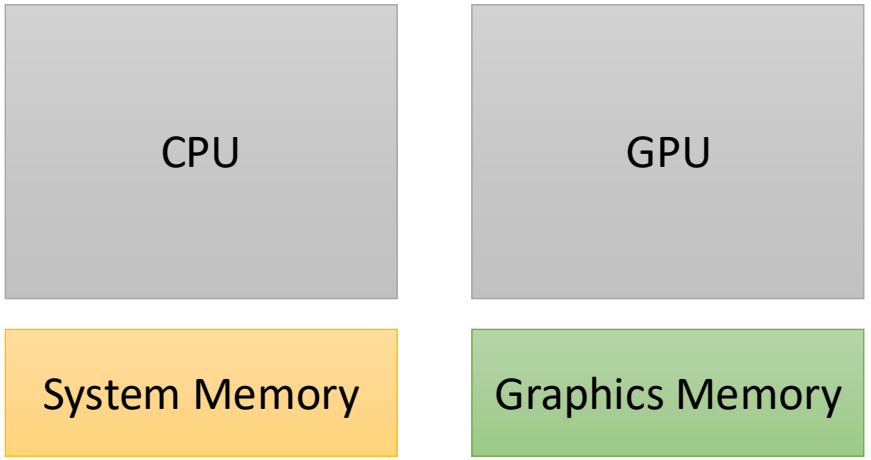
16-lane highway on a windy road driven by semi trucks

GPU Memory

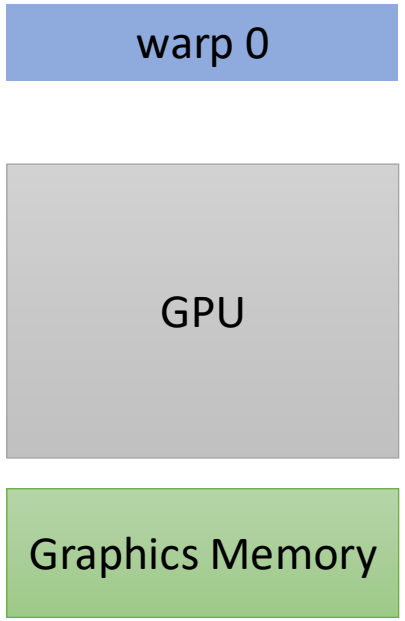
bandwidth:
~**700 GB/s** for GPU
~**50 GB/s** for CPUs

memory Latency:
~**600** cycles for GPU memory
~**200** cycles for CPU memory

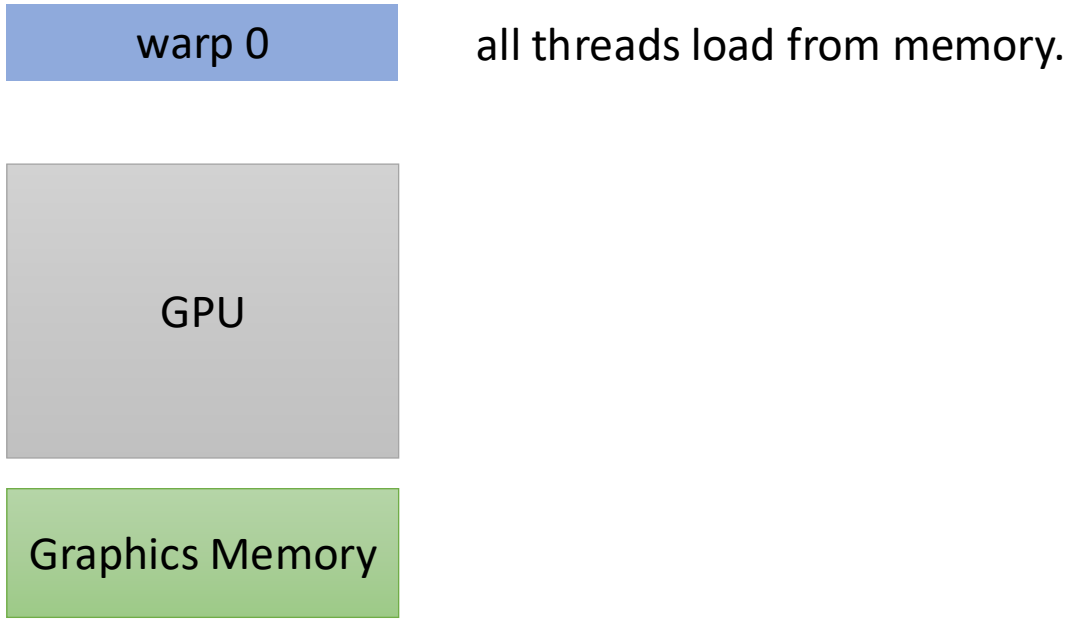
Cache Latency:
~**28** cycles for L1 hit for GPU
~**4** cycles for L1 hit on CPUs



Preemption and concurrency?

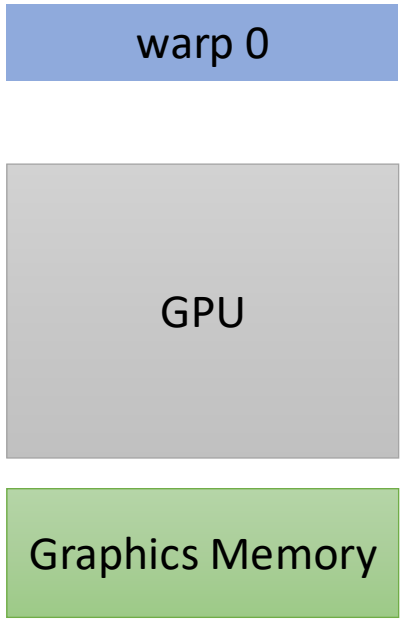


Preemption and concurrency?



all threads load from memory.

Preemption and concurrency?



all threads load from memory.

600 cycles!

Preemption and concurrency?

warp 0

warp 1

warp 2

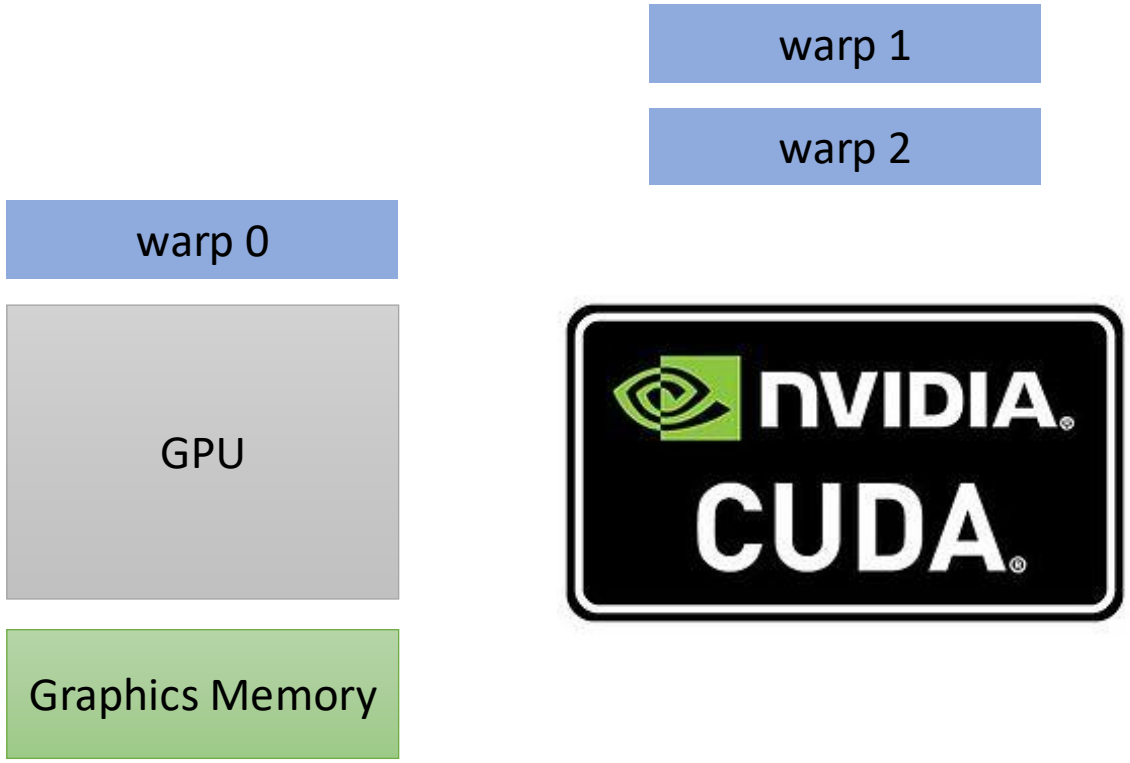
GPU

Graphics Memory



We can hide latency through preemption and concurrency!

Preemption and concurrency?



We can hide latency through preemption and concurrency!

Preemption and concurrency?

memory access
600 cycles

warp 1

warp 2

warp 0

GPU

Graphics Memory



We can hide latency through preemption and concurrency!

Preemption and concurrency?

memory access
600 cycles

warp 1

warp 2

warp 0

GPU

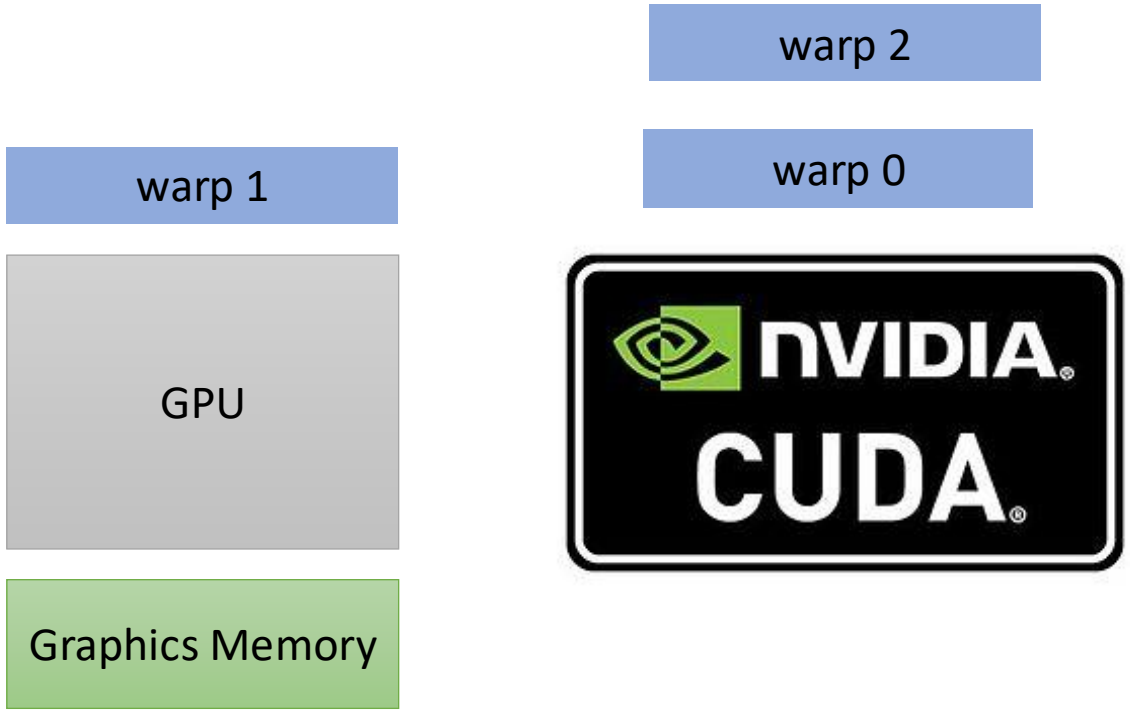
Graphics Memory



preempt warp 0
and put warp 1 on

We can hide latency through preemption and concurrency!

Preemption and concurrency?



We can hide latency through preemption and concurrency!

Preemption and concurrency?

memory access
600 cycles

warp 1

GPU

Graphics Memory

warp 2

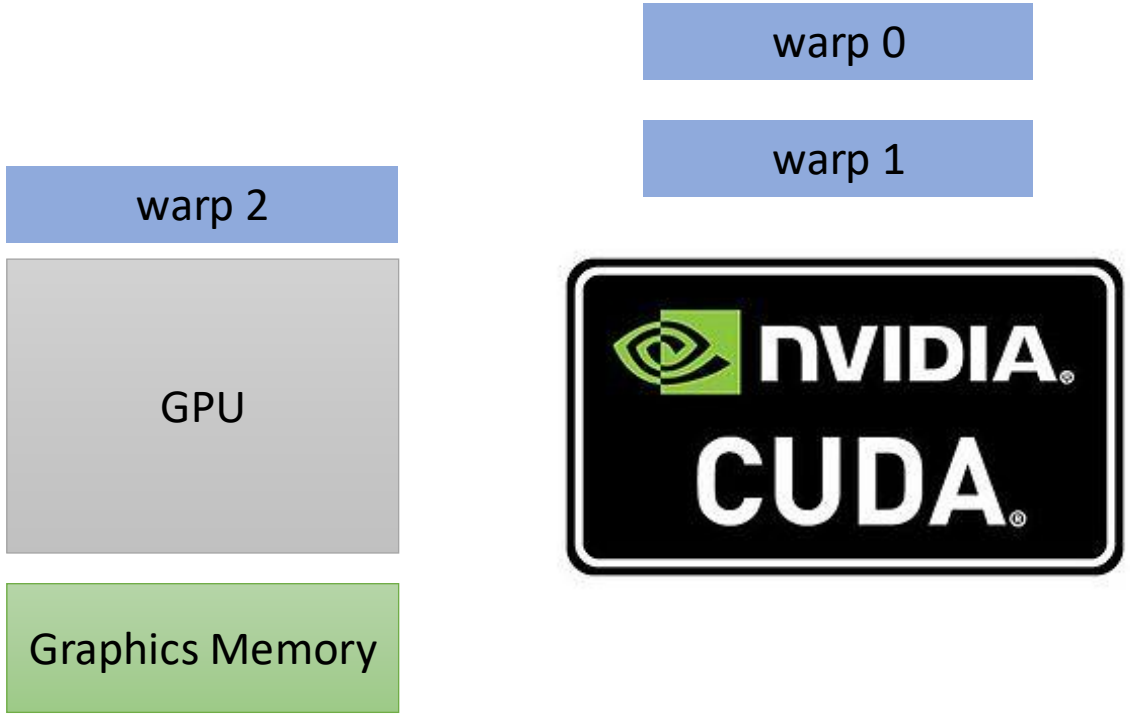
warp 0



preempt warp 1
and put warp 2 on

We can hide latency through preemption and concurrency!

Preemption and concurrency?



We can hide latency through preemption and concurrency!

Preemption and concurrency?

memory access
600 cycles

warp 2

GPU

Graphics Memory

warp 0

warp 1

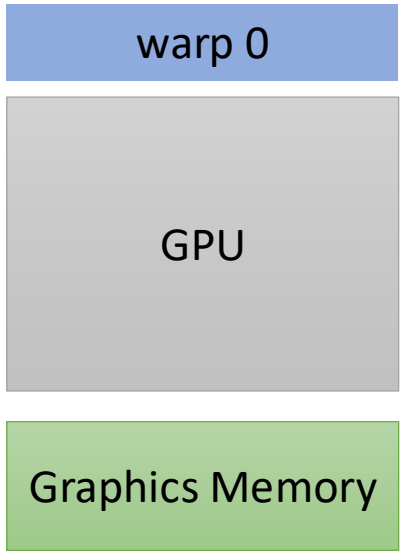


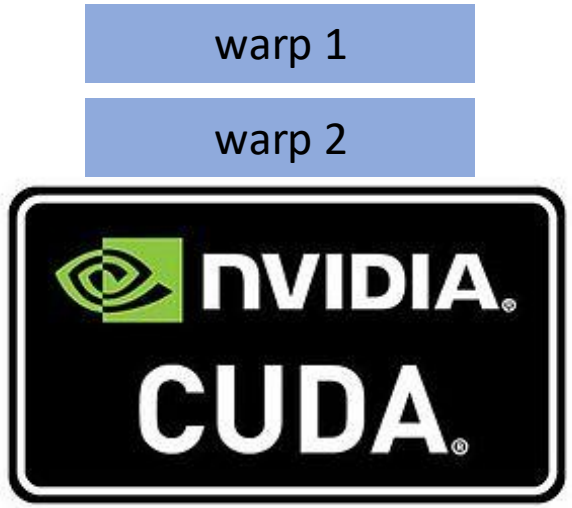
preempt warp 2
and put warp 0 on

We can hide latency through preemption and concurrency!

Preemption and concurrency?

Hey, my memory has arrived!





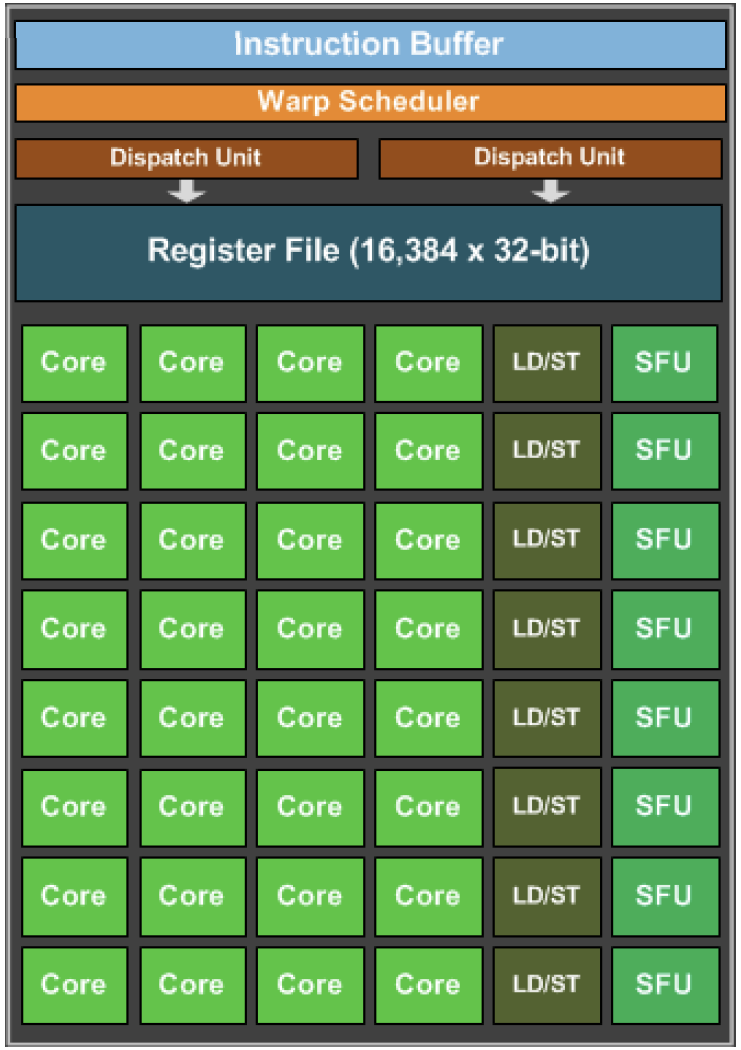
preempt warp 2
and put warp 0 on

We can hide latency through preemption and concurrency!

Preemption and concurrency?

But wait, I thought preemption was expensive?

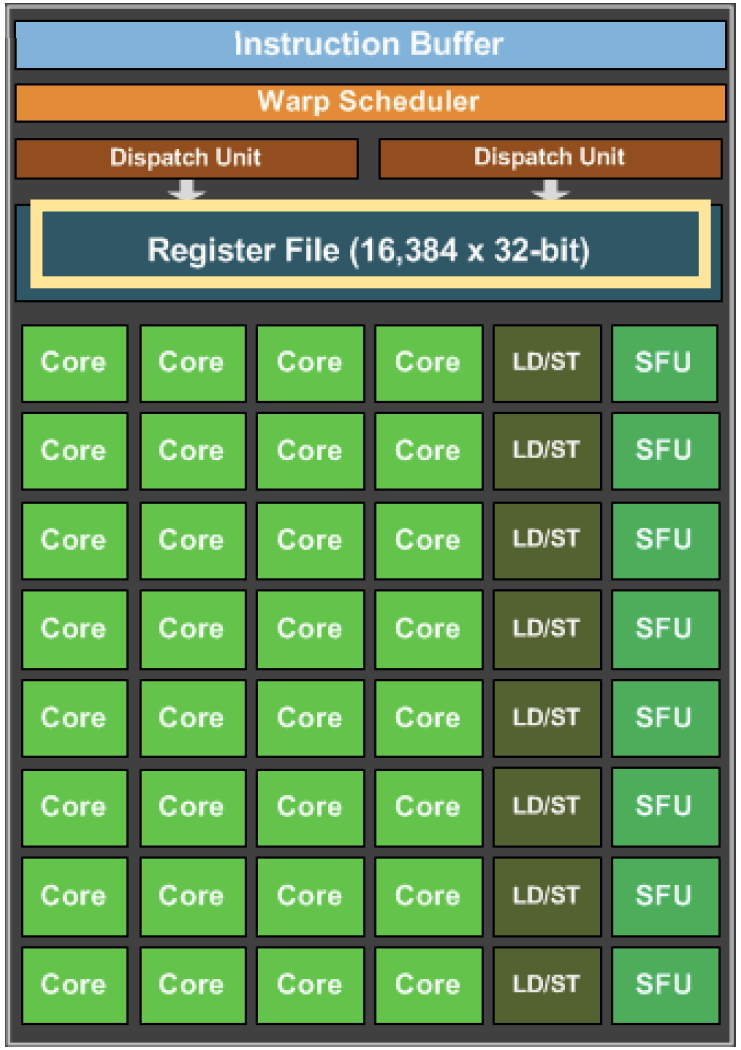
Preemption and concurrency?



But wait, I thought preemption was expensive?

bound on number of warps: 32

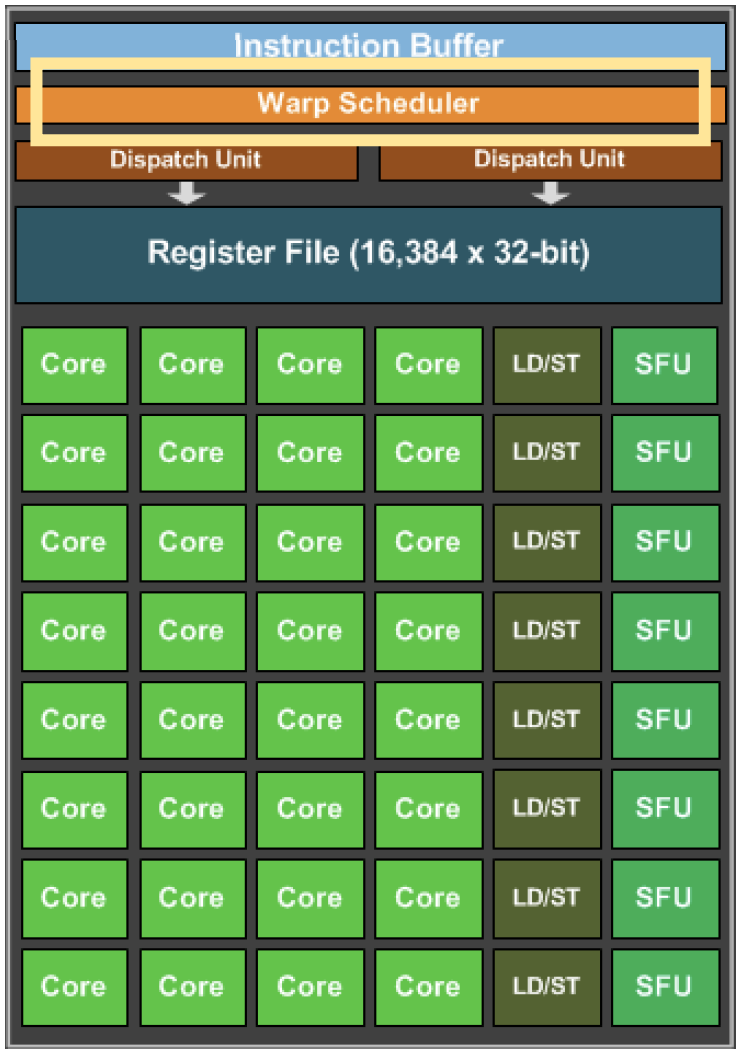
Preemption and concurrency?



But wait, I thought preemption was expensive?

Registers all stay on chip. Large register file.

Preemption and concurrency?



But wait, I thought preemption was expensive?

dedicated scheduler logic

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + end;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

Lets launch with 32 warps

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + end;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

Lets launch with 32 warps

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

Concurrent warps

This is about 2x faster.

Concurrent warps

This is about 2x faster.



Getting better!